



Java Module System

Building Maintainable and Scalable Java Applications

Albert Attard

Principal Java Success Consultant

Java Success Consulting Team

September 2026



Who am I?

Who am I?



- Albert Attard
- Born in Malta (**a long time ago**)
- Passionate Java Developer
- Moved to Köln, Germany, in Oct. 2018
- Joined Oracle in July 2022, as part of the Java Success Consultancy team

Who am I?

Where is Malta?



What do you expect to gain from this session about Java Module System?

Objectives

- Demonstrates the shortcomings of traditional approach (the class path)
- Describes the Java Module System in detail
- Demonstrates how the Java Module System can be adopted

Mastery takes about 10,000 hours...

Good news: We're not doing all that today.

You are not expected to become an expert after this workshop

but rather to simply identify gaps in your knowledge and begin a conversation

*“If there’s something strange, in your **Java**
code, who you gonna call?”*

—
Java Success Team!

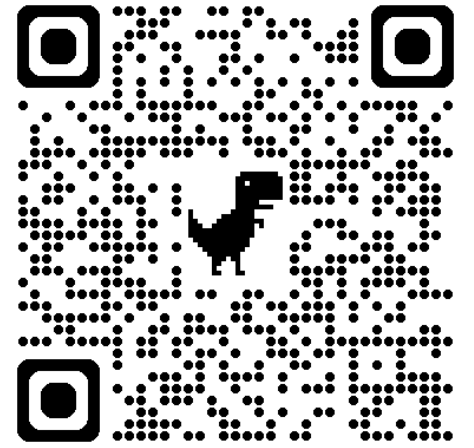


References

Updated for Java 11

The Java Module System

Nicolai Parlog
Foreword by Kevlin Henney

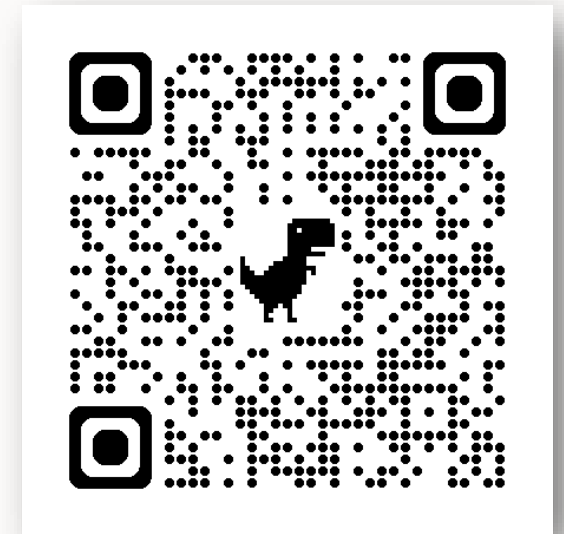
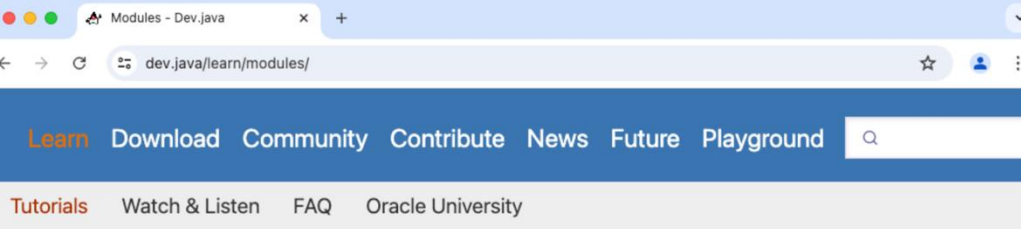


The Java Module System
ISBN 9781617294280

Nicolai Parlog

<https://www.manning.com/books/the-java-module-system>





dev.java

<https://dev.java/learn/modules/>

1. [Introduction to Modules in Java](#)
Understand the module system basics, how to create and build modules, and how to increase maintainability and encapsulation.
2. [Reflective Access with Open Modules and Open Packages](#)
Use open packages and open modules to allow reflective access to otherwise encapsulated packages.
3. [Optional Dependencies with `requires static`](#)
Use `requires static` for optional dependencies - modules required this way are accessible at compile time but can be absent at run time.
4. [Implied Readability with `requires transitive`](#)
Use `requires transitive` to imply readability, where a module passes its dependency on another module on, allowing a other modules to read it without explicitly depending on it.
5. [Qualified exports and opens](#)
Use `exports ... to ...` and `opens ... to ...` to limit accessibility of exported or opened packages to specific modules.
6. [Decoupling Modules with Services](#)
Decouple users and providers of a service with Java's ServiceLoader API, which the module system makes a first-class concept with `uses` and `provides` directives in the module declaration.
7. [Code on the Class Path - the Unnamed Module](#)
All JARs on the class path, modular or not, become part of the unnamed module. This makes 'everything a module', while the chaos of the class path can live on.
8. [Incremental Modularization with Automatic Modules](#)
Plain JARs on the module path become automatic modules, where they can act as a bridge from modular JARs to the class path.
9. [Building Modules on the Command Line](#)
Learn how to use the `javac`, `jar`, and `java` commands to compile, package, and launch your modular application by hand - good to know even though build tools do most of the heavy lifting.
10. [Strong Encapsulation \(of JDK Internals\)](#)
Strong encapsulation is a corner stone of the module system. It avoids (accidental) use of internal APIs,

15.09.2026



The class path

The class path

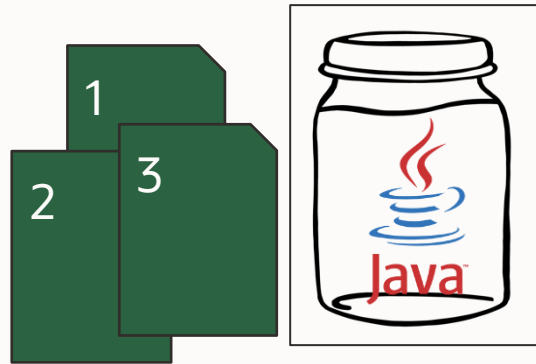
```
> java \  
  --class-path app.jar:domain.jar \  
  demo.Main
```

The class `demo.Main` is the entry point and can be in any of these JAR files

Two JAR files, `app.jar` and `domain.jar`, are added to the class path

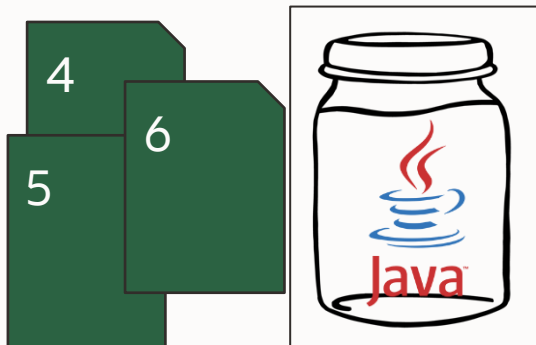
The class path

Class loading



Class 4 is required

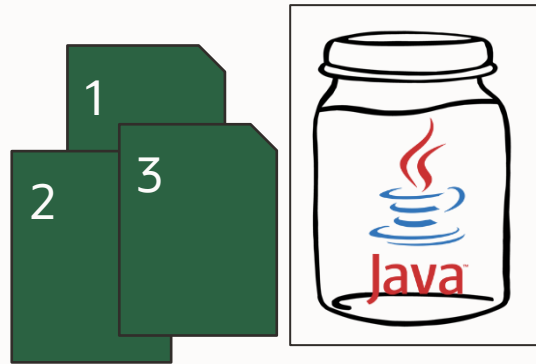
Search for Class 4 here
Not found



Search for Class 4 here
Found
Load class

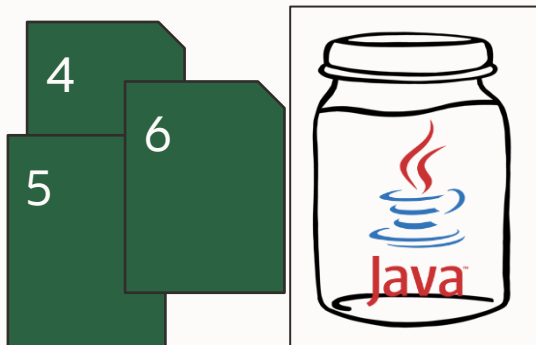
The class path

Class loading



Class 7 is required

Search for Class 7 here
Not found



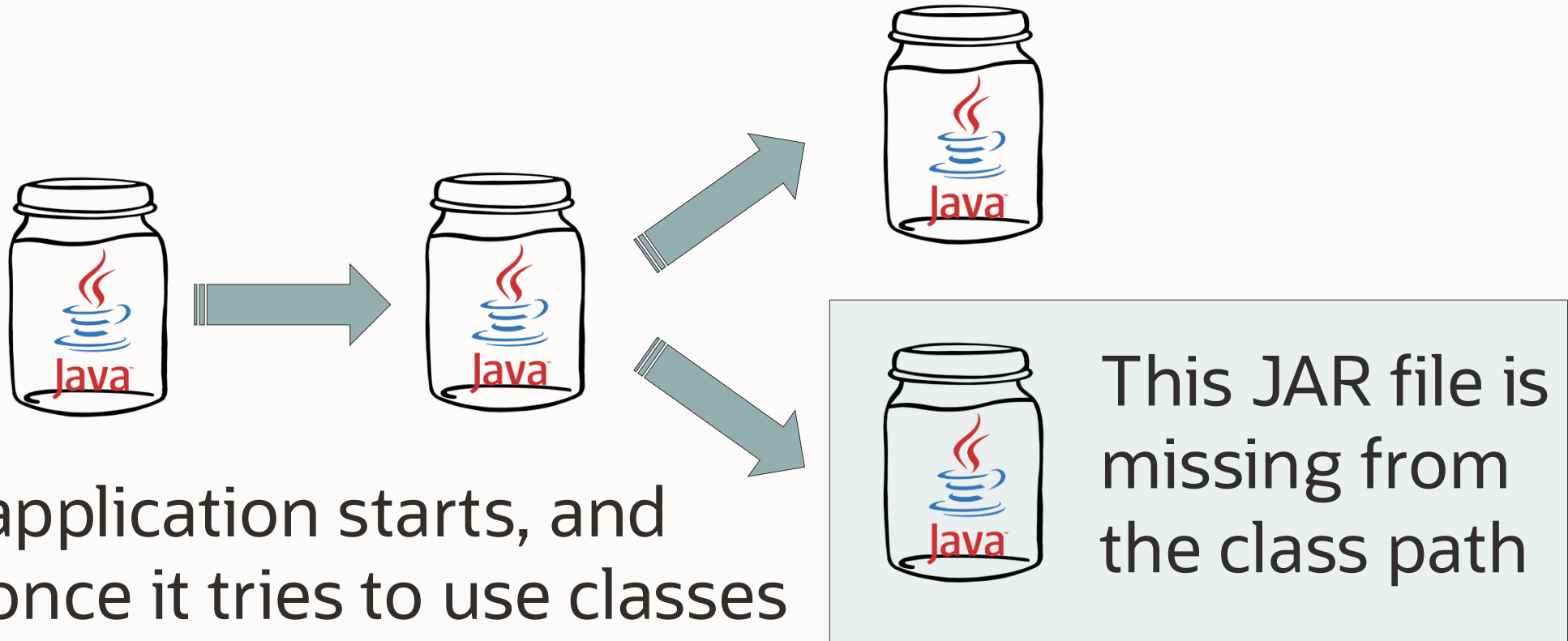
Search for Class 7 here
Not found

Throws `ClassNotFoundException`

What happens if one of the dependencies is missing?

—
Missing dependency

Missing dependency



The application starts, and fails once it tries to use classes found in the missing JAR

Demo

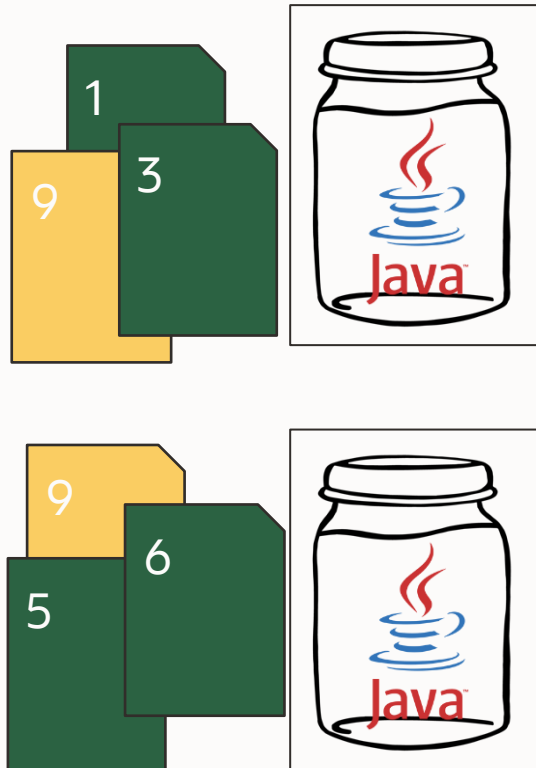
challenges-missing-dependency

What happens if a class with the same canonical name is found in more than one JAR file?

—
Split Package

The class path

Class found in two JAR files



Class 9 is required

Search for Class 9 here
Found
Load class

Class with the same canonical name in two different JAR files

The screenshot shows an IDE with two tabs open, both editing a class named `Formatter` in the `demo` package. The left tab is `app/.../Formatter.java` and the right tab is `domain/.../Formatter.java`. Both files have the same canonical name, `demo.Formatter`, which would cause a conflict if both JAR files were present in the same classpath.

Left Tab: `app/.../Formatter.java`

```
1 package demo;
2
3 import static java.util.Objects.requireNonNull;
4
5 public final class Formatter {
6
7     public static String multiline(final Quote quote)
8         requireNonNull(quote);
9
10    return ""
11        ~~~~~~
12        "%s"
13        ~~~~~~
14        by %s
15        (formatted by the app formatter)""
16        .formatted(quote.quote(), quote.author
17    }
18
19    private Formatter() {}
20
21 }
```

Right Tab: `domain/.../Formatter.java`

```
1 package demo;
2
3 import static java.util.Objects.requireNonNull;
4
5 public final class Formatter {
6
7     public static String multiline(final Quote quote)
8         requireNonNull(quote);
9
10    return ""
11        "%s"
12        by %s
13        (formatted by the domain formatter)""
14        .formatted(quote.quote(), quote.author
15    }
16
17    private Formatter() {}
18
19 }
```

Which version will be used?

```
1 package demo;
2
3 public record Quote(String author, String quote) {
4     @Override
5     public String toString() {
6         return Formatter.multiLine(quote: this);
7     }
8 }
9
```

Which demo.Formatter will it uses?

Running the application

```
> java \  
  --class-path app.jar:domain.jar \  
  demo.Main
```

The one in the `app.jar`
is loaded and used

Running the application

```
> java \  
  --class-path domain.jar:app.jar \  
  demo.Main
```

The one in the `domain.jar`
is loaded and used

Running the application

```
> java \  
  --class-path './libs/*' \  
  demo.Main
```

The order in which this is expanded is **unspecified**

Demo

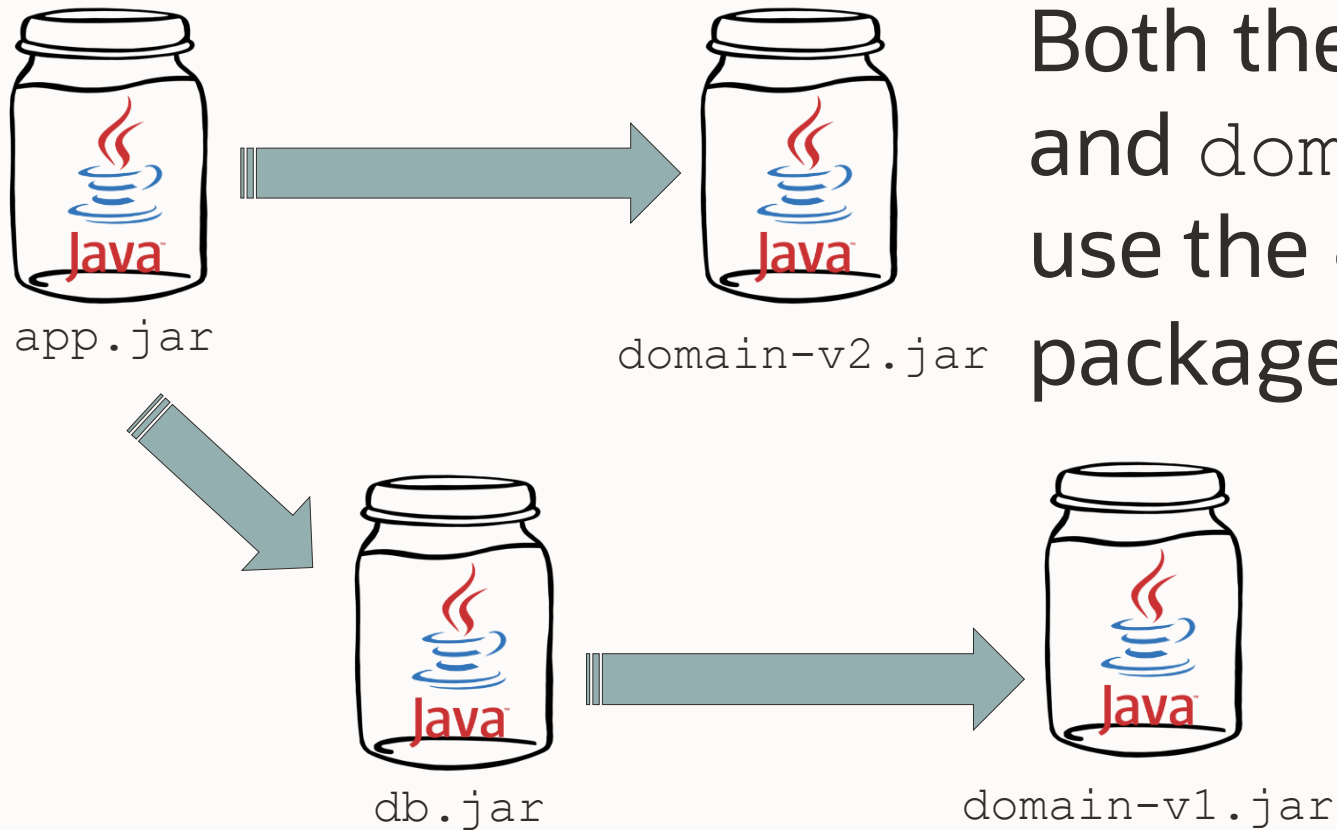
challenges-split-package-duplicate-
class

What happens when multiple versions of the same library are found in the class path?



Split Package

Dependencies



Both the `domain-v1.jar` and `domain-v2.jar` files use the `demo.domain` package

New method

The screenshot shows an IDE with two tabs open: `domain-v1/.../Formatter.java` and `domain-v2/.../Formatter.java`. The left sidebar shows a project structure for `challenges-split-package-multiple-versions` with sub-projects `domain-v1` and `domain-v2`. The `domain-v1` tab shows the following code:

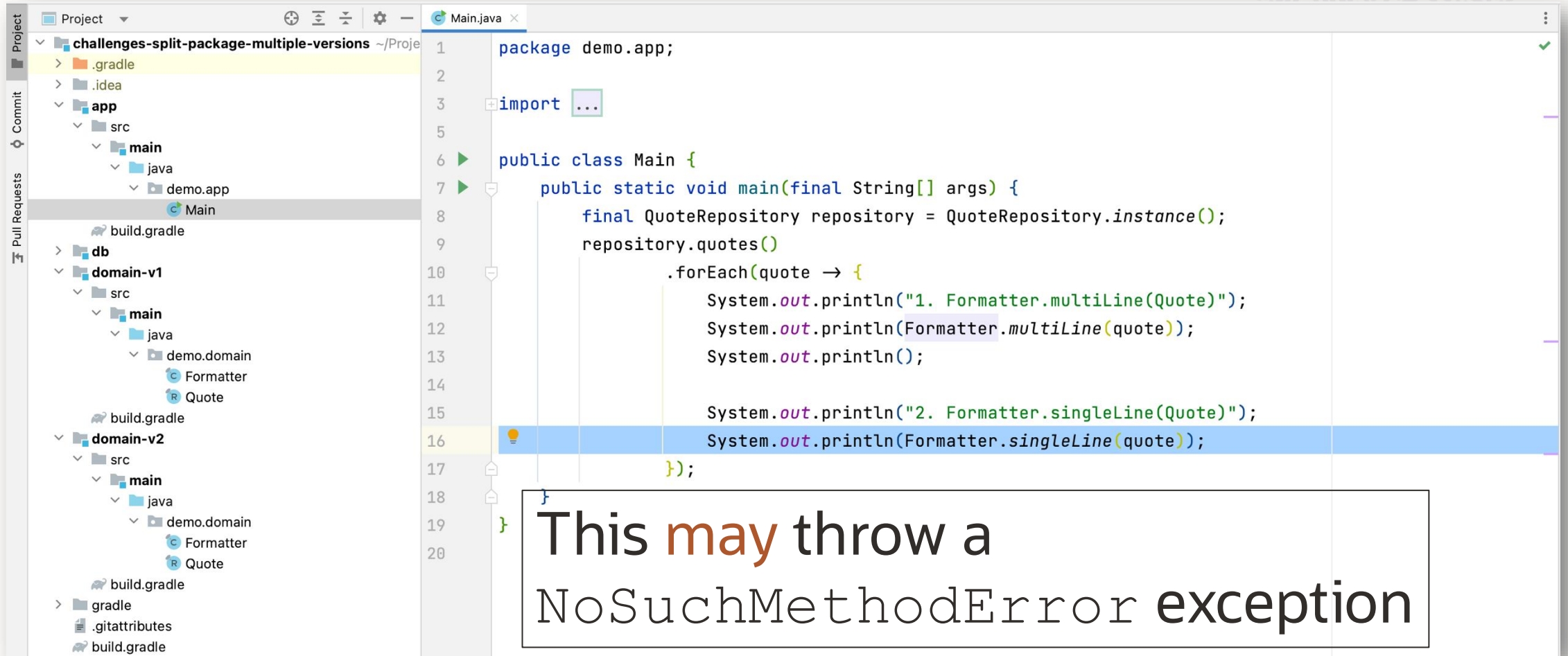
```
1 package demo.domain;
2
3 import static java.util.Objects.requireNonNull;
4
5 public final class Formatter {
6
7     public static String multiLine(final Quote
15
16     private Formatter() {}
17 }
18
```

The `domain-v2` tab shows the same code but with a new method added at line 16:

```
1 package demo.domain;
2
3 import static java.util.Objects.requireNonNull;
4
5 public final class Formatter {
6
7     public static String multiLine(final Quote
15
16     public static String singleLine(final Quote
22
23     private Formatter() {}
24 }
25
```

A text box on the right side of the IDE contains the text: "A new method was added to the new version".

Usage



The screenshot shows an IDE window with a project named "challenges-split-package-multiple-versions". The project structure on the left includes a "demo.app" module with a "Main" class. The "Main.java" file is open in the editor, showing the following code:

```
1 package demo.app;
2
3 import ...
4
5
6 public class Main {
7     public static void main(final String[] args) {
8         final QuoteRepository repository = QuoteRepository.instance();
9         repository.quotes()
10             .forEach(quote -> {
11                 System.out.println("1. Formatter.multiLine(Quote)");
12                 System.out.println(Formatter.multiLine(quote));
13                 System.out.println();
14
15                 System.out.println("2. Formatter.singleLine(Quote)");
16                 System.out.println(Formatter.singleLine(quote));
17             });
18     }
19 }
20
```

Line 16 is highlighted in blue, and a light blue tooltip box is overlaid on the code, containing the text:

This may throw a
NoSuchMethodError exception

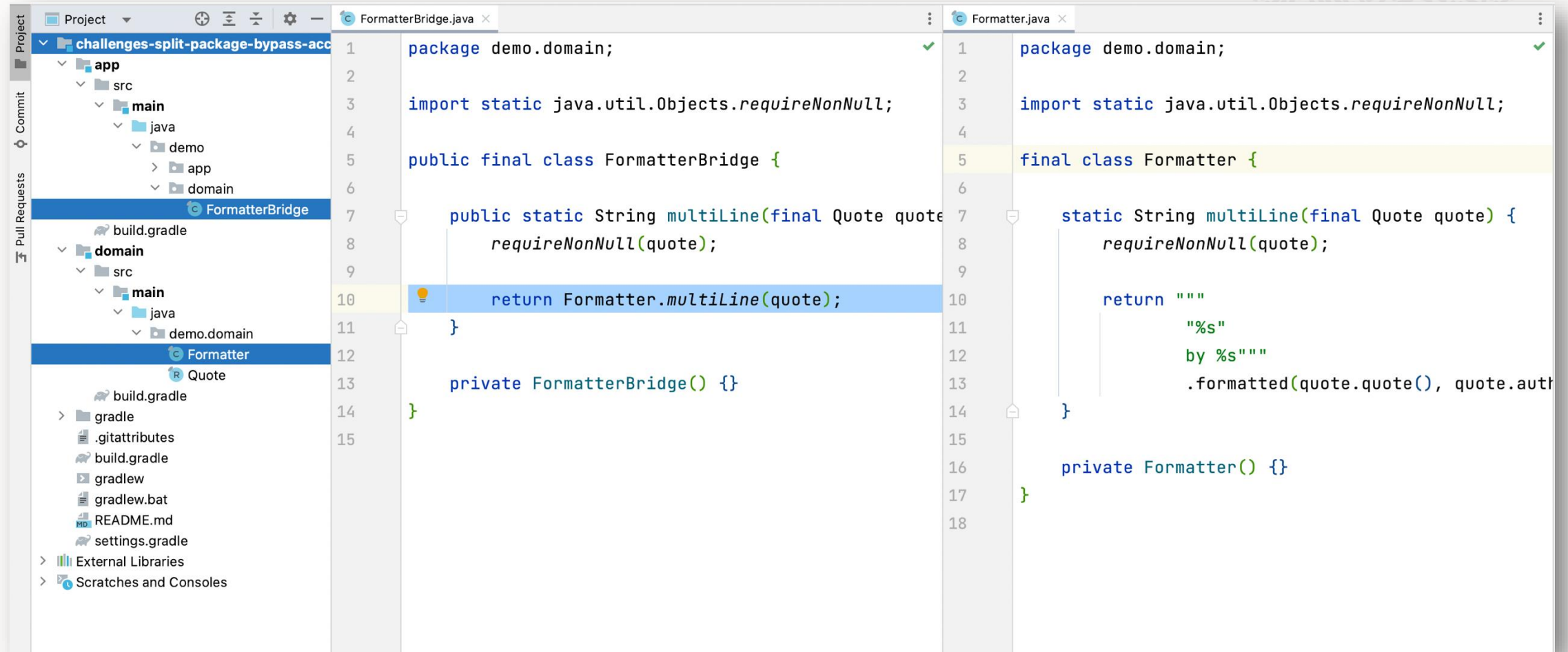
Demo

challenges-split-package-multiple-
versions

Can access control be bypass?

Split Package

Creating a bridge class



The screenshot shows an IDE with two open files: `FormatterBridge.java` and `Formatter.java`. The left sidebar displays the project structure for `challenges-split-package-bypass-acc`, showing a package hierarchy with `app` and `domain` packages. The `FormatterBridge` class is highlighted in the `app` package, and the `Formatter` class is highlighted in the `domain` package.

FormatterBridge.java

```
1 package demo.domain;
2
3 import static java.util.Objects.requireNonNull;
4
5 public final class FormatterBridge {
6
7     public static String multiLine(final Quote quote) {
8         requireNonNull(quote);
9
10        return Formatter.multiLine(quote);
11    }
12
13    private FormatterBridge() {}
14
15 }
```

Formatter.java

```
1 package demo.domain;
2
3 import static java.util.Objects.requireNonNull;
4
5 final class Formatter {
6
7     static String multiLine(final Quote quote) {
8         requireNonNull(quote);
9
10        return ""
11            .formatted(quote.quote(), quote.author())
12            .by "%s""
13            .formatted(quote.quote(), quote.author());
14    }
15
16    private Formatter() {}
17
18 }
```

Bypass access control

```
1 package demo.app;
2
3 import ...
4
5
6 public class Main {
7     public static void main(final String[] args) {
8         final Quote quote = new Quote( author: "Albert Einstein", quote: "Learn from yesterday, live for today, hope for tomorrow");
9         System.out.println(FormatterBridge.multiLine(quote));
10    }
11 }
12
```

The `demo.domain.Formatter` class is accessible from anywhere through the `demo.domain.FormatterBridge` class

Demo

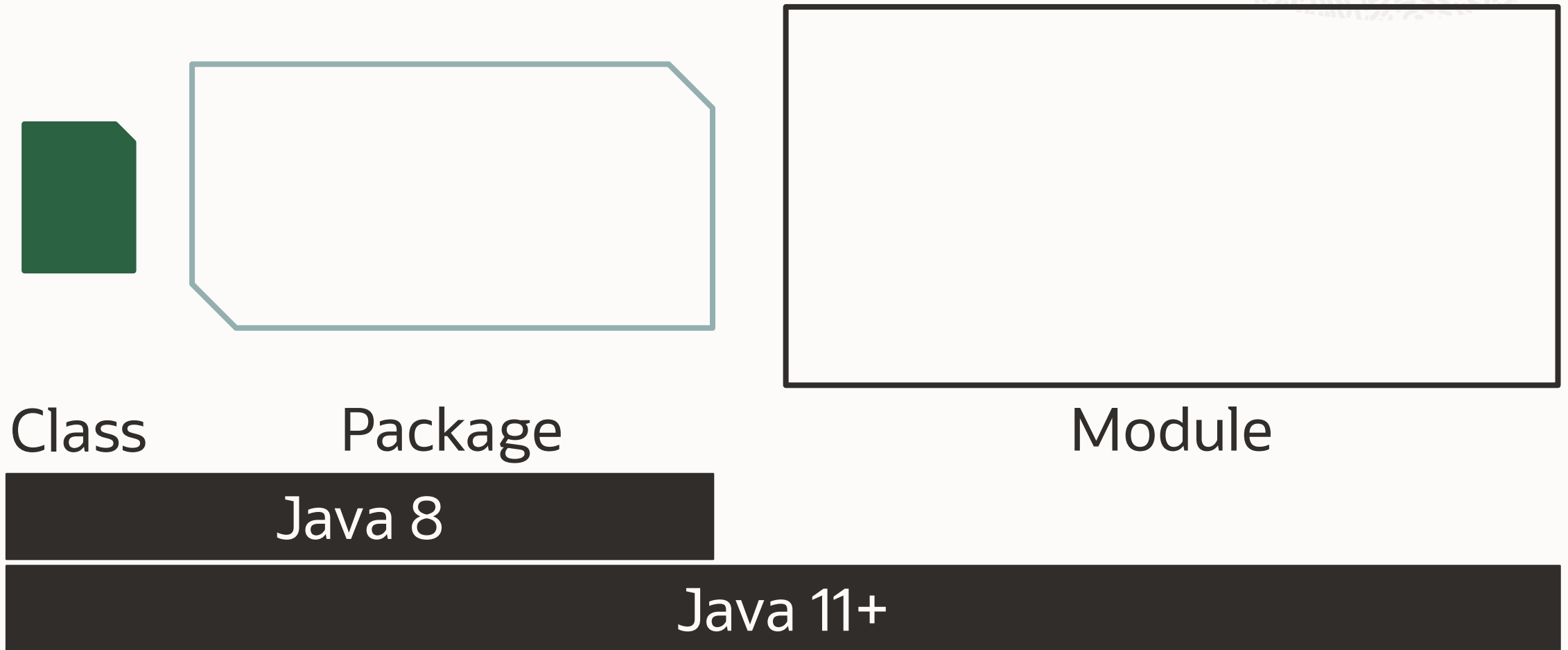
challenges-split-package-bypass-
access-control



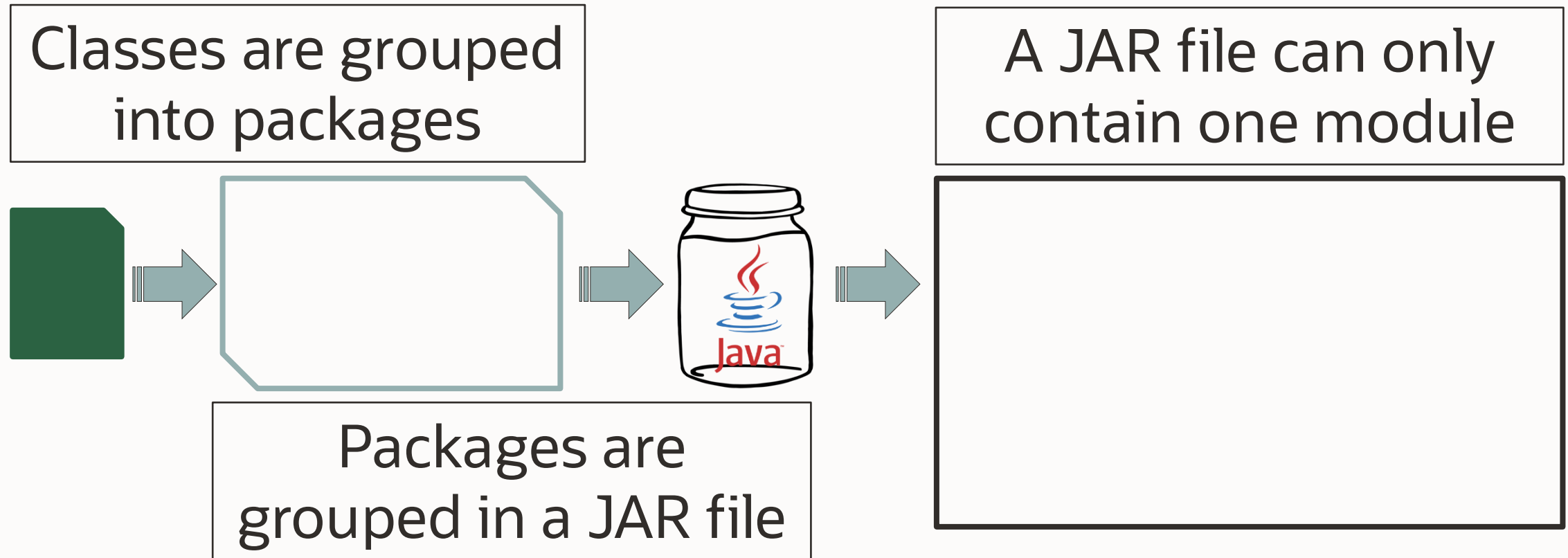
The Java Module System

In a nutshell

Anatomy

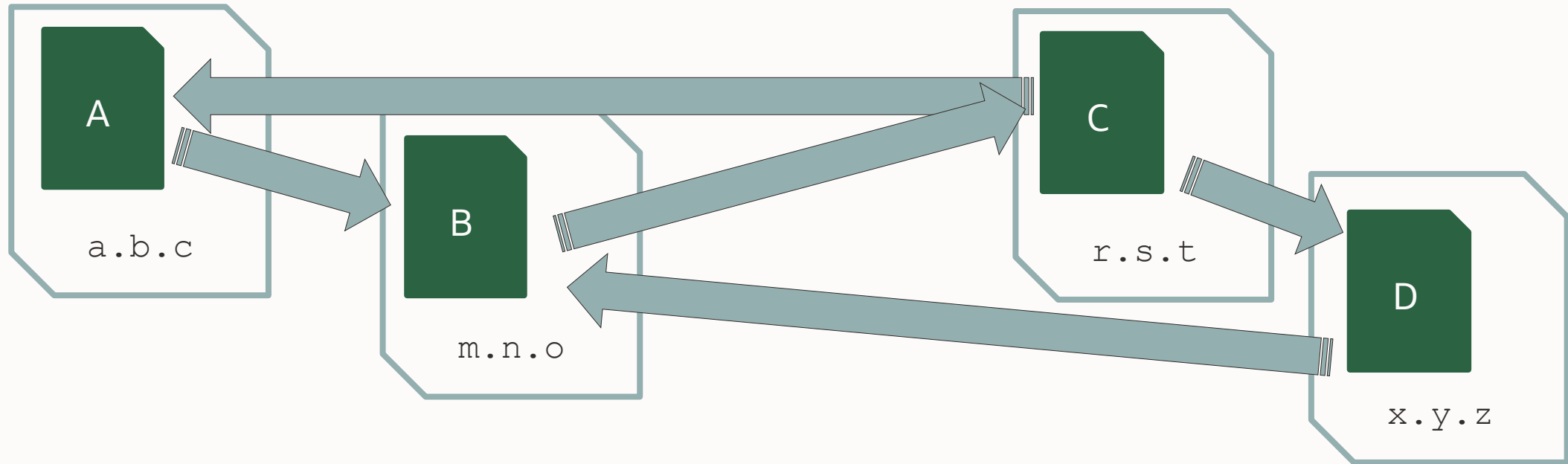


Anatomy



Free range

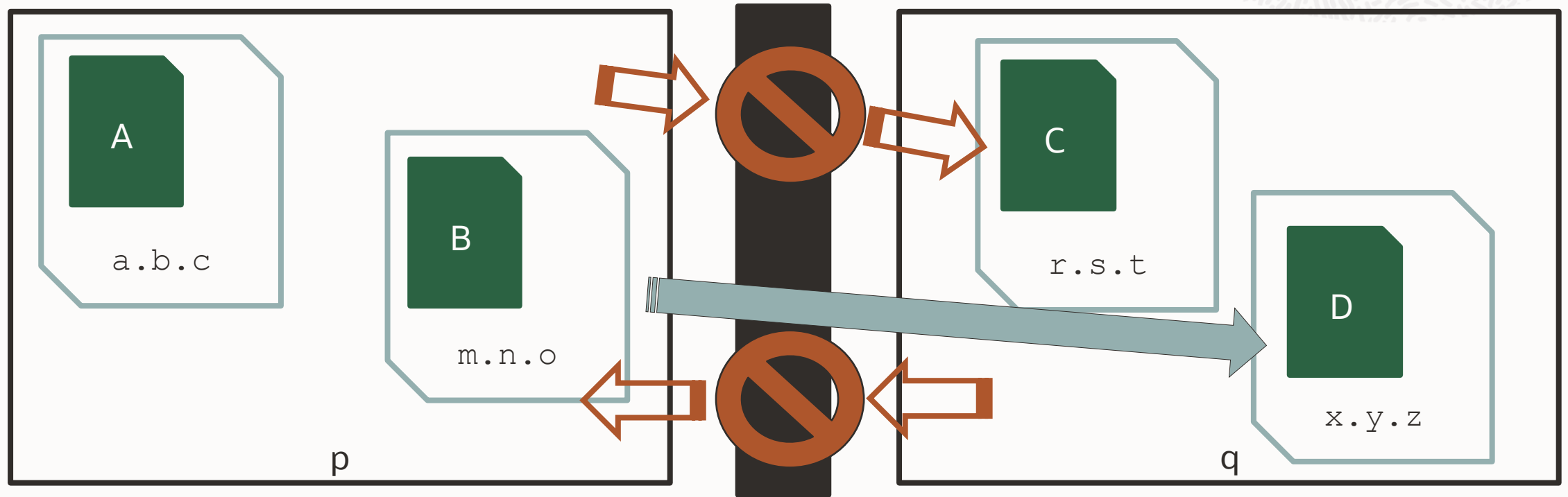
The class path



Any class in any package can access any other class in any other package

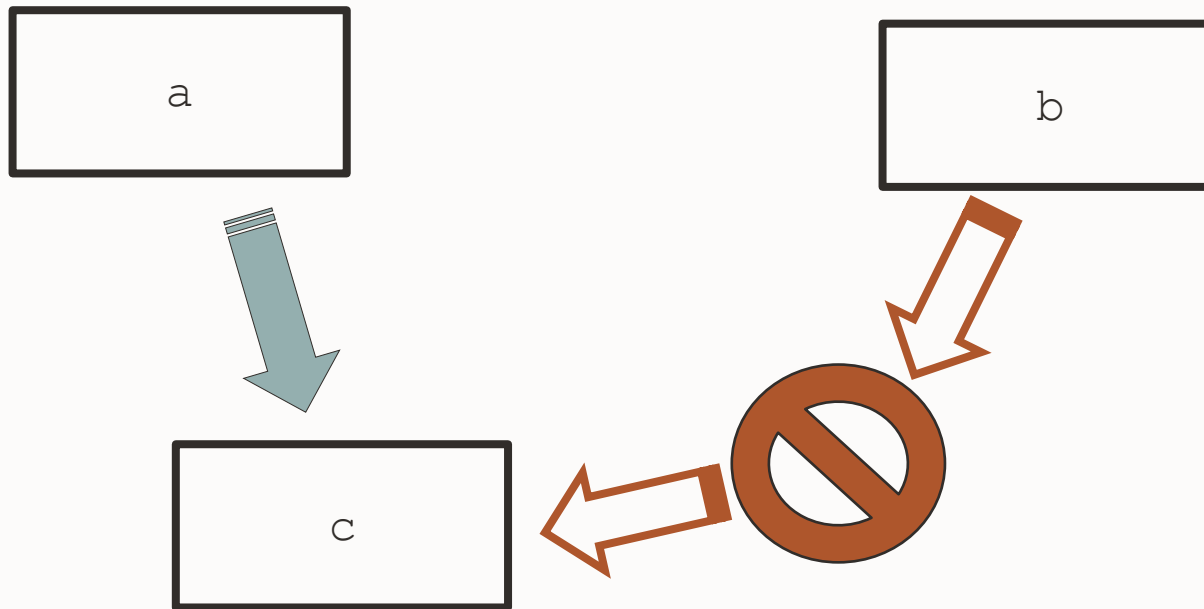
Stronger encapsulation

The module path



The Java runtime controls access between classes from different modules, acting like a firewall.

Access control



Configuration

module-info.java

```
module c {  
    exports x.y.z to a;  
}
```

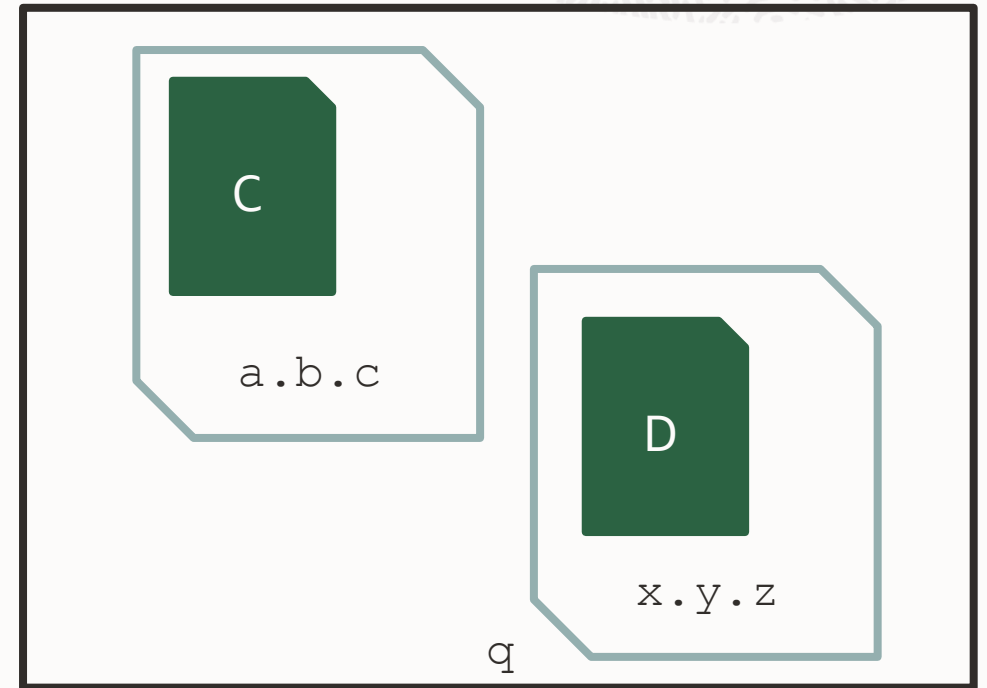
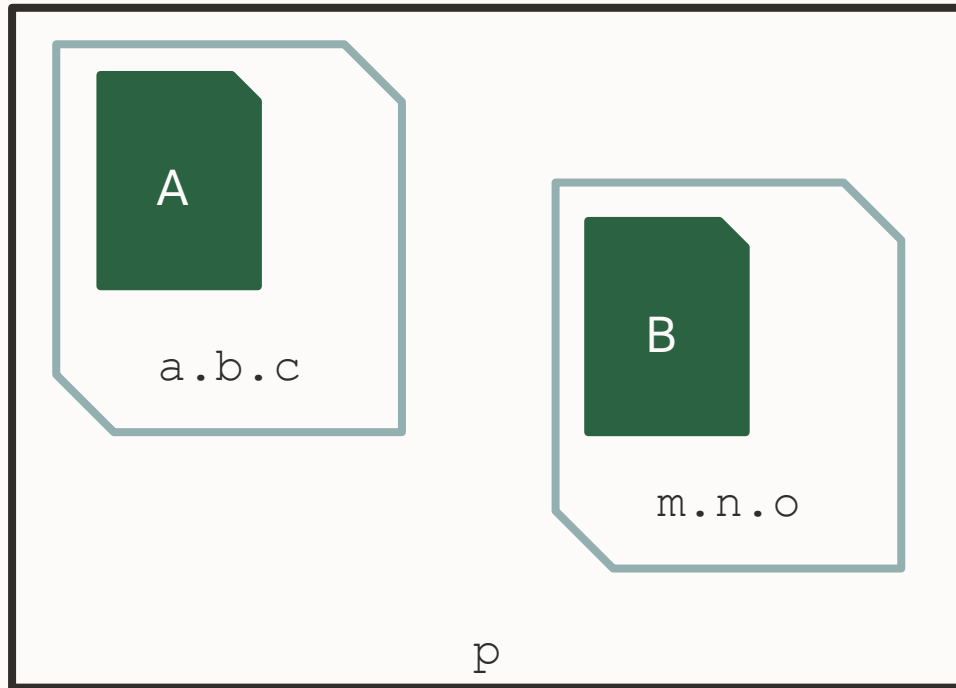
Command Line Arguments

```
> java \  
    --add-exports c/x.y.z=a \  
    ...
```

Module `c` only allows code in module `a` to access package `x.y.z`

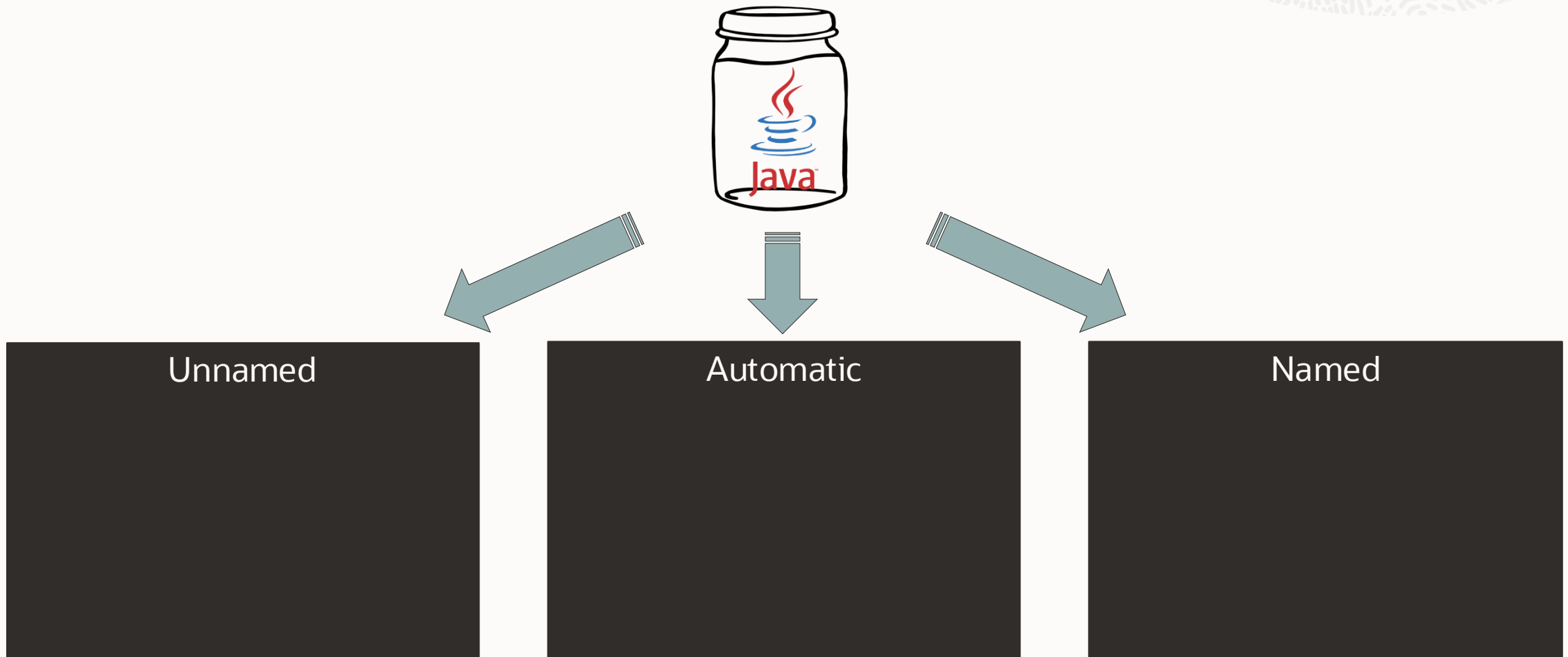
Split Package

The module path



The application will **fail** to start as two modules share the same package, package `a.b.c`

Types of Modules



What is the difference between the types of modules?

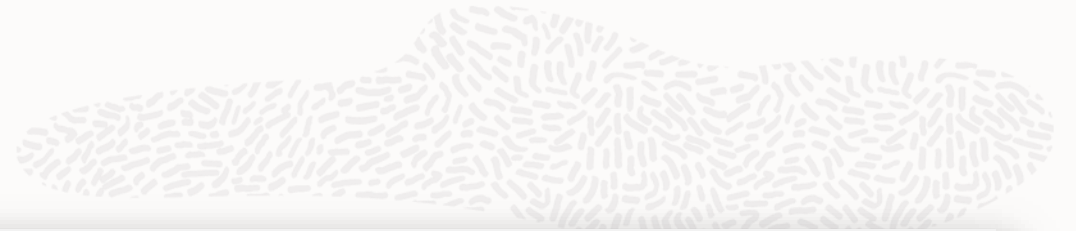
We will come back to this question later on, when discussing migration

Hello World

—
No modules

Hello World

Without Modules



The screenshot shows an IDE window with a project named "simple-hello-world-application-without-modules". The project structure in the left sidebar is as follows:

- Project
 - gradle
 - src
 - main
 - java
 - demo
 - Main (selected)
- .gitattributes
- build.gradle
- gradlew
- gradlew.bat
- README.md
- settings.gradle
- External Libraries
- Scratches and Consoles

The main editor displays the code for `Main.java`:

```
1 package demo;  
2  
3 public class Main {  
4     public static void main(final String[] args) {  
5         System.out.println("Hello world");  
6     }  
7 }  
8
```

Running the application

Using the class path

```
> java \  
  --class-path app.jar \  
  demo.Main
```

Unnamed



Automatic

Named

Running the application

Using the module path

```
> java \  
  --module-path app.jar \  
  --module app
```

Unnamed

Automatic



Named

Why was it loaded as an *automatic* module?

List modules

```
> java --module-path libs --list-modules
```

```
java.base@25.0.1  
java.compiler@25.0.1  
java.datatransfer@25.0.1  
java.desktop@25.0.1  
java.instrument@25.0.1  
...  
app file:///.../libs/app.jar automatic
```

Describe module

```
> jar --describe-module --file app.jar
```

```
No module descriptor found. Derived  
automatic module.
```

```
app automatic  
requires java.base mandated  
contains demo  
main-class demo.Main
```

Demo

`simple-hello-world-application-
without-modules`

How was the *automatic* module name derived?

Deriving *automatic* module name

1. *MANIFEST.MF Automatic-Module-Name* if specified
2. Remove the file extension from the JAR name
3. Remove any version information from the end
4. Replace any remaining characters other than letters and numbers with dots
5. Replace any sequences of dots with a single dot
6. Remove the dot if it is the first or last character of the result

Example

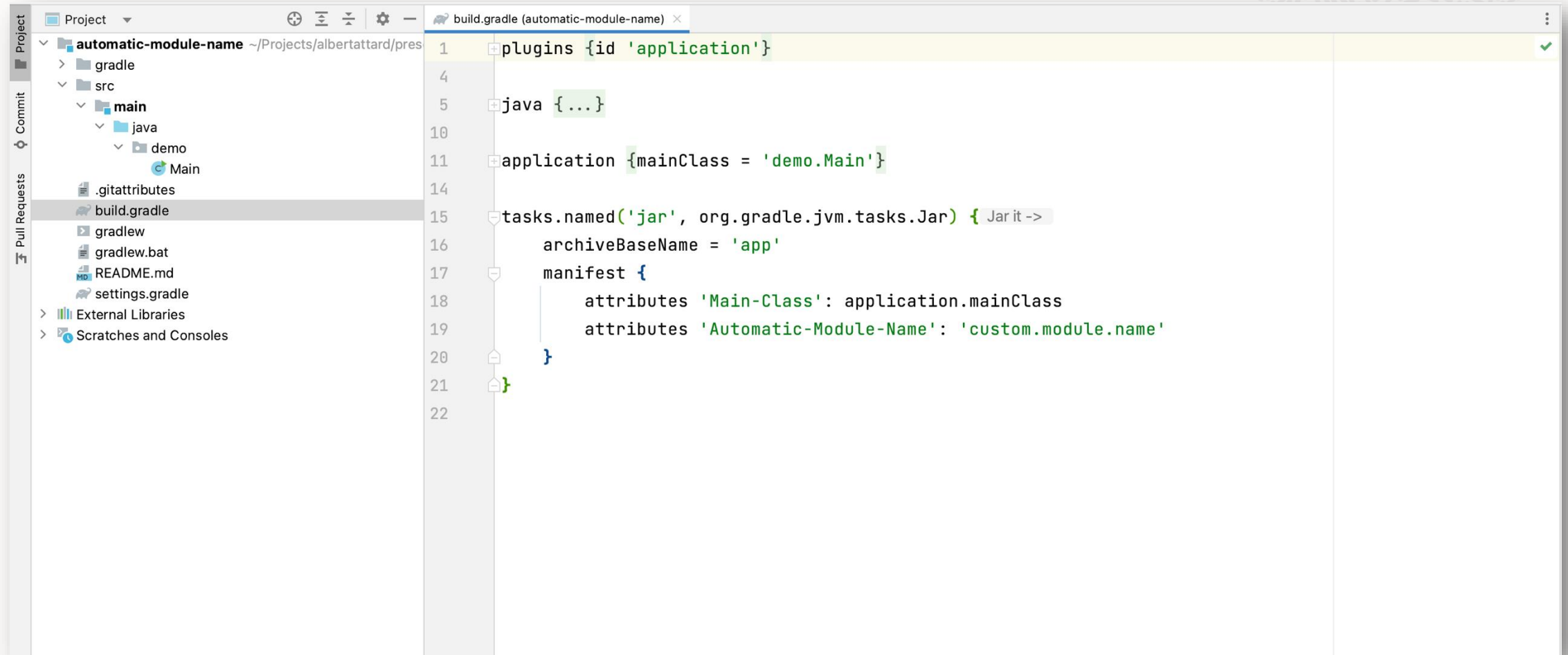
Assuming the *Automatic-Module-Name* is not specified

Description	<code>domain-a2-1.0.0-SNAPSHOT.jar</code>	<code>lib_\$-1.2.3.jar</code>
File extension	<code>domain-a2-1.0.0-SNAPSHOT</code>	<code>lib_\$-1.2.3</code>
Version	<code>domain-a2</code>	<code>lib_\$</code>
Special characters	<code>domain.a2</code>	<code>lib..</code>
Sequence of .	<code>domain.a2</code>	<code>lib.</code>
Leading/trailing .	<code>domain.a2</code>	<code>lib</code>



How can the *Automatic-Module-Name* be set?

Automatic-Module-Name



The generated *META-INF/MANIFEST.MF* file

```
> unzip -p 'app.jar' \
    'META-INF/MANIFEST.MF'
```

```
Manifest-Version: 1.0
```

```
Automatic-Module-Name: custom.module.name
```

```
Build-Jdk-Spec: 1.8
```

```
Created-By: Maven JAR Plugin 3.4.2
```

```
Main-Class: demo.Main
```

Describe module

```
> jar --describe-module --file app.jar
```

```
No module descriptor found. Derived  
automatic module.
```

```
custom.module.name automatic  
requires java.base mandated  
contains demo  
main-class demo.Main
```

Running the application

Using the module path

```
> java \  
  --module-path app.jar \  
  --module custom.module.name
```

Unnamed

Automatic



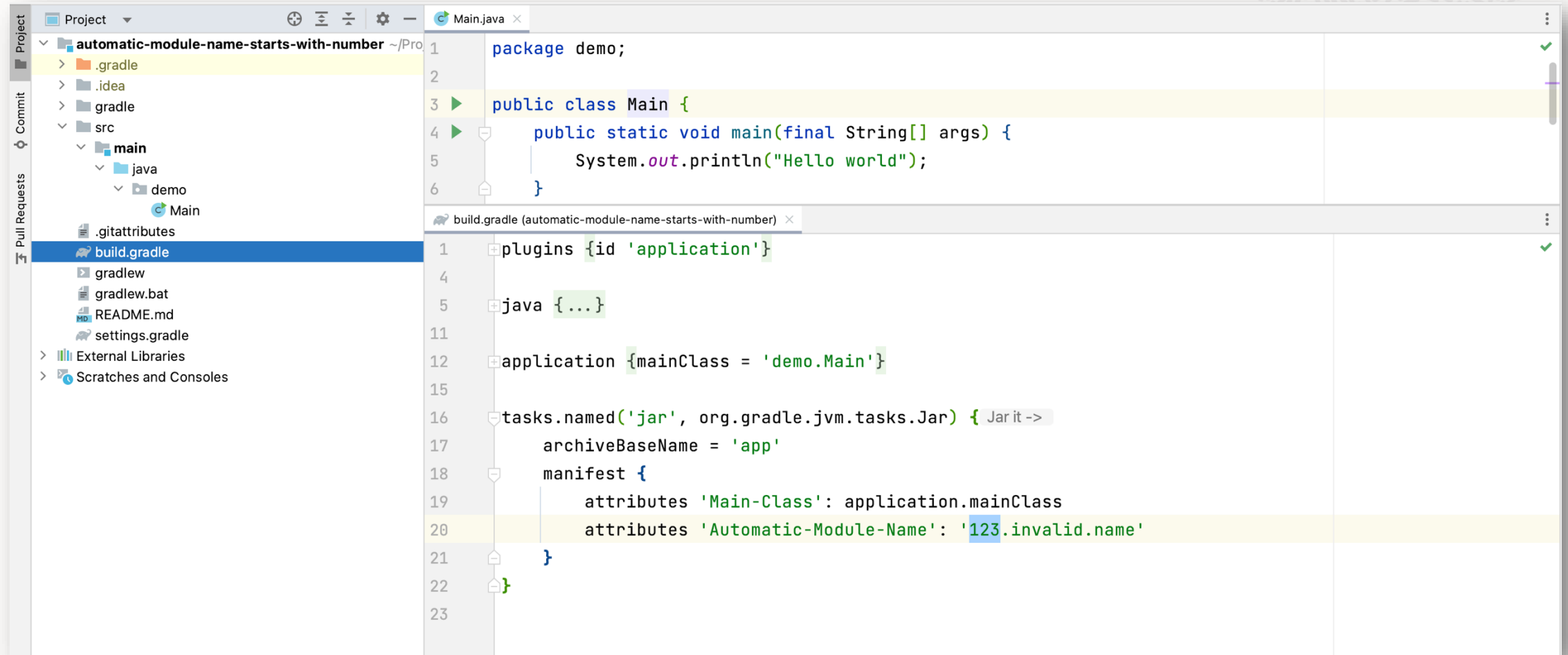
Named

Demo

`automatic-module-name`

Can the module name starts with a number?

Module name starting with a number



Describe module

```
> jar --describe-module --file app.jar
```

```
Unable to derive module descriptor for:  
app.jar
```

```
Automatic-Module-Name: 123.module.name:
```

```
Invalid module name: '123' is not a Java  
identifier
```

Running the application

Using the module path

```
> java \  
  --module-path app.jar \  
  --module 123.invalid.name
```

Running the application

Using the module path

```
Error occurred during initialization of  
boot layer
```

```
java.lang.module.FindException: Unable  
to derive module descriptor for app.jar
```

```
Caused by:
```

```
java.lang.module.FindException: Automatic-  
Module-Name: 123.module.name: Invalid  
module name: '123' is not a Java  
identifier
```

Running the application

Using the class path

```
> java \  
  --class-path app.jar \  
  demo.Main
```

Unnamed



Automatic

Named

Demo

automatic-module-name-starts-with-
number

Can an *automatic* module be packaged as a directory instead of a JAR?

Application

The screenshot displays an IDE interface with two open files: `Main.java` and `MANIFEST.MF`.

Main.java:

```
1 package demo;  
2  
3 public class Main {  
4     public static void main(final String[] args) {  
5         System.out.println("Hello world");  
6     }  
7 }  
8
```

MANIFEST.MF:

```
1 Manifest-Version: 1.0  
2 Main-Class: demo.Main  
3 Automatic-Module-Name: demo  
4
```

Annotations:

- An arrow points from the text "Automatic module name" to the `Automatic-Module-Name: demo` line in `MANIFEST.MF`.
- An arrow points from the text "Module name" to the `demo` folder in the project structure.

Project Structure:

- Project
 - automatic-module-needs-to-be
 - gradle
 - src
 - main
 - java
 - demo
 - Main

The project structure also includes files like `.gitattributes`, `build.gradle`, `gradlew`, `gradlew.bat`, `README.md`, and `settings.gradle`.

List modules

```
> java \  
  --module-path build/automatic \  
  --list-modules
```

```
java.base@25.0.1  
java.compiler@25.0.1  
...  
jdk.xml.dom@25.0.1  
jdk.zipfs@25.0.1
```

Only Java modules

List modules

```
> java \  
  --module-path build/automatic/demo \  
  --list-modules
```

```
java.base@25.0.1  
java.compiler@25.0.1  
...  
jdk.xml.dom@25.0.1  
jdk.zipfs@25.0.1
```

Only Java modules

Running the application

Using the class path

```
> java \  
  --class-path build/automatic/demo \  
  demo.Main
```

Unnamed



Automatic

Named

Running the application

Using the module path

```
> java \  
  --module-path build/automatic \  
  --module demo
```

```
Error occurred during initialization of  
boot layer  
java.lang.module.FindException: Module  
demo not found
```

Running the application

Using the module path

```
> java \  
  --module-path build/automatic/demo \  
  --module demo
```

```
Error occurred during initialization of  
boot layer  
java.lang.module.FindException: Module  
demo not found
```

Demo

automatic-module-needs-to-be-
packaged-as-jar

How can *named* modules be used?

Hello World

Using Modules



The screenshot shows an IDE interface with a project named "simple-hello-world-application-with-modules". The project structure in the left sidebar includes a "main" module containing a "demo" module. The "demo" module is highlighted, and its "module-info.java" file is open in the editor. The code in "module-info.java" is:

```
1 module demo {}
```

The "demo" module name is highlighted with a box. The "Main.java" file is also open, showing the following code:

```
1 package demo;
2
3 public class Main {
4     public static void main(final String[] args) {
5         System.out.println("Hello world");
6     }
7 }
```

The "demo" module name is highlighted with a box. The "Main" class is highlighted with a yellow background.

This is the module name

Java 11+

List modules

```
> java --module-path libs --list-modules
```

```
java.base@25.0.1  
java.compiler@25.0.1  
java.datatransfer@25.0.1  
java.desktop@25.0.1  
java.instrument@25.0.1  
...
```

```
demo file:///.../libs/app.jar
```

No automatic
← **at the end!**

Describe module

```
> jar --describe-module --file app.jar
```

```
demo jar:file:///.../app.jar!/module-  
info.class  
requires java.base mandated  
contains demo  
main-class demo.Main
```

Running the application

Using the class path

```
> java \  
  --class-path app.jar \  
  demo.Main
```

Unnamed



Automatic

Named

Running the application

Using the module path

```
> java \  
  --module-path app.jar \  
  --module demo
```

Unnamed

Automatic

Named



Running the application

Using the module path with the full qualified name

```
> java \  
  --module-path app.jar \  
  --module demo/demo.Main
```

Unnamed

Automatic

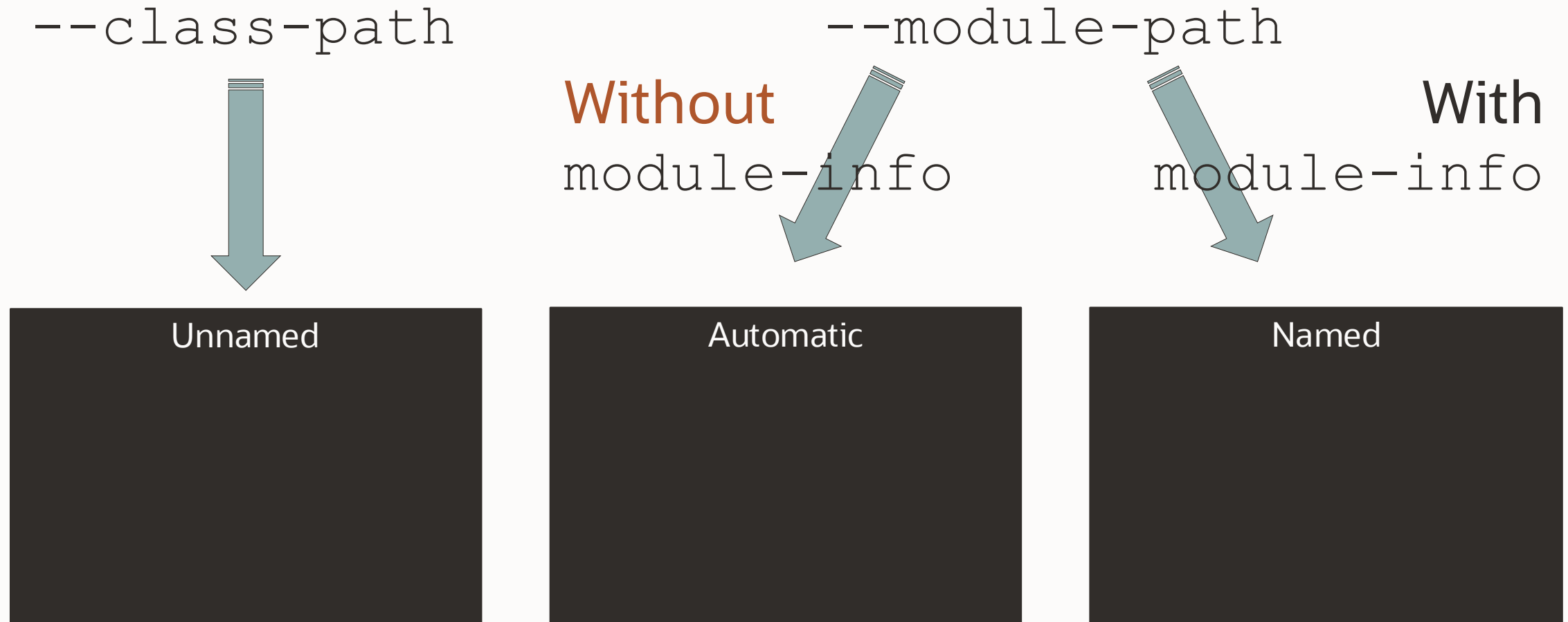
Named



Demo

simple-hello-world-application-with-modules

Module types



Can we include the `module-info.java` file in a Java 8 JAR file?

JEP 238: Multi-Release JAR Files



Multi-Release JAR Files



```
demo
└─ Main.class
META-INF
└─ versions
    └─ 11
        └─ module-info.class
```

This is only used when
running with Java 11+

Multi-Release JAR Files



```
demo
└─ Main.class
META-INF
└─ versions
```

```
└─ 11
   └─ demo
      └─ Main.class
```

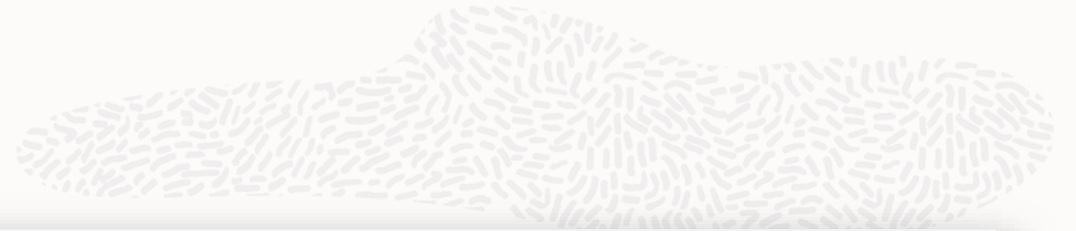
When running with
Java 11+, this version
of the class is used
instead

Multi-Release JAR Files



```
demo
└─ Main.class
META-INF
└─ versions
    └─ 11
        └─ ...
    └─ 17
        └─ ...
```

Application



The screenshot shows an IDE interface with a project named "multi-release-jar-file". The project structure in the left sidebar includes:

- multi-release-jar-file
├── .mvn
├── src
│ ├── main
│ │ ├── java
│ │ │ ├── demo
│ │ │ │ └── Main
│ │ └── java11
│ │ └── module-info.java
├── .gitattributes
├── mvnw
├── mvnw.cmd
├── pom.xml
├── README.md
├── External Libraries
└── Scratches and Consoles

The main editor displays two files:

```
Main.java
1 package demo;
2
3 public class Main {
4     public static void main(final String[] args) {
5         System.out.printf("Java version: %s\n", System.getProperty("java.version"));
6     }
7 }
8
```

```
module-info.java (demo)
1 module demo {}
2
```

Java 11+



Maven Configuration

Base Java Version

```
<?xml version='1.0' encoding='UTF-8'?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0"
  <modelVersion>4.0.0</modelVersion>

  <groupId>demo</groupId>
  <artifactId>multi-release-jar-file</artifactId>
  <version>1.0.0-SNAPSHOT</version>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>1.8</maven.compiler.source>
    <maven.compiler.target>1.8</maven.compiler.target>
  </properties>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.10.1</version>
```

Base Java version

Java 11+

Maven Configuration

Compiler Plugin

The screenshot displays the Maven `pom.xml` configuration for the `maven-compiler-plugin` in an IDE. The configuration is as follows:

```
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.10.1</version>
<executions>
  <execution>
    <id>compile-java-11</id>
    <phase>compile</phase>
    <goals>
      <goal>compile</goal>
    </goals>
    <configuration>
      <release>11</release>
      <compileSourceRoots>
        <compileSourceRoot>${project.basedir}/src/main/java11</compileSourceRoot>
      </compileSourceRoots>
      <multiReleaseOutput>true</multiReleaseOutput>
    </configuration>
  </execution>
</executions>
</plugin>
```

Annotations in the image highlight the following configuration elements:

- Java version:** Points to the `<release>11</release>` configuration element.
- Source path:** Points to the `<compileSourceRoot>${project.basedir}/src/main/java11</compileSourceRoot>` configuration element.
- Multi release:** Points to the `<multiReleaseOutput>true</multiReleaseOutput>` configuration element.
- Java 11+:** A dark box in the bottom right corner indicating the target Java version.

Maven Configuration

JAR Plugin

```
36 </configuration>
37 </execution>
38 </executions>
39 </plugin>
40 <plugin>
41 <groupId>org.apache.maven.plugins</groupId>
42 <artifactId>maven-jar-plugin</artifactId>
43 <version>3.3.0</version>
44 <configuration>
45 <finalName>app</finalName>
46 <outputDirectory>${project.build.directory}/libs</outputDirectory>
47 <archive>
48 <manifest>
49 <addClasspath>true</addClasspath>
50 <mainClass>demo.Main</mainClass>
51 </manifest>
52 <manifestEntries>
53 <Multi-Release>true</Multi-Release>
54 </manifestEntries>
55 </archive>
56 </configuration>
57 </plugin>
58 </plugins>
```

Multi release
JAR file

Java 11+

Bytecode major version

```
> javap -v demo/Main.class | grep major  
major version: 52
```

This is Java 8 bytecode

```
> javap \  
-v META-INF/versions/11/module-  
info.class \  
| grep major  
major version: 55
```

This is Java 11 bytecode

Describe module

```
> jar --describe-module --file app.jar
```

```
releases: 11
```

```
No root module descriptor, specify --  
release
```

Describe module

```
> jar --describe-module --file app.jar \  
    --release 11
```

```
...  
demo jar:file:///.../app.jar!/META-  
INF/versions/11/module-info.class  
requires java.base mandated  
contains demo  
main-class demo.Main
```

Running the application



Java 8

```
> java -jar app.jar  
Java version: 1.8.0_471
```

Java 11

```
> java --module-path app.jar --module demo  
Java version: 11.0.29
```

What happens if the application is executed with a newer version of Java?

Running the application



Java 17

```
> java --module-path app.jar --module demo  
Java version: 17.0.17
```

Java 21

```
> java --module-path app.jar --module demo  
Java version: 21.0.9
```

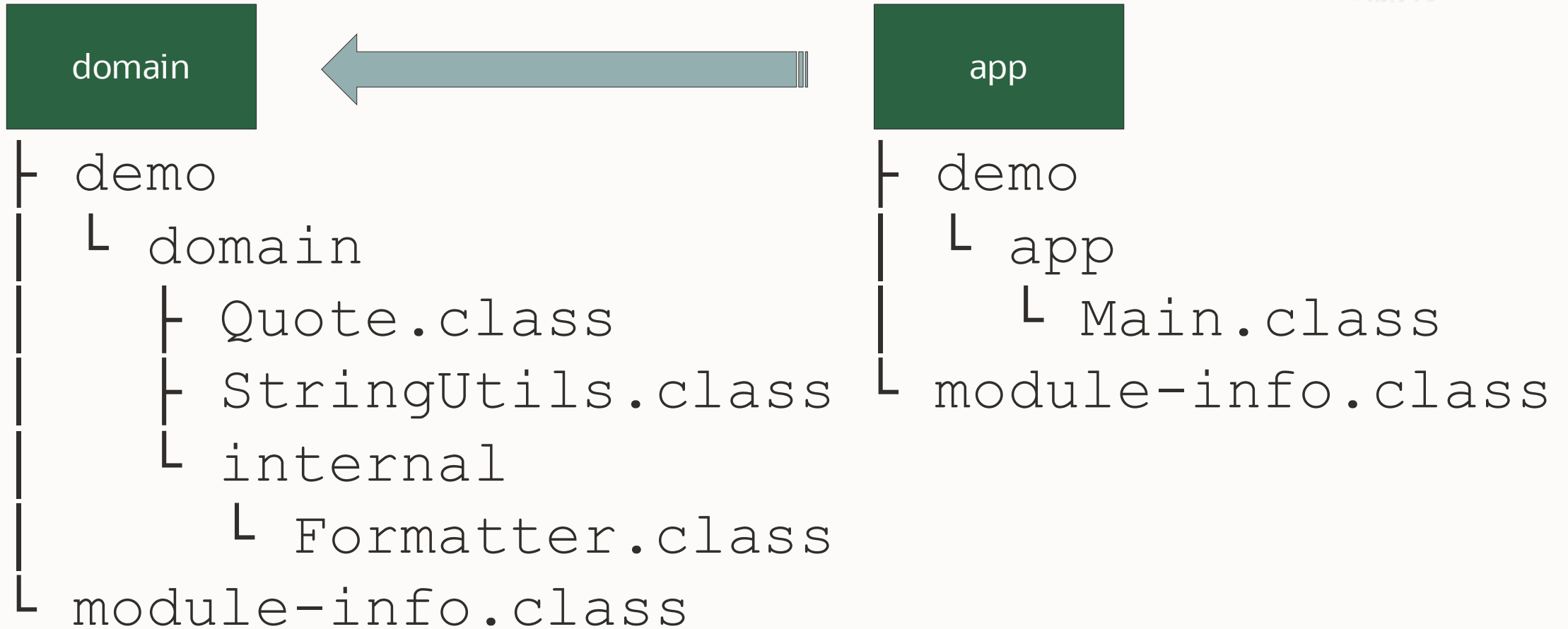
Demo

multi-release-jar-file

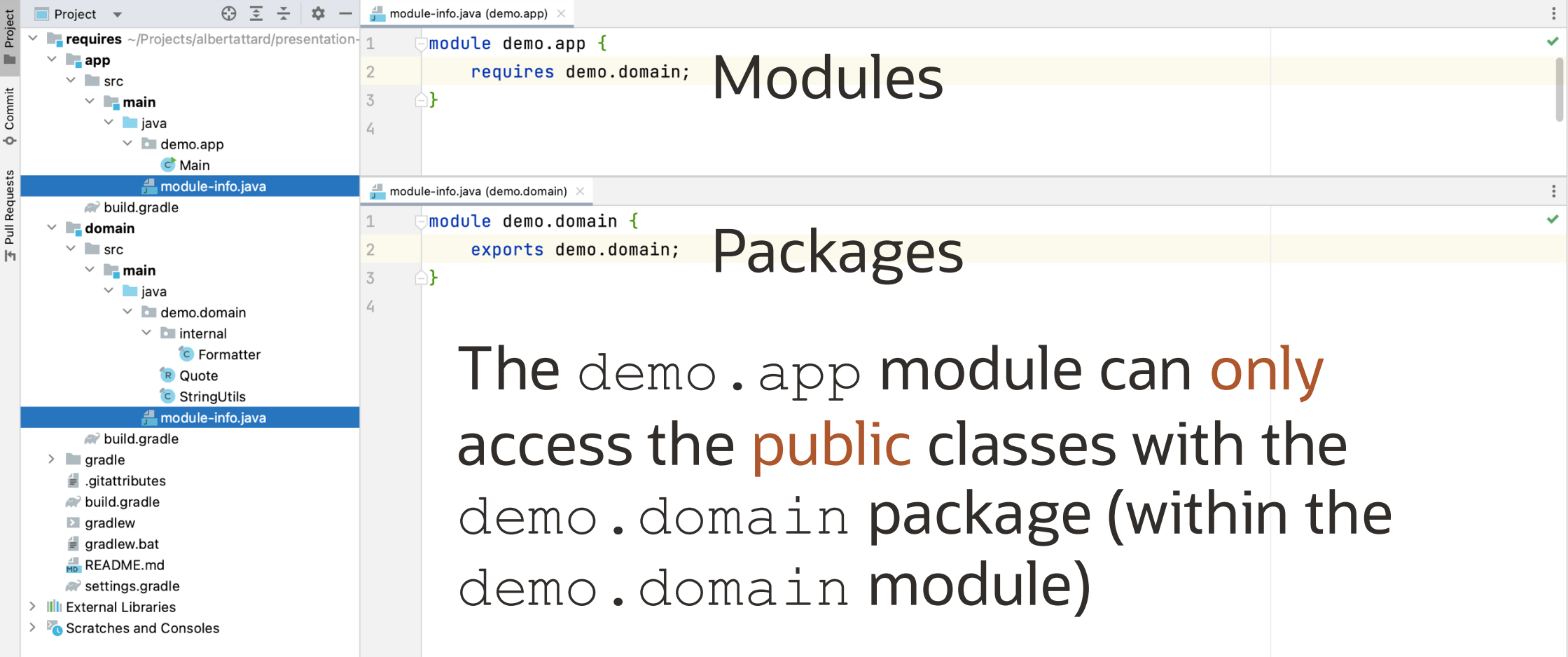
An example with two modules

exports and requires

Two modules



Two modules



The screenshot shows an IDE with two modules: `demo.app` and `demo.domain`. The `demo.app` module is highlighted in blue. The `demo.domain` module is also visible. The `demo.app` module has a `Main` class and a `module-info.java` file. The `demo.domain` module has a `module-info.java` file. The `demo.app` module is highlighted in blue.

```
1 module demo.app {  
2     requires demo.domain;  
3 }  
4
```

```
1 module demo.domain {  
2     exports demo.domain;  
3 }  
4
```

Modules

Packages

The `demo.app` module can **only** access the **public** classes with the `demo.domain` package (within the `demo.domain` module)

The default (*package-private*) access modifier

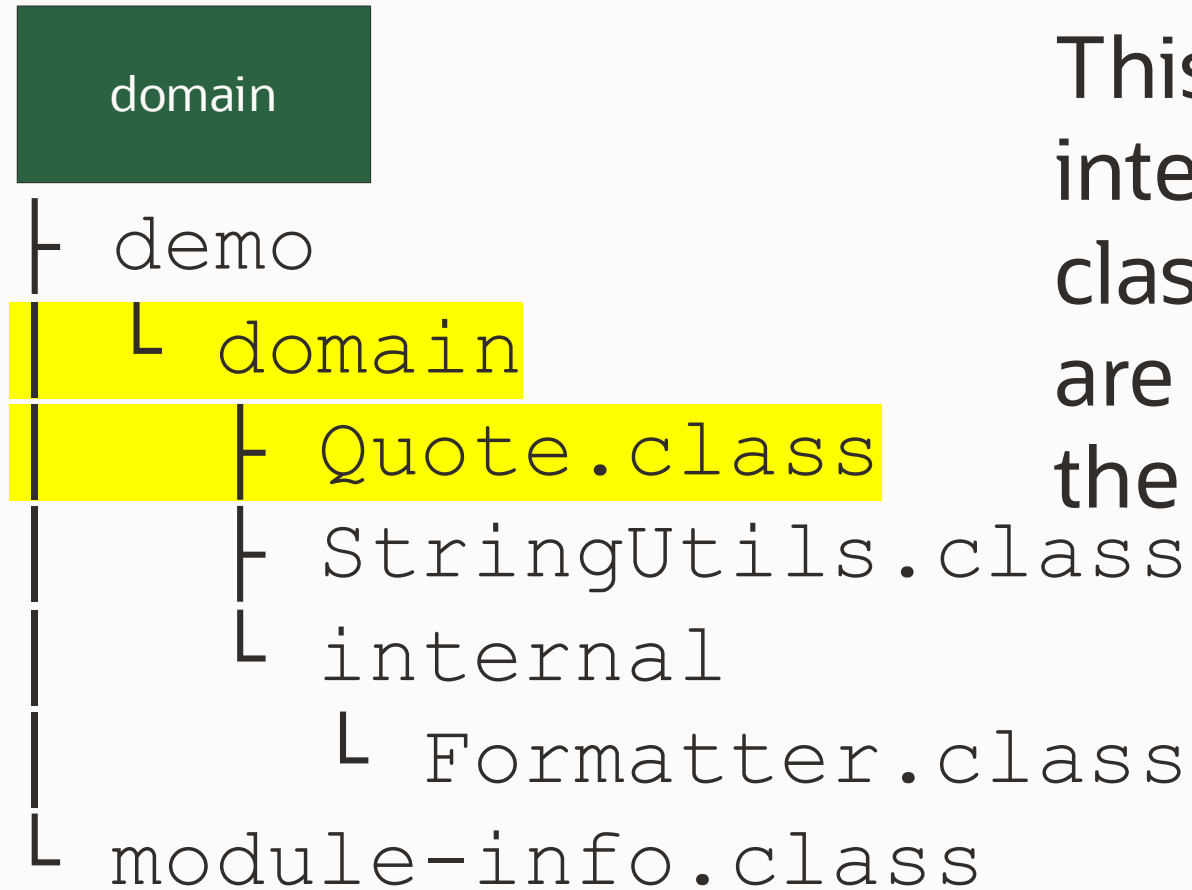


The screenshot shows an IDE with a project structure on the left and a code editor on the right. The project structure includes a 'requires' module with an 'app' sub-module, and a 'domain' module with a 'main' sub-module. The 'main' sub-module of 'domain' contains a 'java' package with a 'demo.domain' sub-package. The 'demo.domain' sub-package contains an 'internal' sub-package and a 'StringUtils' class. The 'StringUtils' class is highlighted in the project structure. The code editor shows the following code:

```
1 package demo.domain;
2
3 final class StringUtils {
4
5     @
6     static String failIfBlank(final String value) {
7         if (value == null || value.isBlank()) {
8             throw new IllegalArgumentException("Provided a blank string");
9         }
10
11         return value;
12     }
13
14     private StringUtils() {}
15 }
```

Only accessible from within the `demo.domain` module

Module's public interface



This is the module's public interface and only the **public** classes within this package are accessible from outside the module

Describe module

```
> jar --describe-module --file domain.jar
```

```
demo.domain
```

```
jar:file:///.../domain.jar!/module-  
info.class
```

```
exports demo.domain
```

```
requires java.base mandated
```

Describe module

```
> jar --describe-module --file app.jar
```

```
demo.app jar:file:///.../app.jar!/module-  
info.class
```

```
requires demo.domain
```

```
requires java.base mandated
```

```
contains demo.app
```

```
main-class demo.app.Main
```

Running the application

Using the class path

```
> java \  
  --class-path app.jar:domain.jar \  
  demo.Main
```

Unnamed



Automatic

Named

Running the application

Using the module path

```
> java \  
  --module-path app.jar:domain.jar \  
  --module demo
```

Unnamed

Automatic

Named



There is a 1:1 mapping between JAR files and *automatic* and *named* modules

The number of *unnamed* modules is implementation specific; thus, this ratio may not hold for *unnamed* modules

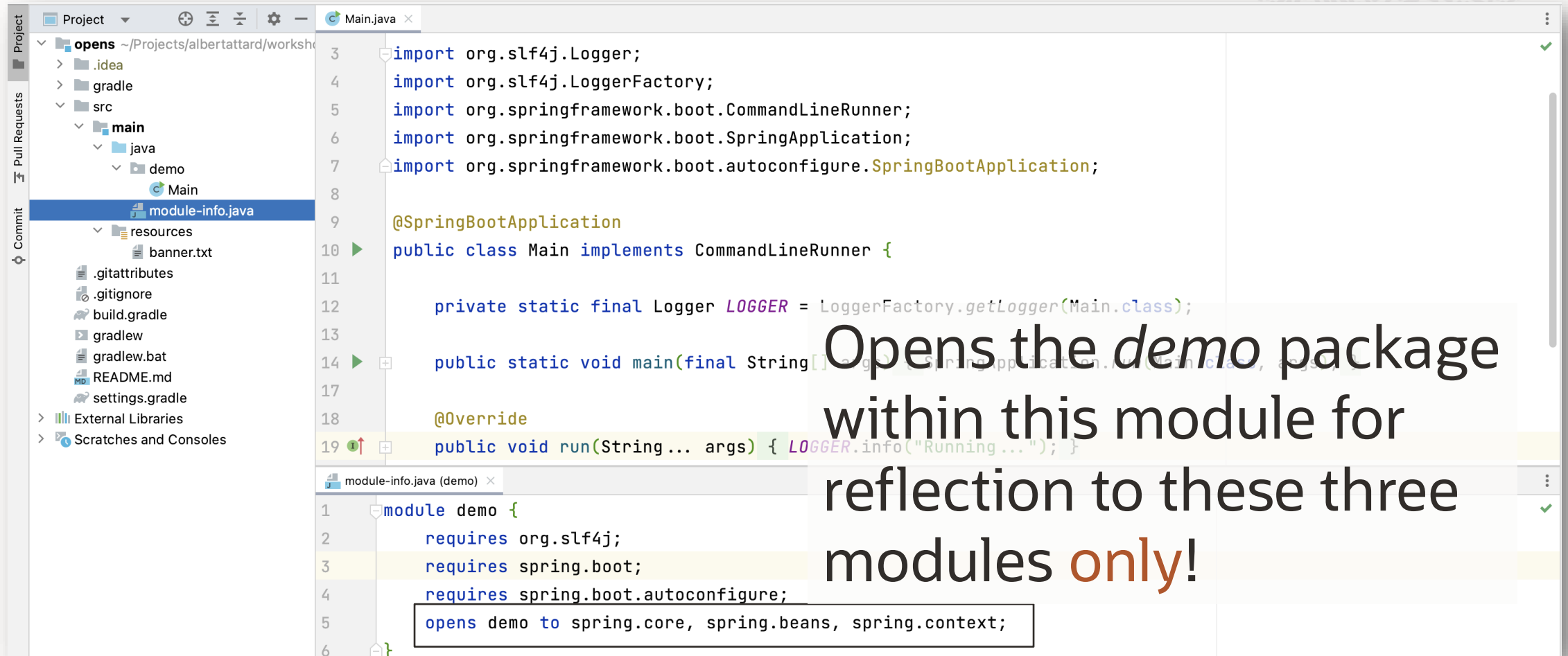
Demo

requires

How does reflection work?

—
opens

Reflection



The screenshot shows an IDE with two files open. The top file, `Main.java`, is a Spring Boot application. The bottom file, `module-info.java (demo)`, is a module descriptor. A text overlay explains the purpose of the `opens` statement in the module-info.java file.

module-info.java (demo)

```
1 module demo {
2     requires org.slf4j;
3     requires spring.boot;
4     requires spring.boot.autoconfigure;
5     opens demo to spring.core, spring.beans, spring.context;
6 }
```

Main.java

```
3 import org.slf4j.Logger;
4 import org.slf4j.LoggerFactory;
5 import org.springframework.boot.CommandLineRunner;
6 import org.springframework.boot.SpringApplication;
7 import org.springframework.boot.autoconfigure.SpringBootApplication;
8
9 @SpringBootApplication
10 public class Main implements CommandLineRunner {
11
12     private static final Logger LOGGER = LoggerFactory.getLogger(Main.class);
13
14     public static void main(final String[] args) {
15         SpringApplication.run(Main.class, args);
16     }
17
18     @Override
19     public void run(String... args) { LOGGER.info("Running..."); }
```

Opens the *demo* package within this module for reflection to these three modules **only!**

How does this work with *automatic* and *unnamed* modules?

The *automatic* and *unnamed* modules open everything to everyone

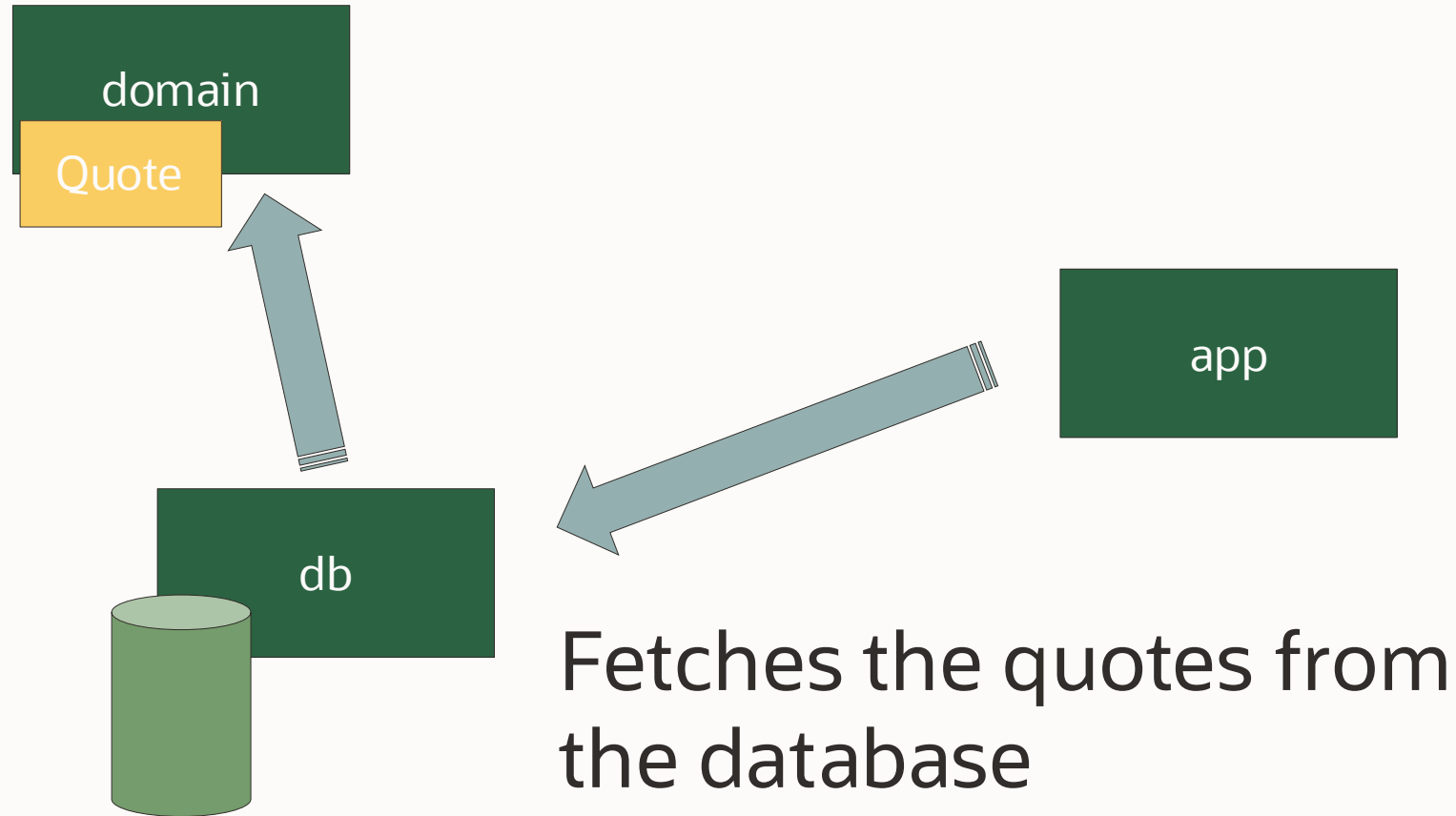
Demo

opens

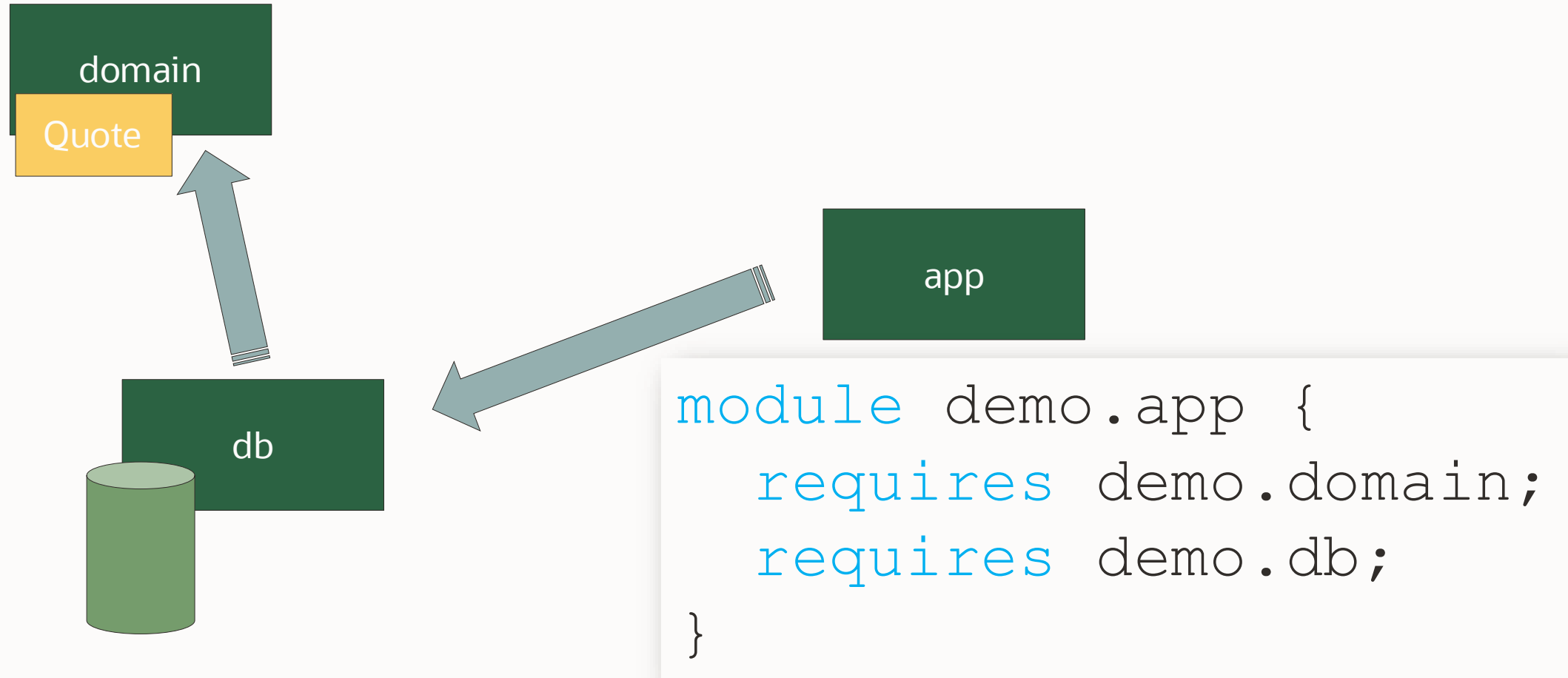
Transitive dependencies

requires transitive

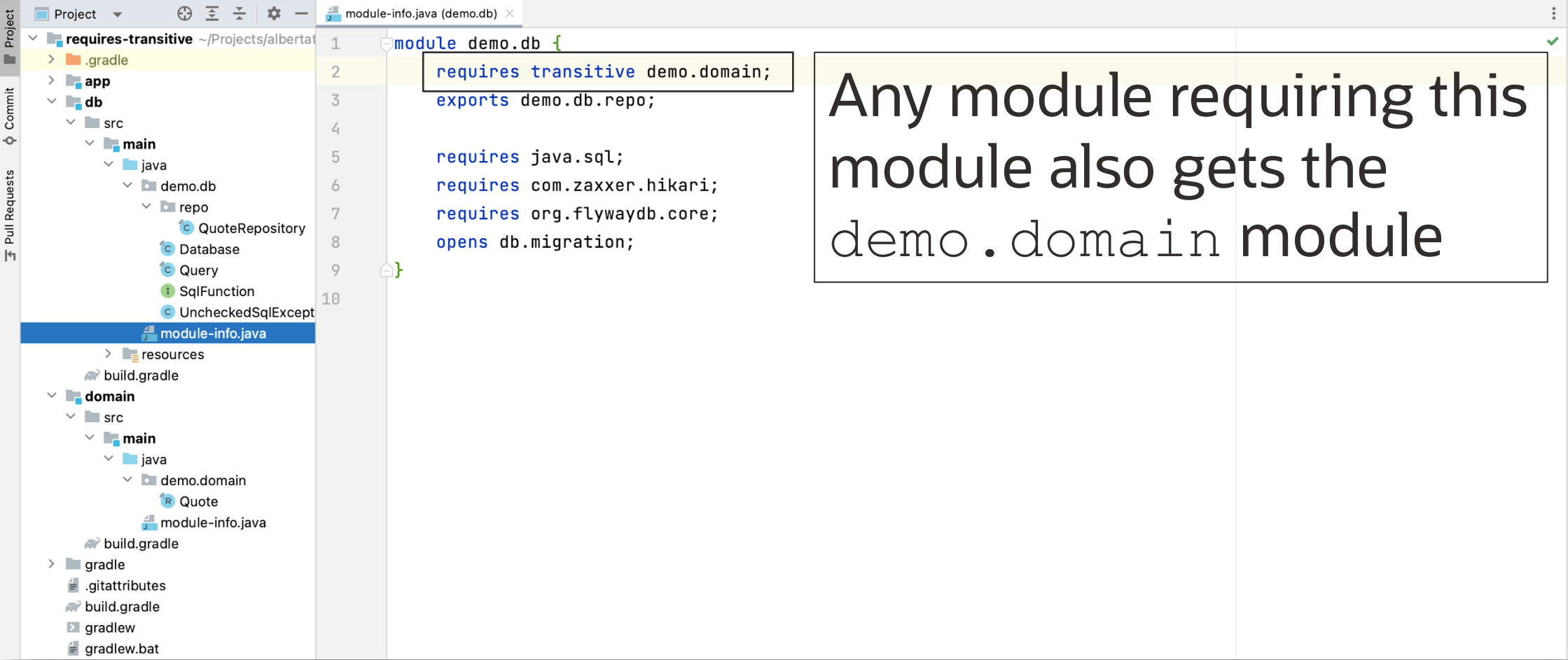
Application



Application



Using transitive



The screenshot shows an IDE with a project structure on the left and a code editor on the right. The project structure includes a `requires-transitive` module with a `demo.db` sub-module. The `demo.db` module is defined in `module-info.java` with the following code:

```
1 module demo.db {  
2     requires transitive demo.domain;  
3     exports demo.db.repo;  
4  
5     requires java.sql;  
6     requires com.zaxxer.hikari;  
7     requires org.flywaydb.core;  
8     opens db.migration;  
9 }  
10
```

A text box on the right explains the effect of the `requires transitive` statement:

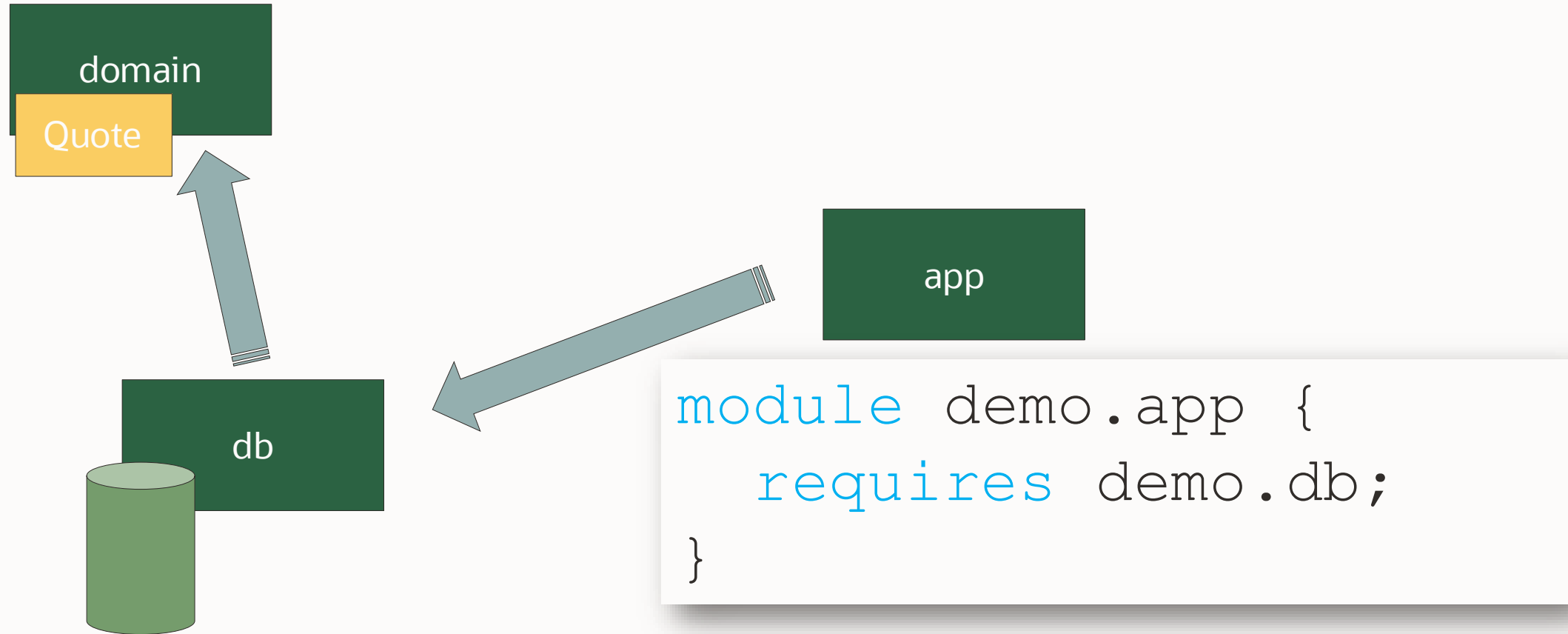
Any module requiring this module also gets the `demo.domain` module

Describe module

```
> jar --describe-module --file db.jar

demo.db jar:file:///.../db.jar!/module-
info.class
exports demo.db.repo
requires com.zaxxer.hikari
requires demo.domain transitive
...
opens db.migration
```

Application



Describe module

```
> jar --describe-module --file app.jar
```

```
demo.app jar:file:///.../app.jar!/module-  
info.class
```

```
requires demo.db
```

```
requires java.base mandated
```

```
contains demo.app
```

```
main-class demo.app.Main
```

Does not require
demo.domain

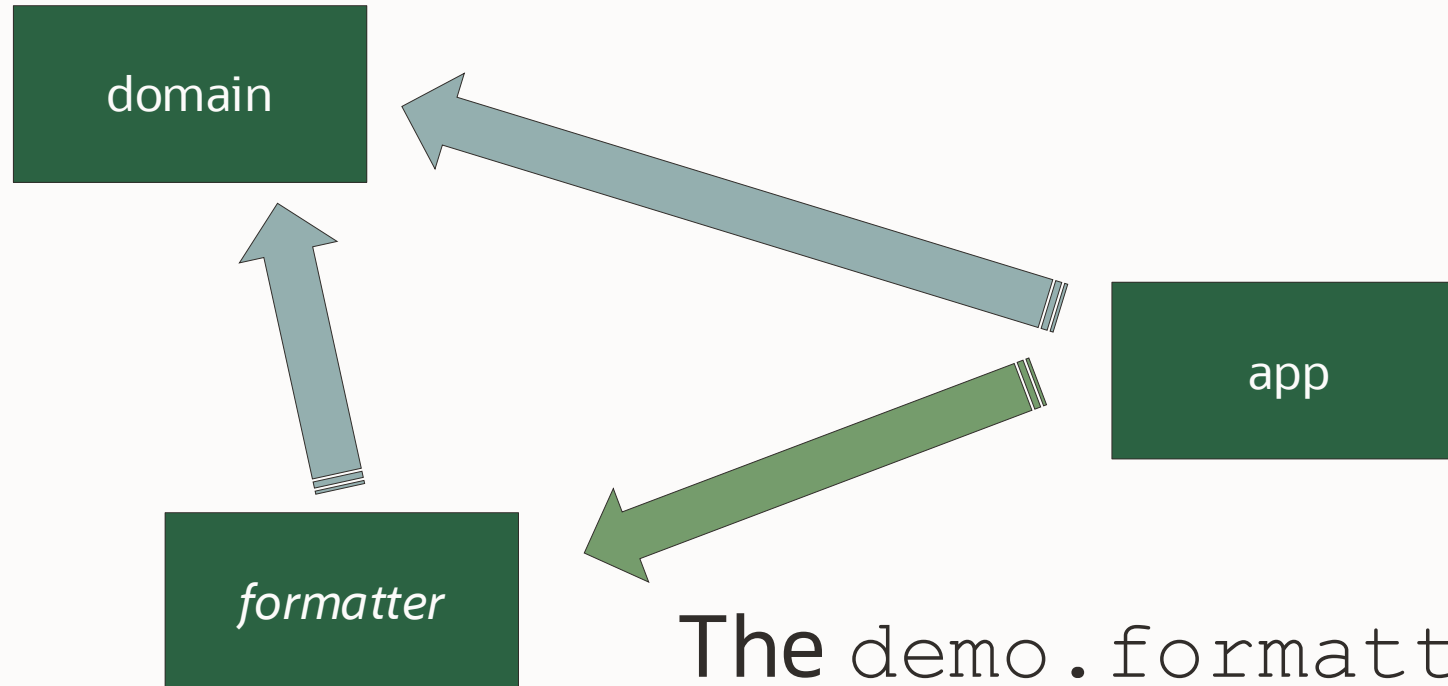
Demo

requires-transitive

Is it possible to have optional modules (module may or may not be present at runtime)?

requires static

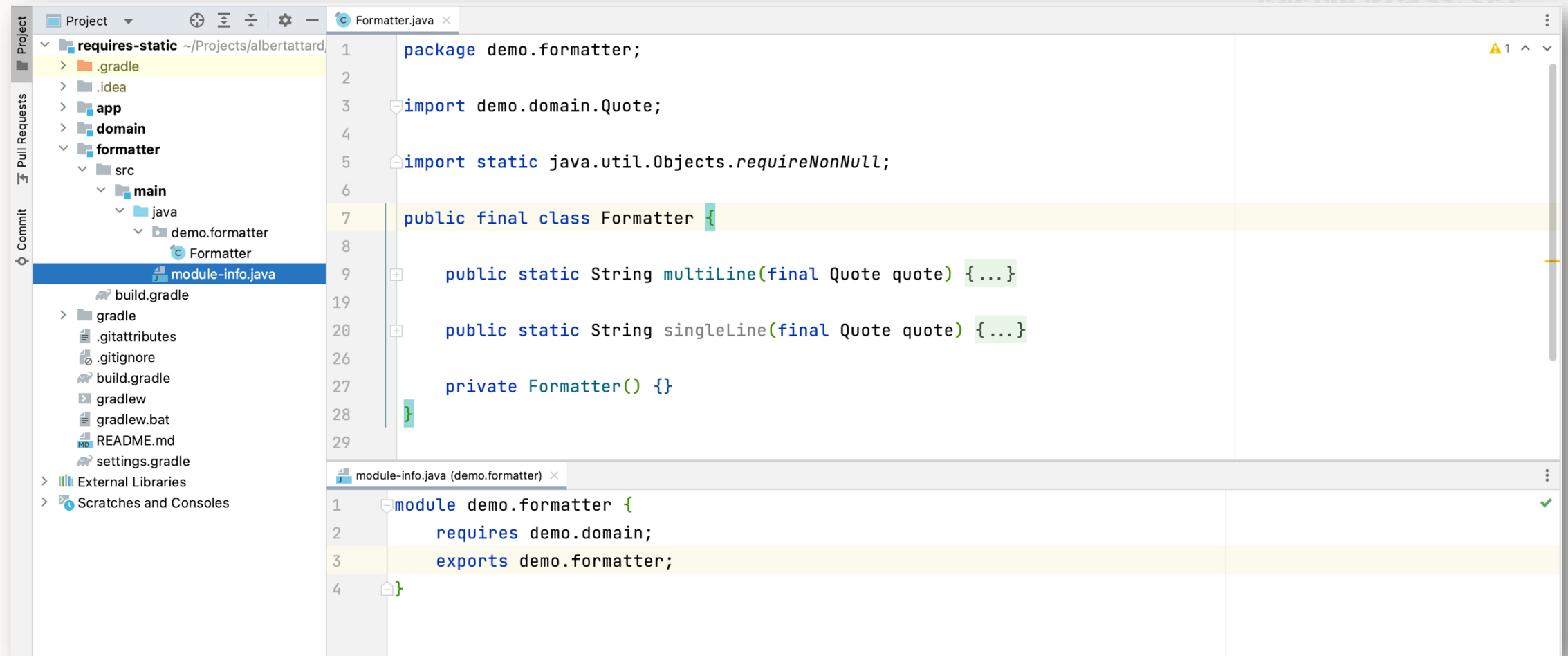
Application



The `demo.formatter` module is optional for the `demo.app` module

Application

The demo.formatter module



Describe module

```
> jar --describe-module \  
  --file formatter.jar
```

```
demo.formatter
```

```
jar:file:///.../formatter.jar!/module-  
info.class
```

```
exports demo.formatter
```

```
requires java.base mandated
```

Application

The demo .app module

```
1 package demo.app;
2
3 import demo.domain.Quote;
4 import demo.formatter.Formatter;
5
6 public class Main {
7     public static void main(final String[] args) {
8         final Quote quote = new Quote( author: "Albert Einstein", quote: "Learn from yesterday, live for today, hope
9
10         if (isModulePresent( moduleName: "demo.formatter")) {
11             String multiLine = Formatter.multiLine(quote);
12             System.out.println(multiLine);
13         } else {
14             System.out.println(quote);
15         }
16     }
17 }
```

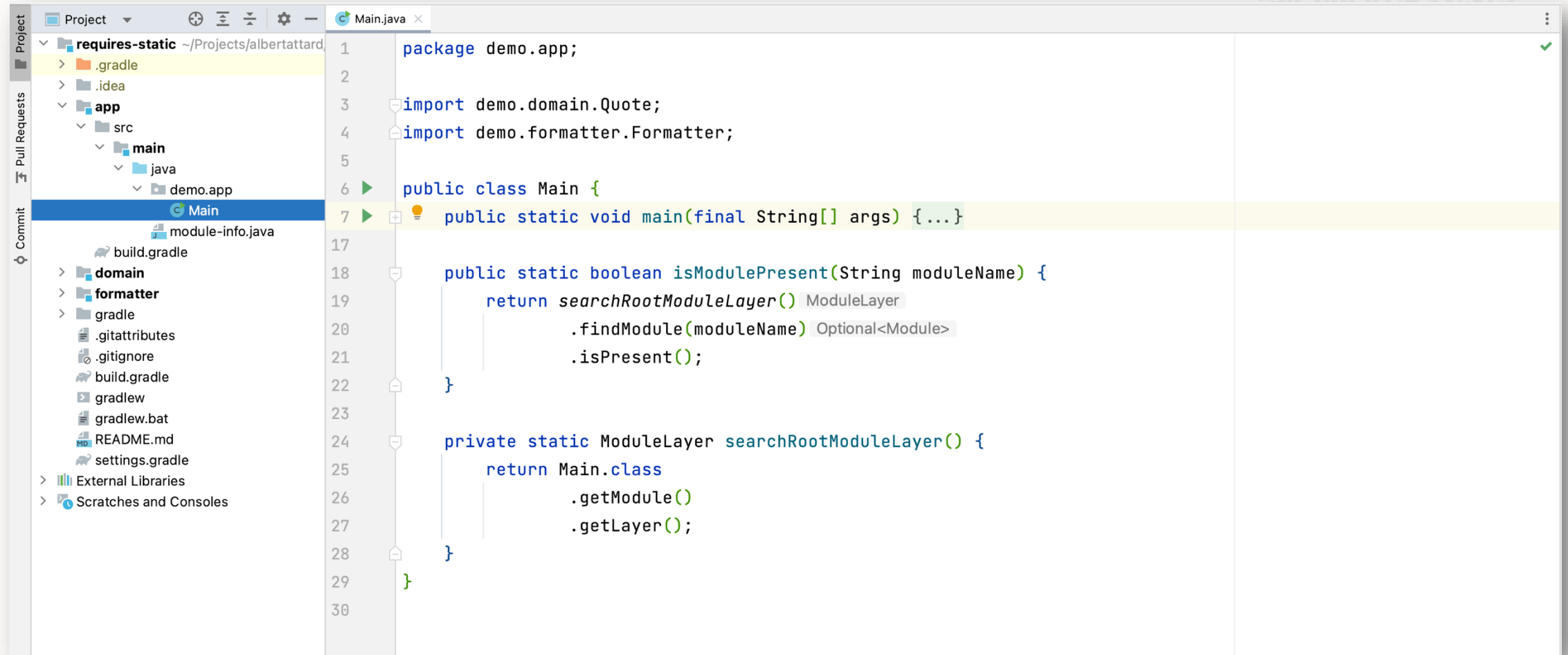
Check before use!!

```
1 module demo.app {
2     requires demo.domain;
3     requires static demo.formatter;
4 }
```

Makes this module optional

Application

The isModulePresent () method



The screenshot shows an IDE window with a project explorer on the left and a code editor on the right. The project explorer shows a project named 'requires-static' with a directory structure including 'src/main/java/demo.app'. The code editor displays the 'Main.java' file with the following code:

```
1 package demo.app;
2
3 import demo.domain.Quote;
4 import demo.formatter.Formatter;
5
6 public class Main {
7     public static void main(final String[] args) {...}
8
9     public static boolean isModulePresent(String moduleName) {
10         return searchRootModuleLayer().ModuleLayer
11             .findModule(moduleName).Optional<Module>
12             .isPresent();
13     }
14
15     private static ModuleLayer searchRootModuleLayer() {
16         return Main.class
17             .getModule()
18             .getLayer();
19     }
20 }
21
22 }
```

Describe module

```
> jar --describe-module --file app.jar
```

```
demo.app jar:file:///.../app.jar!/module-  
info.class
```

```
requires demo.db
```

```
requires demo.formatter static
```

```
requires java.base mandated
```

```
contains demo.app
```

```
main-class demo.app.Main
```

List modules

```
> java --module-path libs --list-modules
```

```
java.base@25.0.1  
java.compiler@25.0.1  
java.datatransfer@25.0.1  
...  
demo.app file:///.../app.jar  
demo.domain file:///.../domain.jar  
demo.formatter file:///.../formatter.jar
```

Running the application

Using the module path

```
> java \  
  --module-path libs \  
  --module demo.app
```

Did **not** use the `demo.formatter` module!

```
Quote[author=Albert Einstein, quote=Learn  
from yesterday, live for today, hope for  
tomorrow. The important thing is not to  
stop questioning.]
```

Running the application

Using the module path and *--add-modules*

```
> java \  
  --module-path libs \  
  --add-modules demo.formatter \  
  --module demo.app
```

The `demo.formatter` module is found and used!

Running the application

Using the module path and *--add-modules*



```
~~~~~  
~~~~~  
"Learn from yesterday, live for today,  
hope for tomorrow. The important thing is  
not to stop questioning."  
~~~~~  
~~~~~  
by Albert Einstein
```

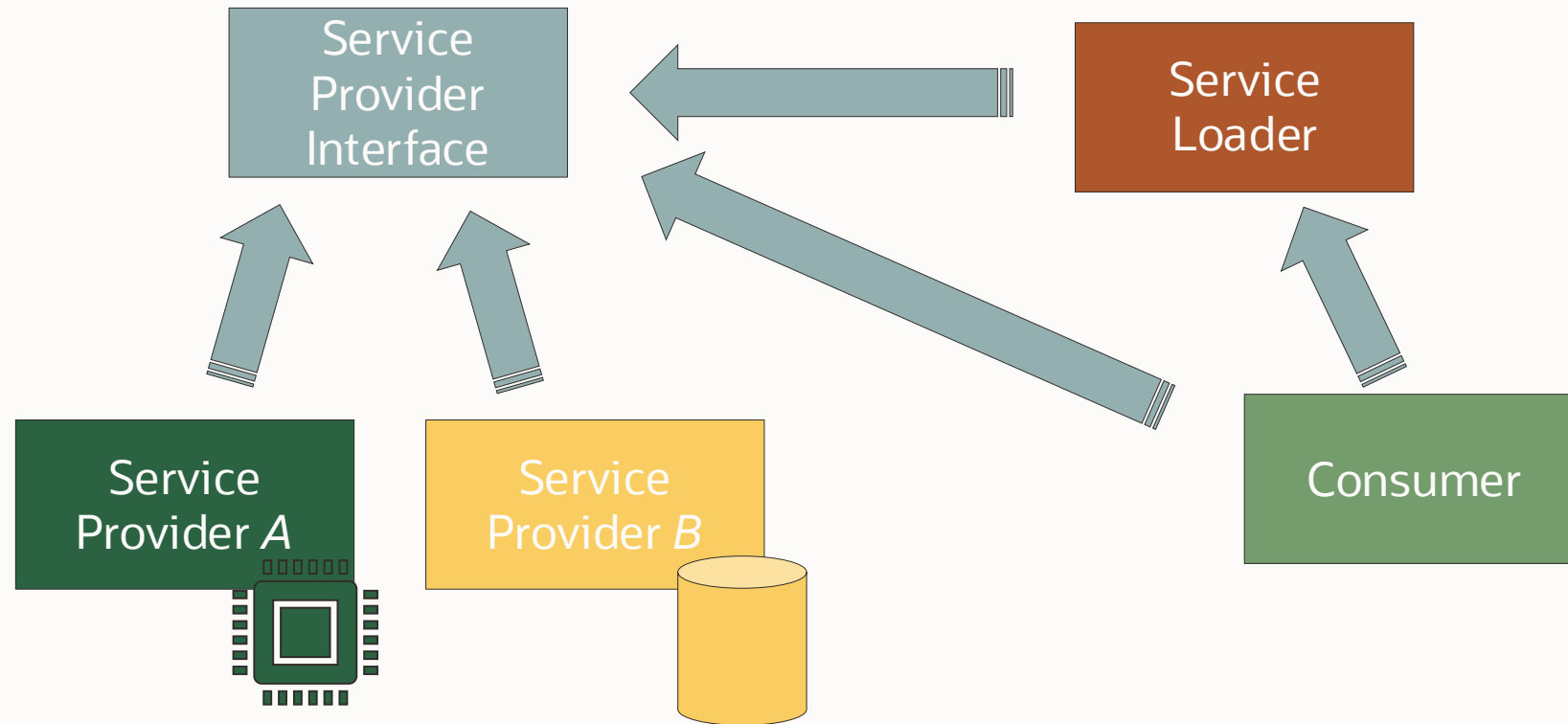


Demo

`requires-static`

Service Provider Interface

Service Provider Interface

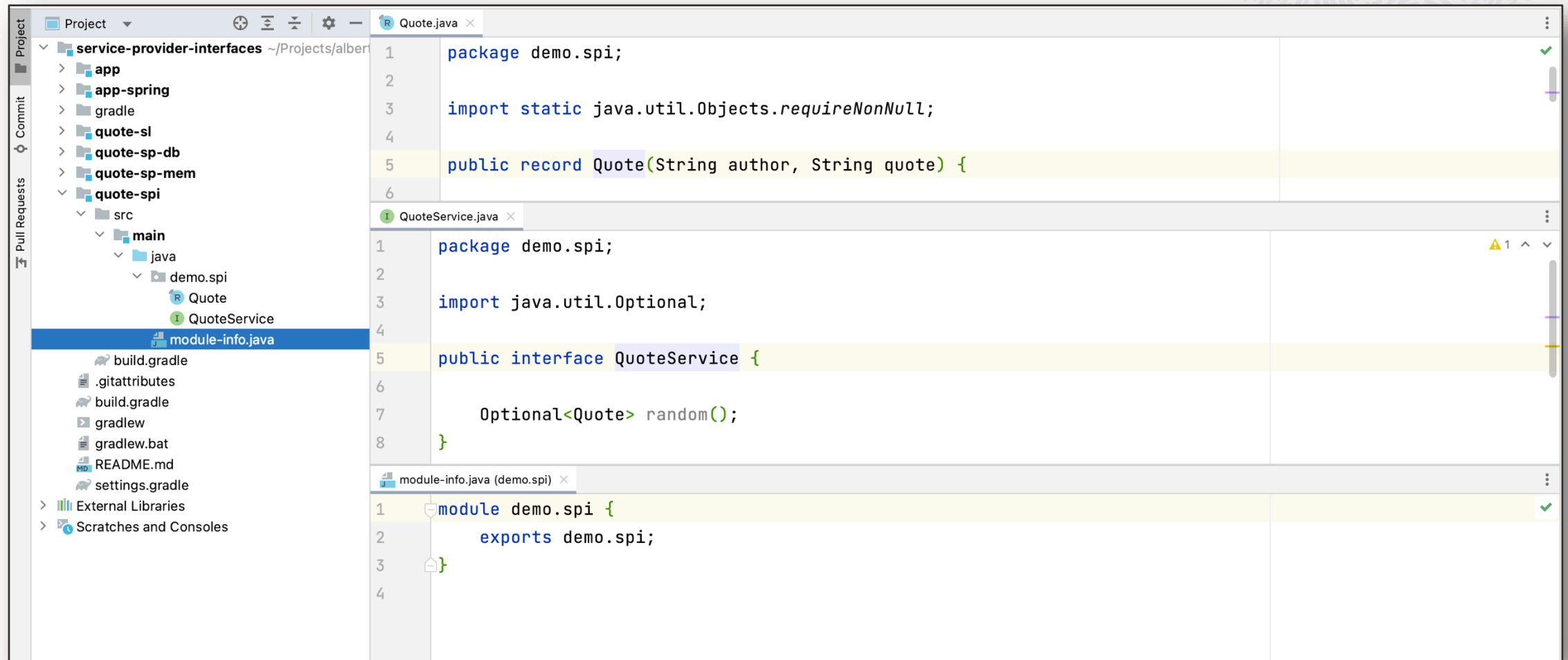


Service Provider Interface



Service
Provider
Interface

Service Provider Interface



The screenshot displays an IDE interface with a project structure on the left and three code files open in the editor.

Project Structure:

- service-provider-interfaces
 app
 app-spring
 gradle
 quote-sl
 quote-sp-db
 quote-sp-mem
 quote-spi
 src
 main
 java
 demo.spi
 Quote
 QuoteService
 module-info.java
build.gradle
.gitattributes
build.gradle
gradlew
gradlew.bat
README.md
settings.gradle
- External Libraries
- Scratches and Consoles

Quote.java

```
1 package demo.spi;  
2  
3 import static java.util.Objects.requireNonNull;  
4  
5 public record Quote(String author, String quote) {  
6
```

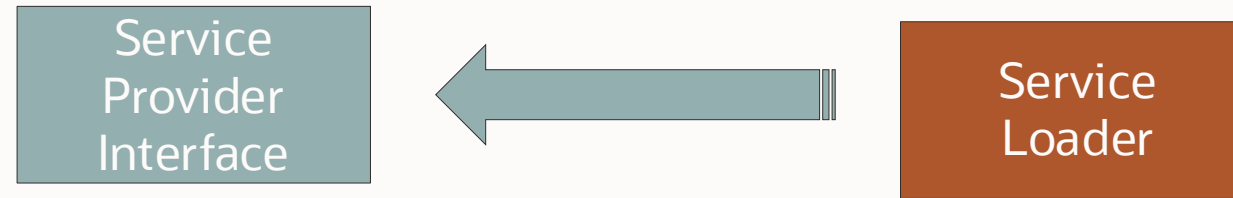
QuoteService.java

```
1 package demo.spi;  
2  
3 import java.util.Optional;  
4  
5 public interface QuoteService {  
6  
7     Optional<Quote> random();  
8 }
```

module-info.java (demo.spi)

```
1 module demo.spi {  
2     exports demo.spi;  
3 }  
4
```

Service Loader



*SL requires the
SPI module and
uses the SPI*

*SL exports
the loader
package*

Service Loader



The screenshot displays an IDE with two open files. The top file, `QuoteServiceLoader.java`, contains the following code:

```
1 package demo.sl;  
2  
3 import demo.spi.QuoteService;  
4  
5 import java.util.ServiceLoader;  
6  
7 public final class QuoteServiceLoader {  
8  
9     @ public Iterable<QuoteService> services() {  
10         return ServiceLoader.load(QuoteService.class);  
11     }  
12
```

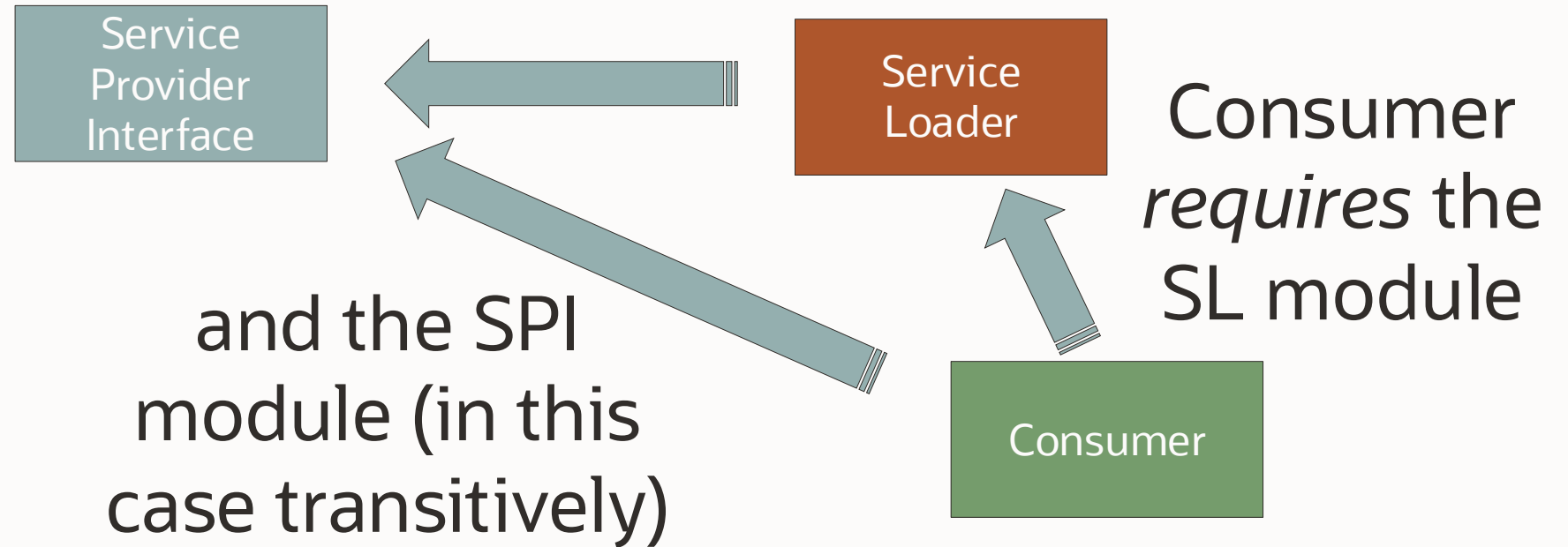
The bottom file, `module-info.java (demo.sl)`, contains the following code:

```
1 import demo.spi.QuoteService;  
2  
3 module demo.sl {  
4     requires transitive demo.spi;  
5     uses QuoteService;  
6  
7     exports demo.sl;  
8 }  
9
```

On the right side of the IDE, there is a large text overlay that reads:

Java will find and bind the services providers at runtime

Consumer



Consumer

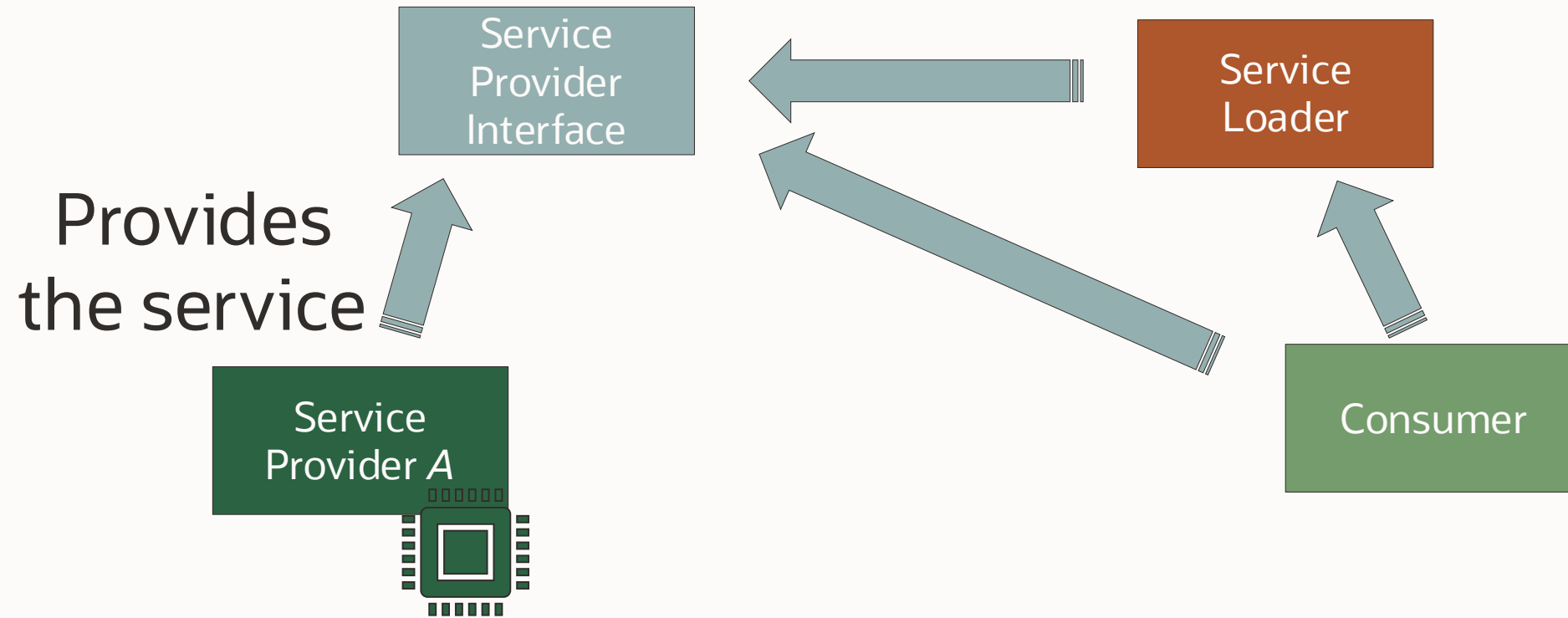
```
Project
├── service-provider-interfaces ~/Projects/alber...
│   └── app
│       ├── src
│       │   └── main
│       │       ├── java
│       │       │   └── demo.app
│       │       │       ├── Main
│       │       │       └── module-info.java
│       ├── build.gradle
│       ├── app-spring
│       ├── gradle
│       ├── quote-sl
│       ├── quote-sp-db
│       ├── quote-sp-mem
│       ├── quote-spi
│       ├── .gitattributes
│       ├── build.gradle
│       ├── gradlew
│       ├── gradlew.bat
│       ├── README.md
│       └── settings.gradle
└── External Libraries
    └── Scratches and Consoles
```

```
Main.java
3  import demo.sl.QuoteServiceLoader;
4  import demo.spi.Quote;
5  import demo.spi.QuoteService;
6
7  import java.util.Optional;
8
9  public class Main {
10     public static void main(final String[] args) {
11         final QuoteServiceLoader serviceLocator = QuoteServiceLoader.instance();
12         for (final QuoteService quoteService : serviceLocator.services()) {
13             final Optional<Quote> quote = quoteService.random();
14
15             System.out.printf("%s (Service: %s)%n",
16                               quote.map(Quote::toString).orElse("No Quote!!"),
17                               quoteService.getClass().getCanonicalName());
18         }
19     }
20 }
```

```
module-info.java (demo.app)
1  module demo.app {
2      requires demo.sl;
3  }
4
```

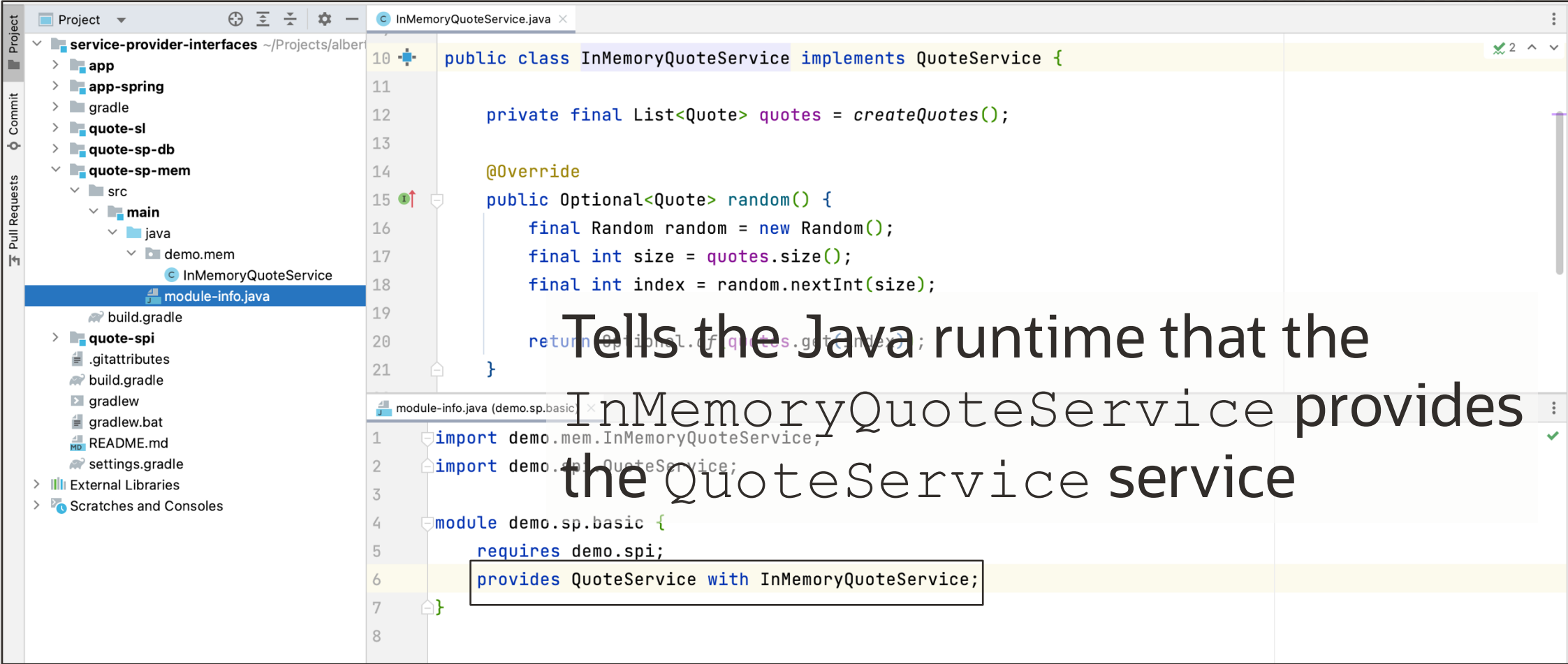
Service Provider

In Memory



Service Provider

In Memory



The screenshot shows an IDE with two files open. The top file, `InMemoryQuoteService.java`, contains the following code:

```
10 public class InMemoryQuoteService implements QuoteService {
11
12     private final List<Quote> quotes = createQuotes();
13
14     @Override
15     public Optional<Quote> random() {
16         final Random random = new Random();
17         final int size = quotes.size();
18         final int index = random.nextInt(size);
19
20         return Optional.of(quotes.get(index));
21     }
}
```

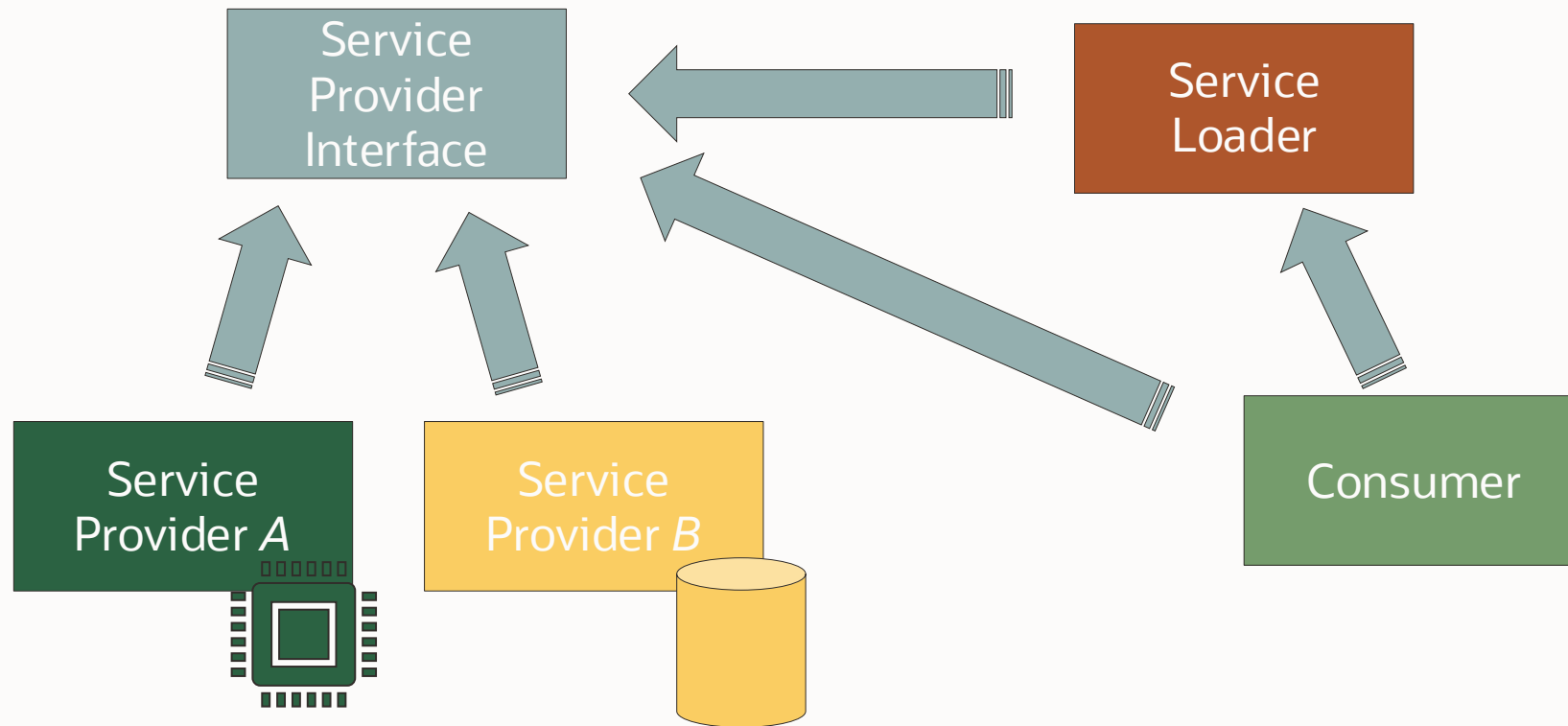
The bottom file, `module-info.java`, contains the following code:

```
1 import demo.mem.InMemoryQuoteService;
2 import demo.spi.QuoteService;
3
4 module demo.sp.basic {
5     requires demo.spi;
6     provides QuoteService with InMemoryQuoteService;
7 }
8
```

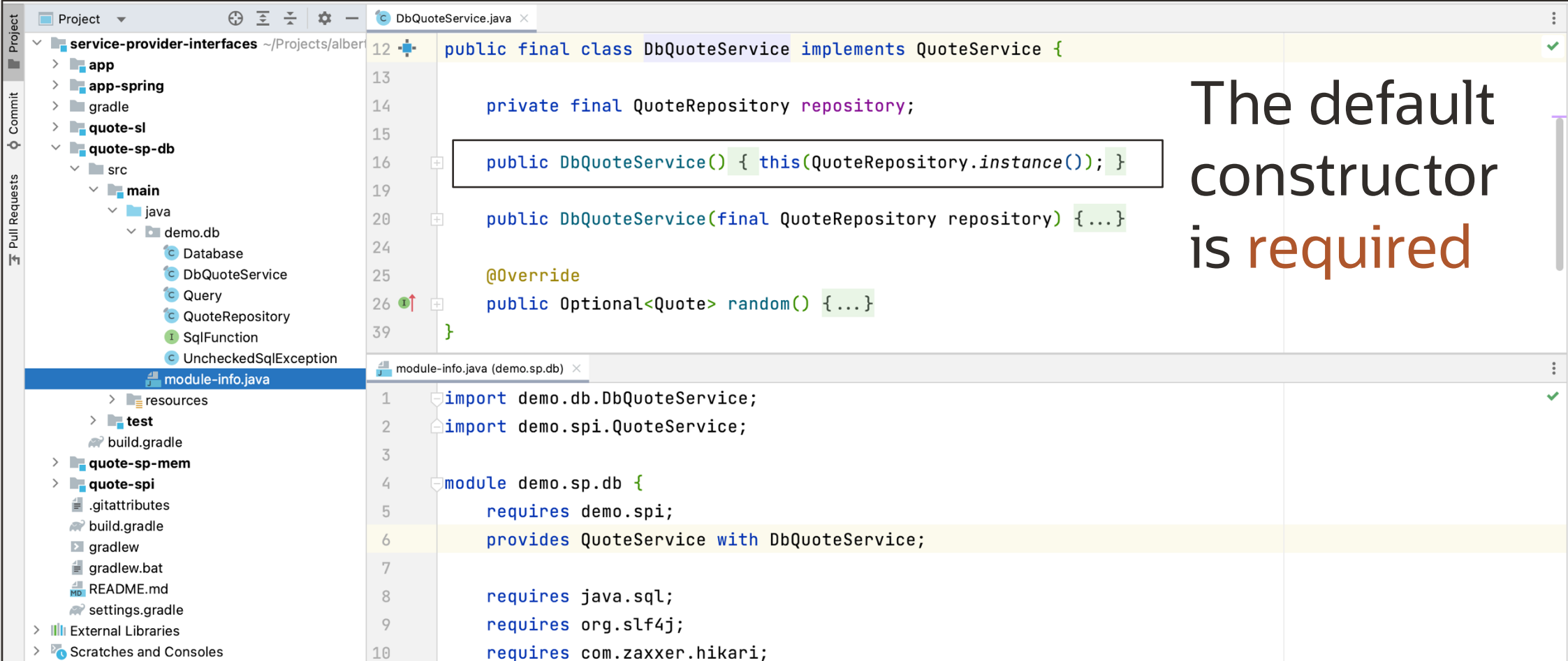
Overlaid on the code is the text: "Tells the Java runtime that the InMemoryQuoteService provides the QuoteService service". A box highlights the `provides` line in `module-info.java`.

Service Provider

Database



Service Provider Database



The screenshot displays an IDE with two open files. The left pane shows a project structure with a tree view containing folders like 'service-provider-interfaces', 'app', 'app-spring', 'quote-si', and 'quote-sp-db', along with various source files and build scripts. The main editor area shows the implementation of `DbQuoteService` in `DbQuoteService.java`, which implements the `QuoteService` interface. The code includes a private final `QuoteRepository` instance, a default constructor that initializes the repository, and a `random()` method. A text box highlights the default constructor. The bottom editor shows the `module-info.java` file for the `demo.sp.db` module, which imports the necessary classes and provides the `QuoteService` with `DbQuoteService`.

```
12 public final class DbQuoteService implements QuoteService {
13
14     private final QuoteRepository repository;
15
16     public DbQuoteService() { this(QuoteRepository.instance()); }
17
18
19     public DbQuoteService(final QuoteRepository repository) {...}
20
21
24
25     @Override
26     public Optional<Quote> random() {...}
27
28
39 }
```

```
1 import demo.db.DbQuoteService;
2 import demo.spi.QuoteService;
3
4 module demo.sp.db {
5     requires demo.spi;
6     provides QuoteService with DbQuoteService;
7
8     requires java.sql;
9     requires org.slf4j;
10    requires com.zaxxer.hikari;
```

The default constructor is required

Running the application

Using the module path

```
> java \  
  --module-path libs \  
  --module demo.app
```

```
... (Service: demo.db.DbQuoteService)  
... (Service:  
demo.mem.InMemoryQuoteService)
```

Spring Framework

The screenshot displays an IDE interface with the following components:

- Project Structure (Left Sidebar):**
 - Project: `~/Projects/alber`
 - Modules:
 - `service-provider-interfaces`
 - `app`
 - `app-spring`
 - `src`
 - `main`
 - `java`
 - `demo.spring`
 - `Demo`
 - `Main`
 - `ServiceConfiguration`
 - Files:
 - `resources`
 - `build.gradle`
 - `gradle`
 - `quote-si`
 - `quote-sp-db`
 - `quote-sp-mem`
 - `quote-spi`
 - `.gitattributes`
 - `build.gradle`
 - `gradlew`
 - `gradlew.bat`
 - `README.md`
 - `settings.gradle`
 - External Libraries
 - Scratches and Consoles

- Code Editor (Main Area):**
- `Demo.java` (lines 12-20):

```
12 @Component
13 public class Demo implements CommandLineRunner {
14
15     private final List<QuoteService> quoteServices;
16
17     public Demo(final @Qualifier("quoteServices") List<QuoteService> quoteServices) {...}
18
19
20
```
- `ServiceConfiguration.java` (lines 8-16):

```
8 @Configuration
9 public class ServiceConfiguration {
10
11     @Bean
12     public ServiceListFactoryBean quoteServices() {
13         final ServiceListFactoryBean serviceListFactoryBean = new ServiceListFactoryBean();
14         serviceListFactoryBean.setServiceType(QuoteService.class);
15         return serviceListFactoryBean;
16     }
17
```
- `module-info.java (demo.spring)` (lines 3-6):

```
3 module demo.spring {
4     requires demo.spi;
5     uses QuoteService;
6
```

Demo

service-provider-interfaces

Migration

Terminology

Terminology

Platform modules

- Modules that make up the JDK
- These are defined by the Java SE Platform Specification (prefixed with *java.*) as well as JDK-specific modules (prefixed with *jdk.*)

```
> java --list-modules  
java.base@25.0.1  
java.compiler@25.0.1  
...
```

Terminology

Incubator modules

- Non-finalised platform modules whose names always start with *jdk.incubator*
- They contain experimental APIs available to early adopters

```
> java --list-modules | grep jdk.incubator  
jdk.incubator.foreign@25.0.1  
jdk.incubator.vector@25.0.1
```

Terminology

Base module

- All modules are the same to the Java Module System, except one: the platform module *java.base*
- The base module is analogous to the *java.lang* package
 - no need to include it as it is always included for you

```
> jar --describe-module --file app.jar  
requires java.base mandated  
contains demo  
main-class demo.Main
```

Terminology

Application modules

- A non-JDK module
- The modules created by Java developers and used for their own projects, including libraries and frameworks

Terminology

Initial module

- Application module where compilation starts (for *javac*) or containing the *main()* method (for *java*)

```
> java \  
  --module-path app.jar:domain.jar \  
  --module demo.app
```

Terminology

Observable modules

- All platform modules in the current runtime as well as all application modules specified on the command line
- The modules that the Java Module System can use to fulfil dependencies

```
> java --module-path libs --list-modules  
java.base@25.0.1  
...  
demo.app file:///.../libs/app.jar
```

Terminology

Root modules

- Where the Java Module System starts resolving dependencies
- In addition to containing the *main()* class, the initial module is also a root module
- Modules defined by the *--add-modules* are also part of the root modules

```
> java --module-path libs \  
    --add-modules demo.formatter \  
    --module demo.app
```

How are modules resolved?

—
The application module graph

Module resolution

Application



```
module demo.app {  
  requires demo.domain;  
}
```

```
module demo.domain {  
  exports demo.domain;  
}
```

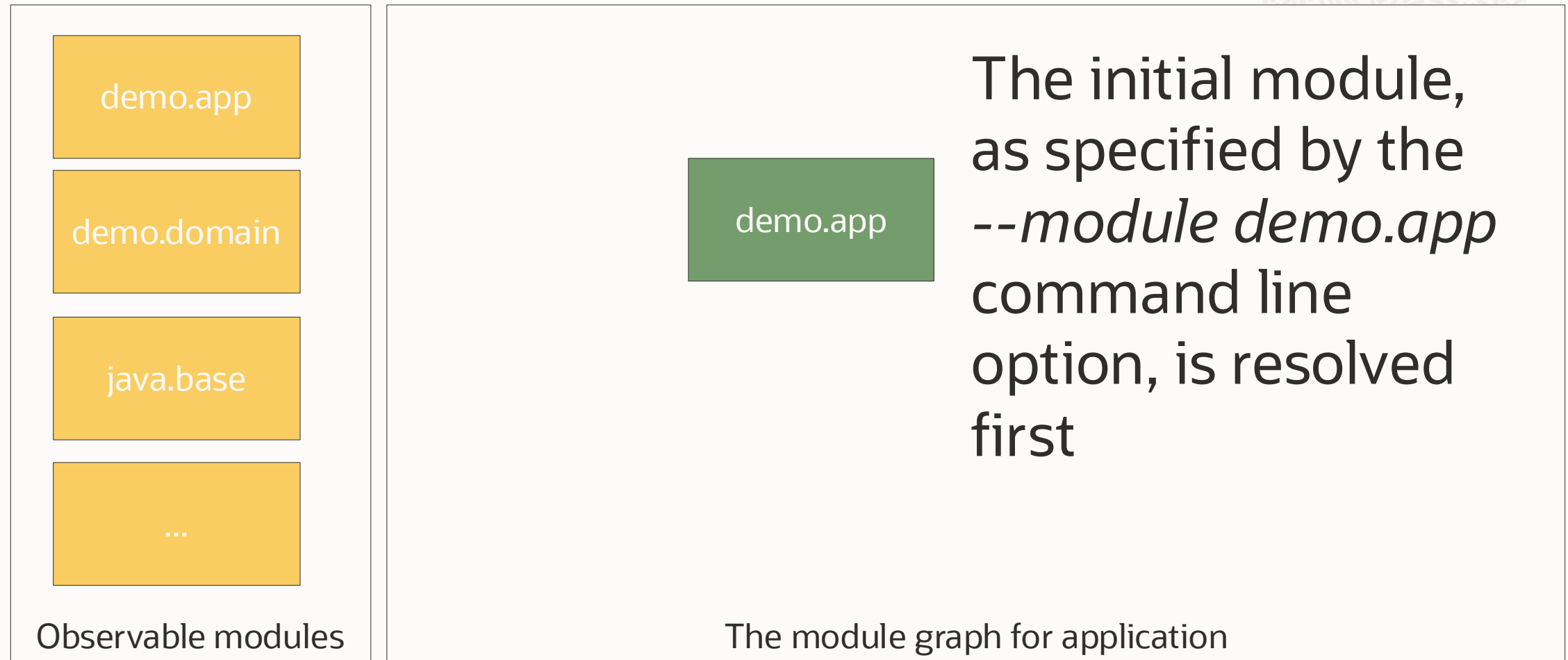
Module resolution

Run the application using the module path

```
> java \  
  --module-path app.jar:domain.jar \  
  --module demo.app
```

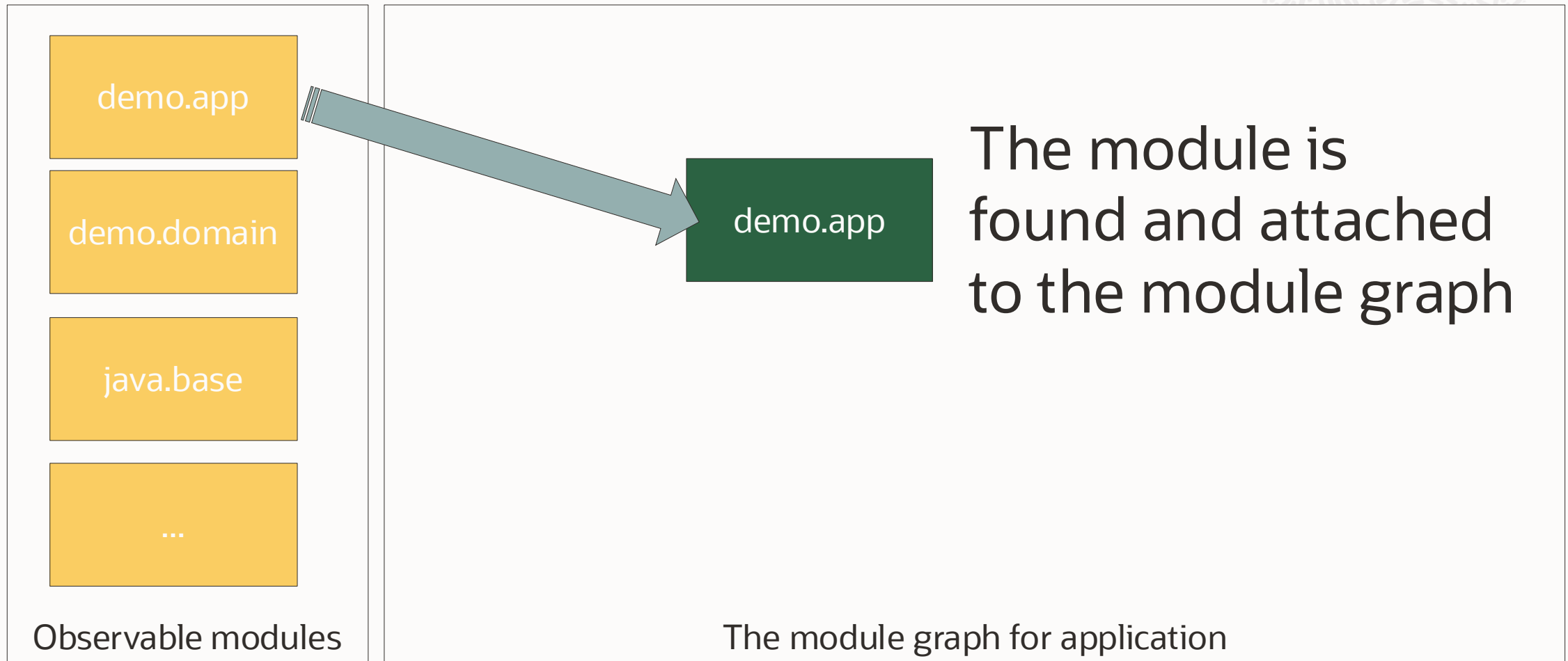
Module resolution

Module graph



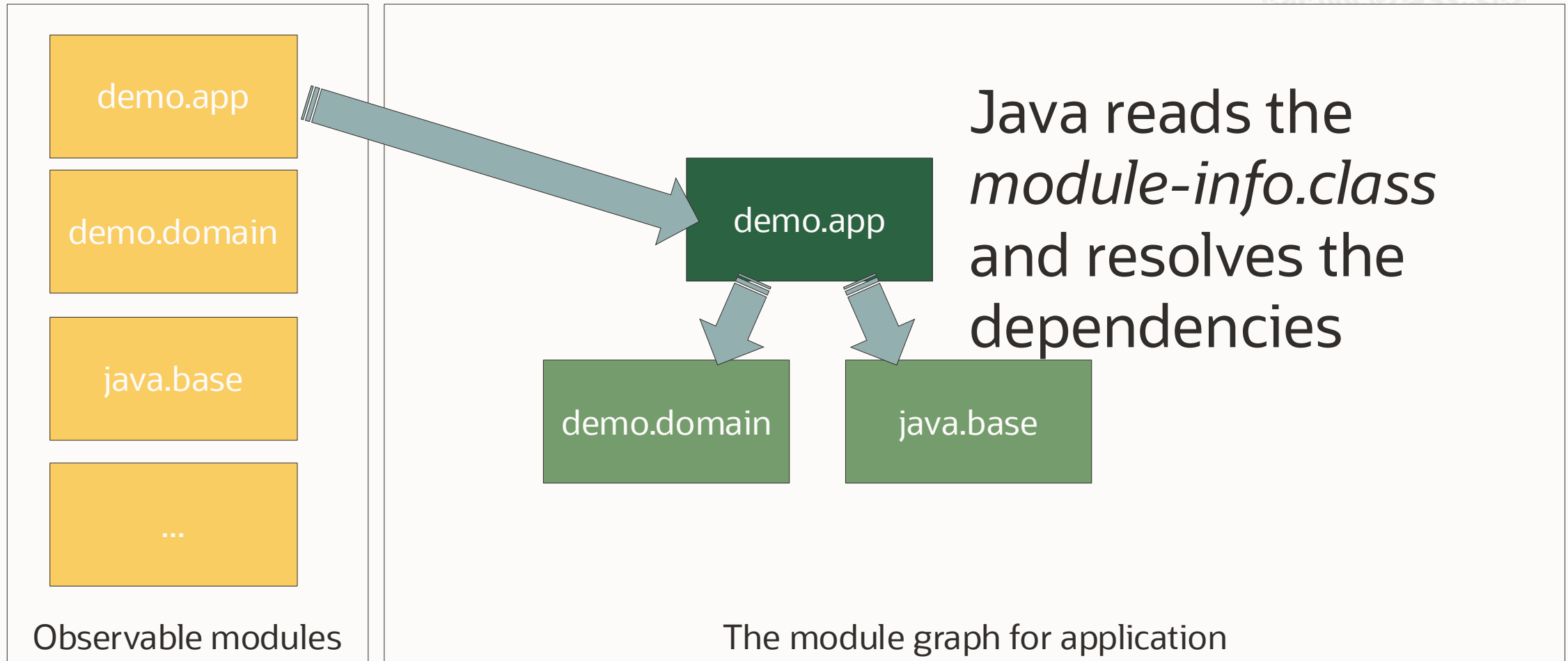
Module resolution

Module graph



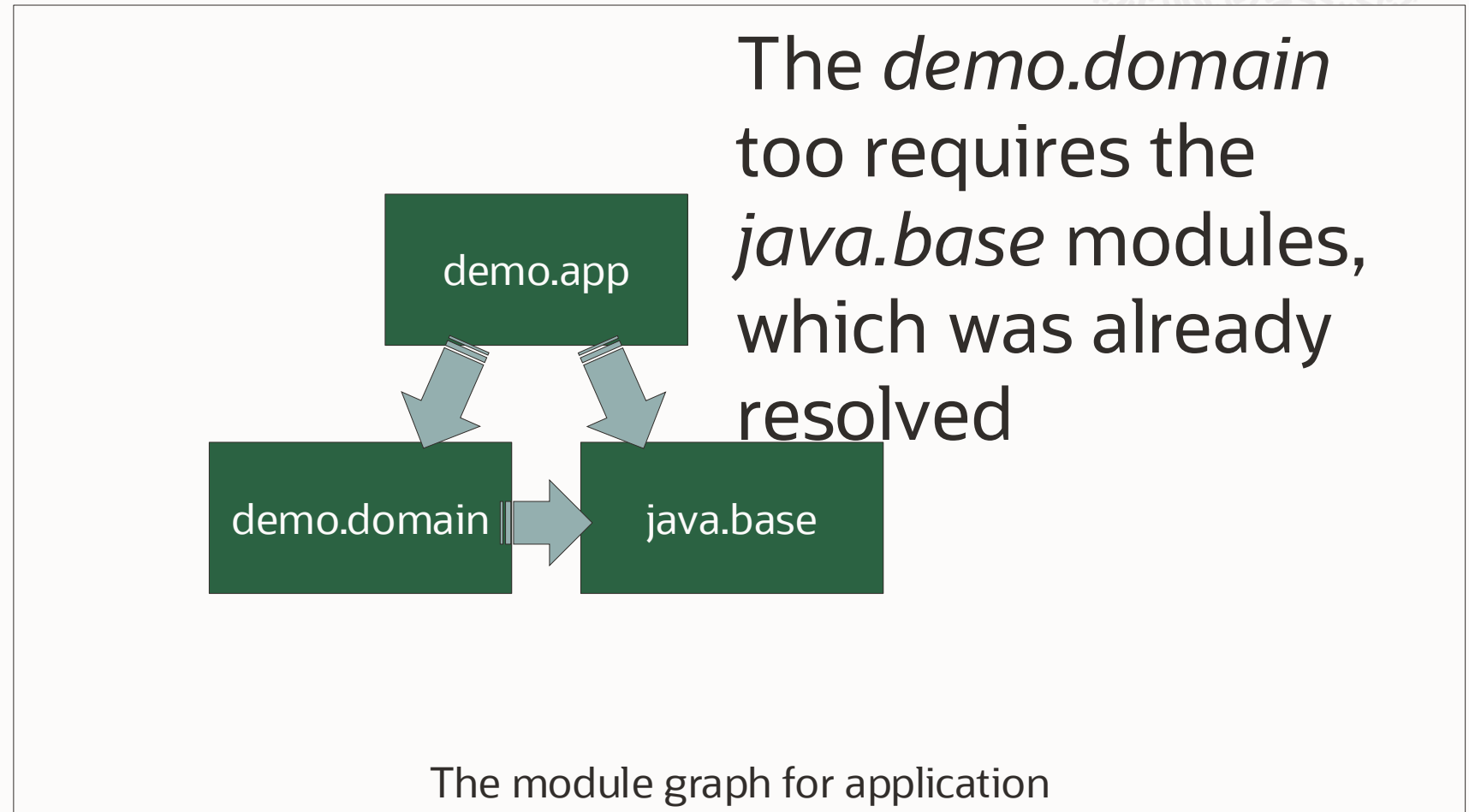
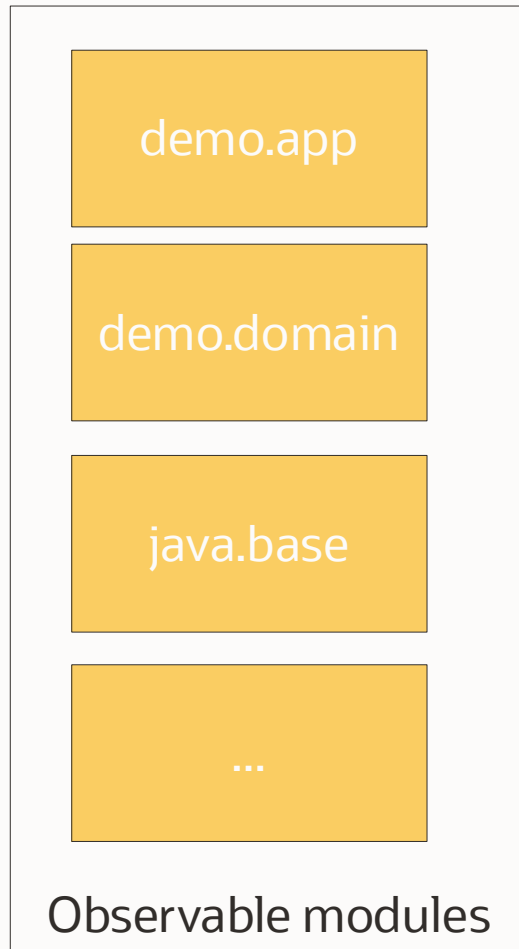
Module resolution

Module graph



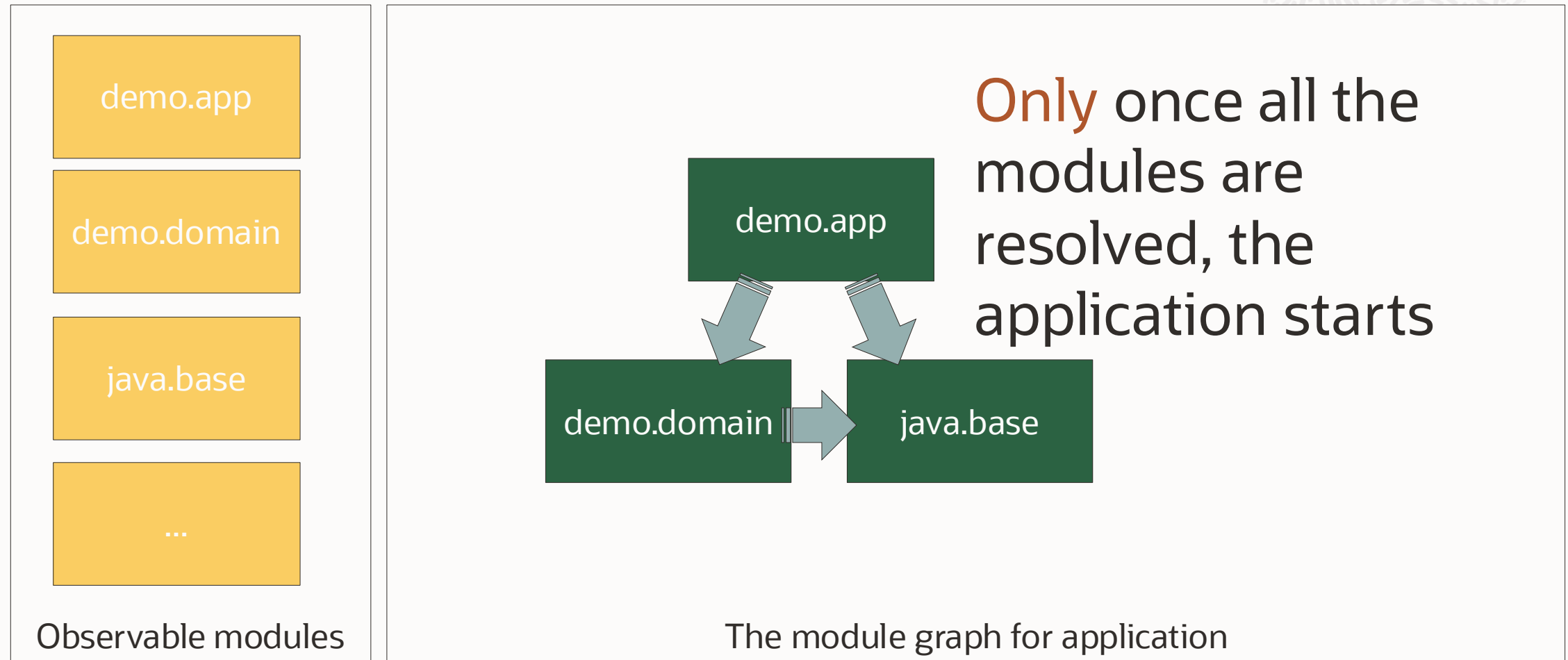
Module resolution

Module graph



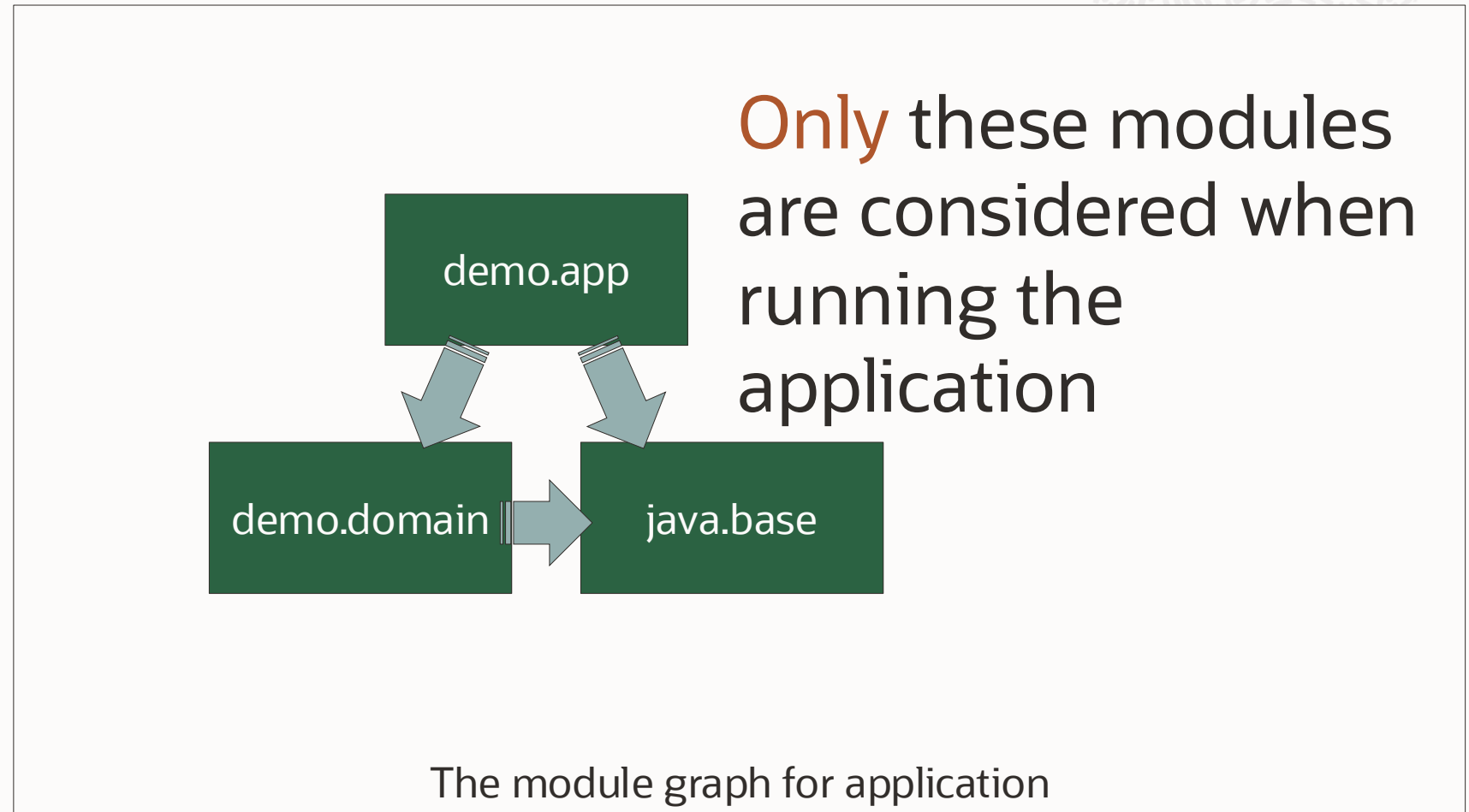
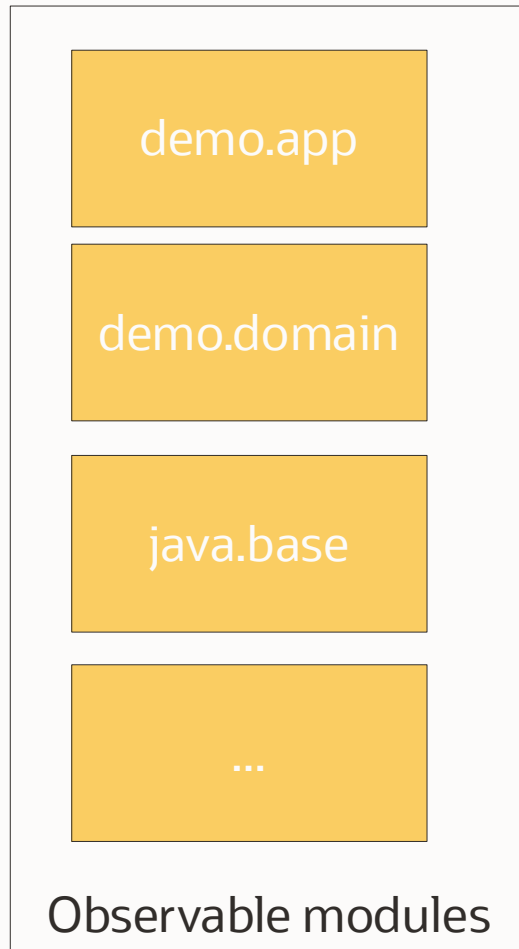
Module resolution

Module graph



Module resolution

Module graph



What happens if a module is not found?

Module not found

```
> java --module demo.unknown.module
```

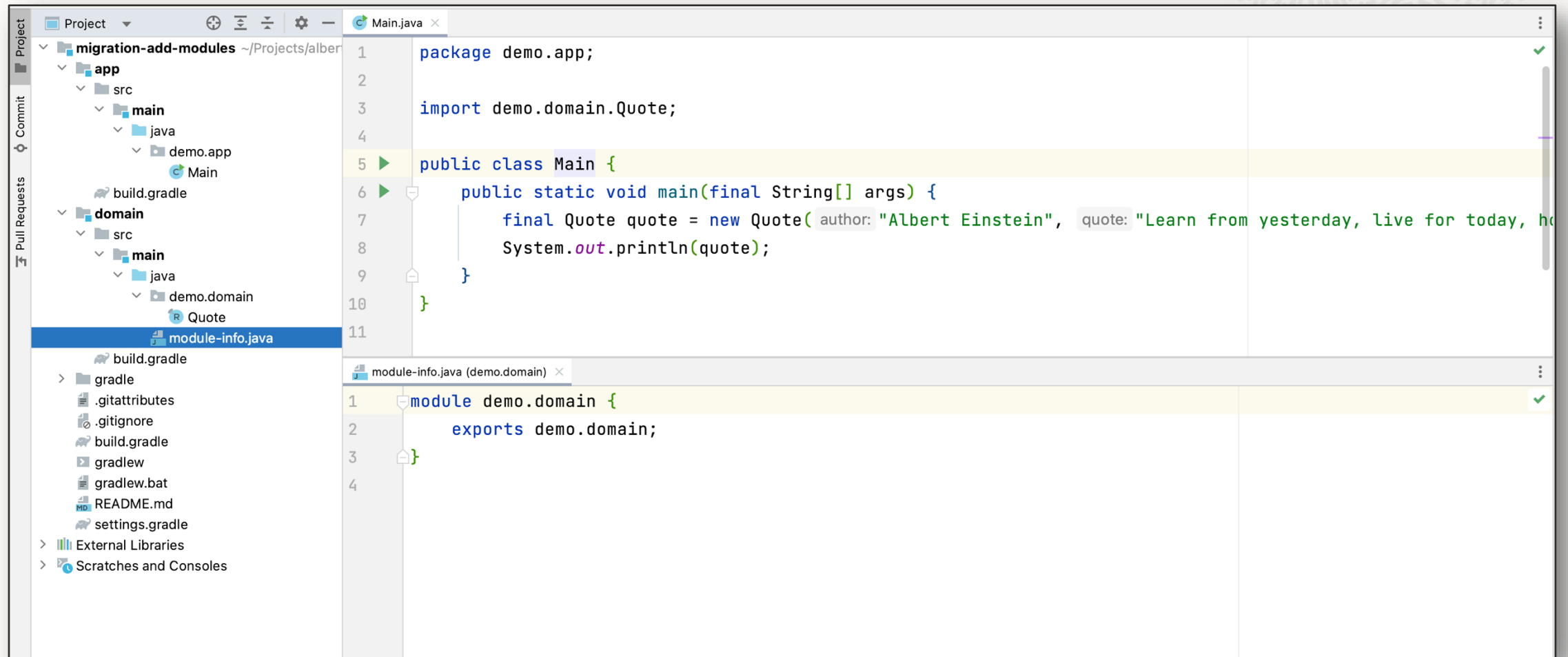
Assuming that the *demo.unknown.module*
does **not** exist

```
Error occurred during initialization of  
boot layer  
java.lang.module.FindException: Module  
demo.unknown.module not found
```

How does Java find the dependencies of *automatic* and *unnamed* modules?

--add-modules

Initial *automatic* module



The screenshot displays an IDE interface with the following components:

- Project View (Left):** Shows a project named "migration-add-modules" with a directory structure including "app", "domain", and "module-info.java".
- Main.java (Top Editor):**

```
1 package demo.app;  
2  
3 import demo.domain.Quote;  
4  
5 public class Main {  
6     public static void main(final String[] args) {  
7         final Quote quote = new Quote( author: "Albert Einstein", quote: "Learn from yesterday, live for today, h  
8         System.out.println(quote);  
9     }  
10 }  
11
```
- module-info.java (demo.domain) (Bottom Editor):**

```
1 module demo.domain {  
2     exports demo.domain;  
3 }  
4
```

List modules

```
> java --module-path libs --list-modules
```

```
java.base@25.0.1  
java.compiler@25.0.1  
java.datatransfer@25.0.1  
java.desktop@25.0.1  
...  
demo.app file:///.../app.jar automatic  
demo.domain file:///.../domain.jar
```

Running the application

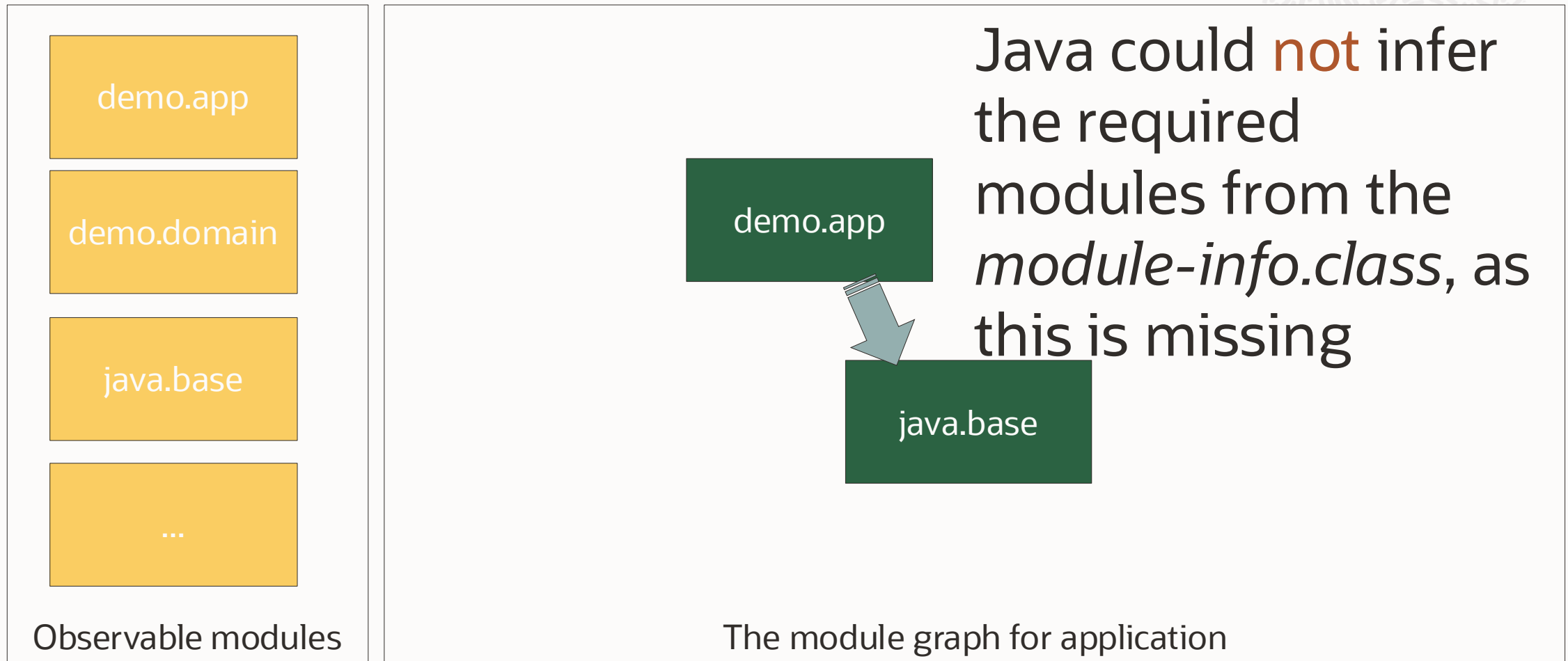
Using the module path

```
> java \  
  --module-path app.jar:domain.jar \  
  --module demo.app
```

```
Exception in thread "main"  
java.lang.NoClassDefFoundError:  
demo/domain/Quote  
    at  
demo.app/demo.app.Main.main(Main.java:7)
```

Module resolution

Module graph



Running the application

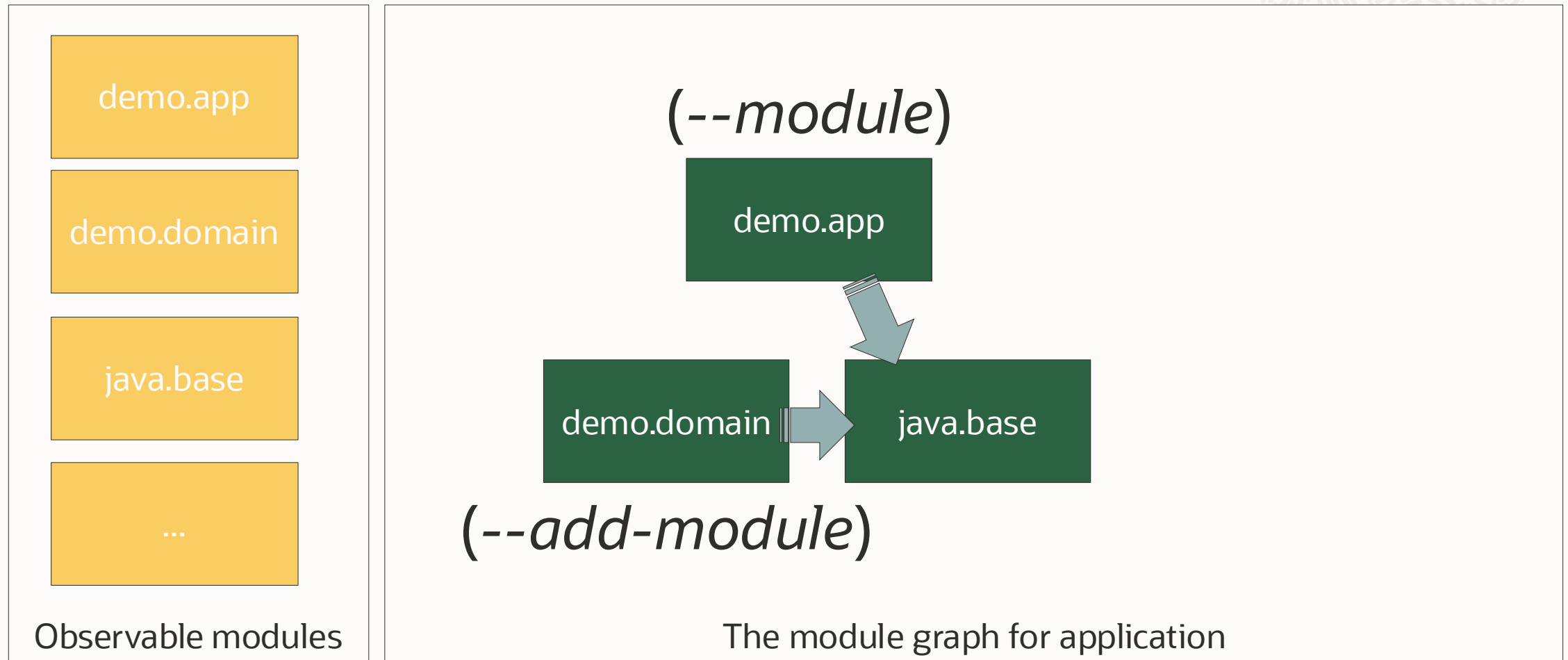
Using the module path and the *--add-modules*

```
> java \  
  --module-path app.jar:domain.jar \  
  --add-modules demo.domain \  
  --module demo.app
```

Works!! (*Output omitted for brevity*)

Module resolution

Module graph using *--add-modules*



How does it work with optional modules?

--add-modules

Module resolution

Application

```
module demo.app {  
  requires demo.domain;  
  requires static demo.formatter;  
}
```

```
module demo.domain {  
  exports demo.domain;
```

```
module demo.formatter {  
  requires demo.domain;  
  exports demo.formatter;  
}
```

List modules

```
> java --module-path libs --list-modules
```

```
java.base@25.0.1  
java.compiler@25.0.1  
java.datatransfer@25.0.1  
...  
demo.app file:///.../app.jar  
demo.domain file:///.../domain.jar  
demo.formatter file:///.../formatter.jar
```

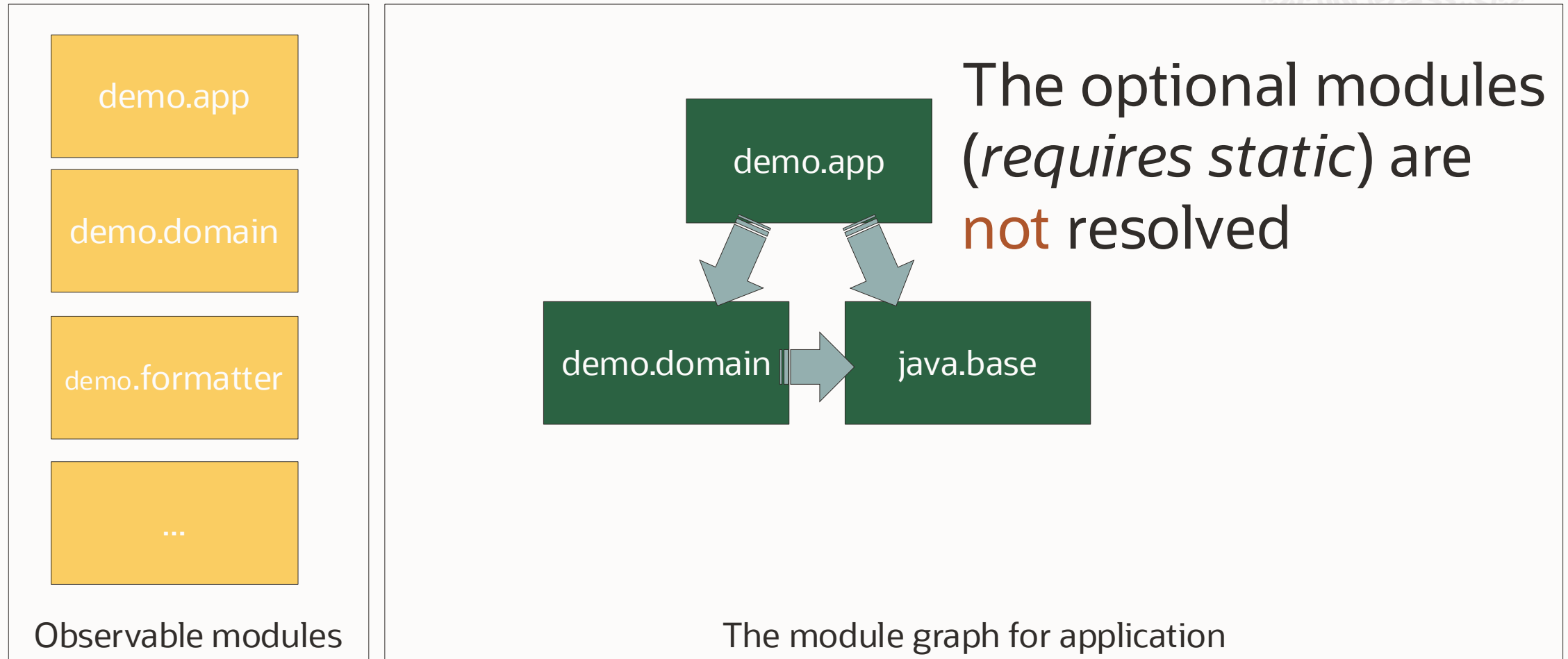
Running the application

Using the module path

```
> java \  
  --module-path libs \  
  --module demo.app
```

Module resolution

Module graph



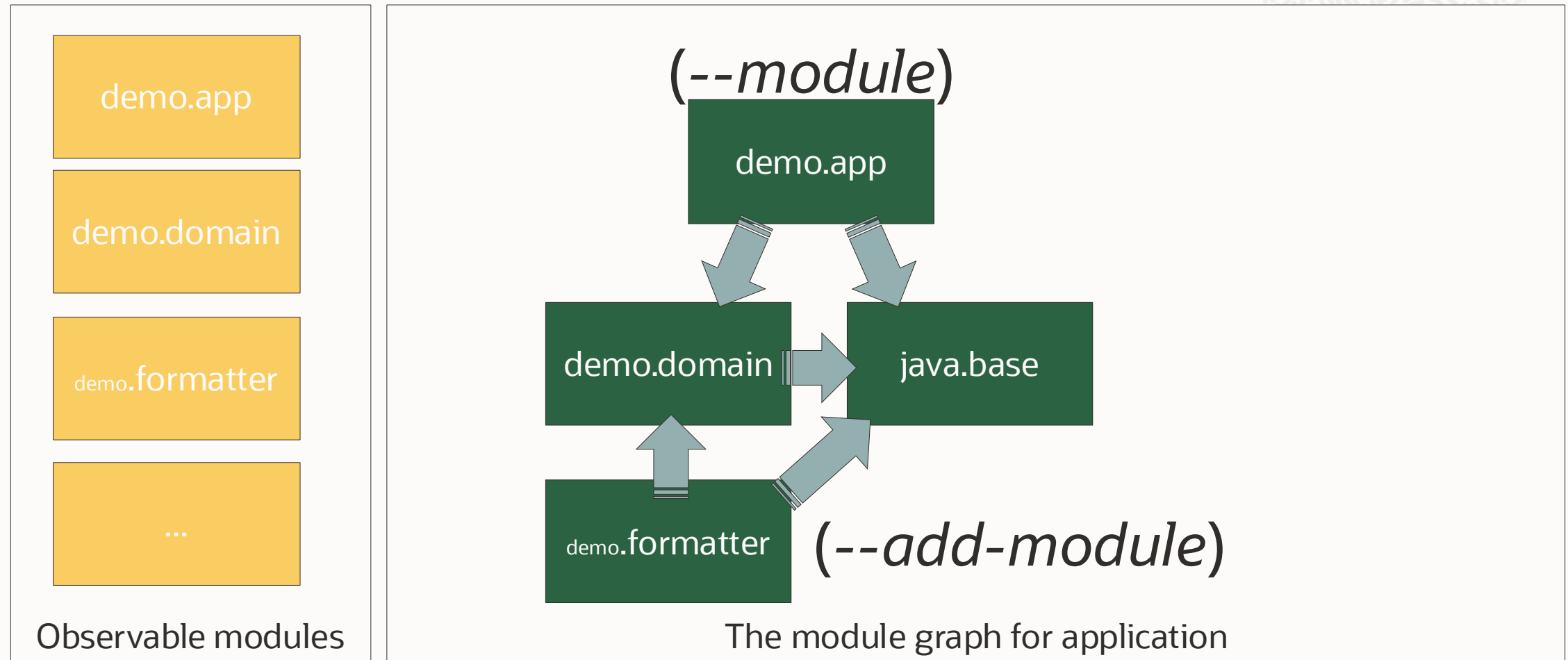
Running the application

Using the module path and the *--add-modules*

```
> java \  
  --module-path libs \  
  --add-modules demo.formatter \  
  --module demo.app
```

Module resolution

Module graph



What happens if an optional module is not an observable module?

Missing optional modules



Dependency available

	Compile Time	Runtime
<i>requires</i>	reads	reads
<i>requires static</i>	reads	reads

Dependency missing

	Compile Time	Runtime
<i>requires</i>	fails	fails
<i>requires static</i>	fails	ignores

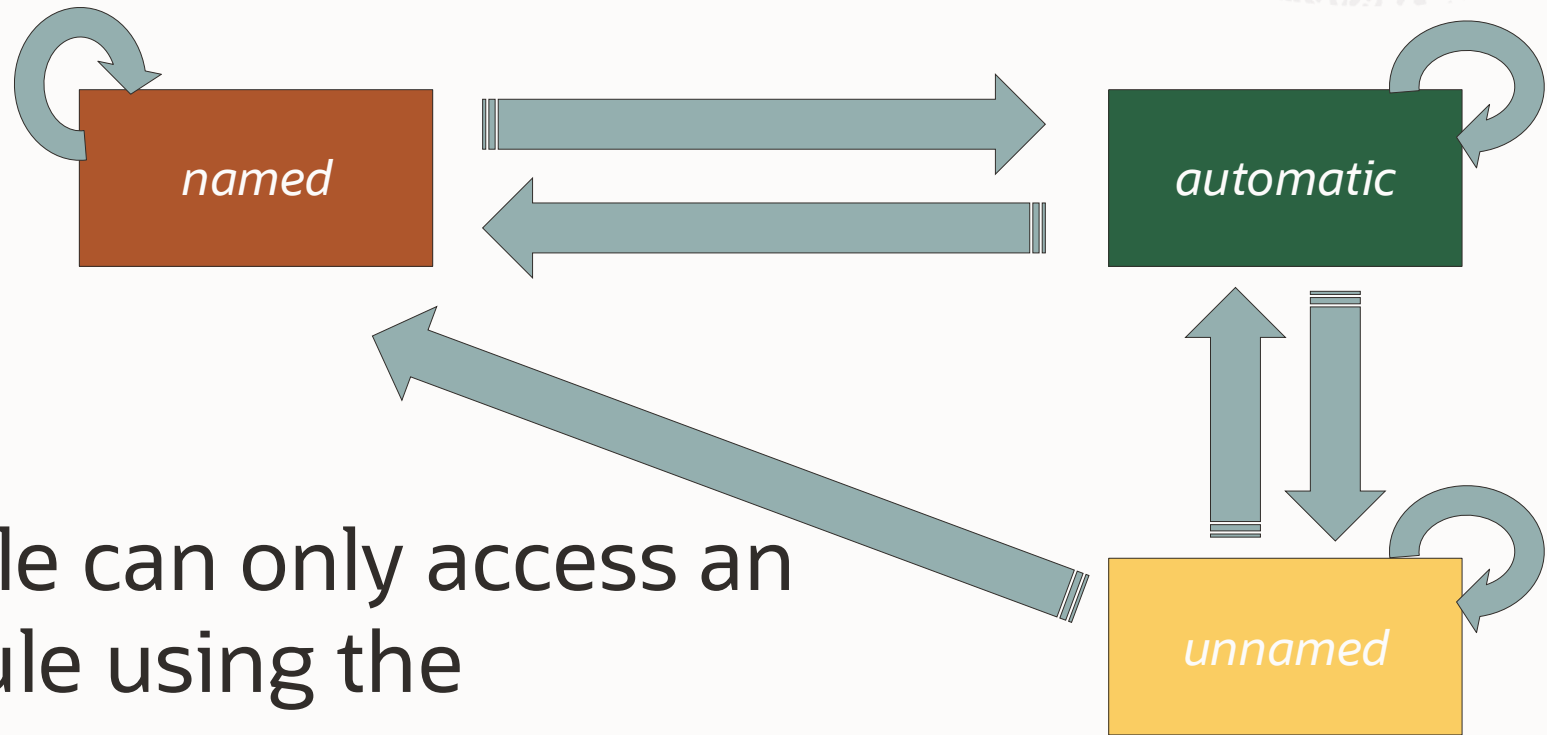
Optional modules
are required at
compile time and
optional at runtime



What is the difference between the types of modules?

—
Revisited

Difference between module types



A named module can only access an unnamed module using the `--add-reads <module>=ALL-UNNAMED` command line option.

automatic vs. *named* modules

- *named* module has a *module-info.java* file
 - cannot access *unnamed* module
 - can be overridden by command line options
- *automatic* module does not have a *module-info.java* file
 - automatically exports all its packages
 - automatically reads all available modules (both *named* and the *unnamed* module)

Access Modifiers

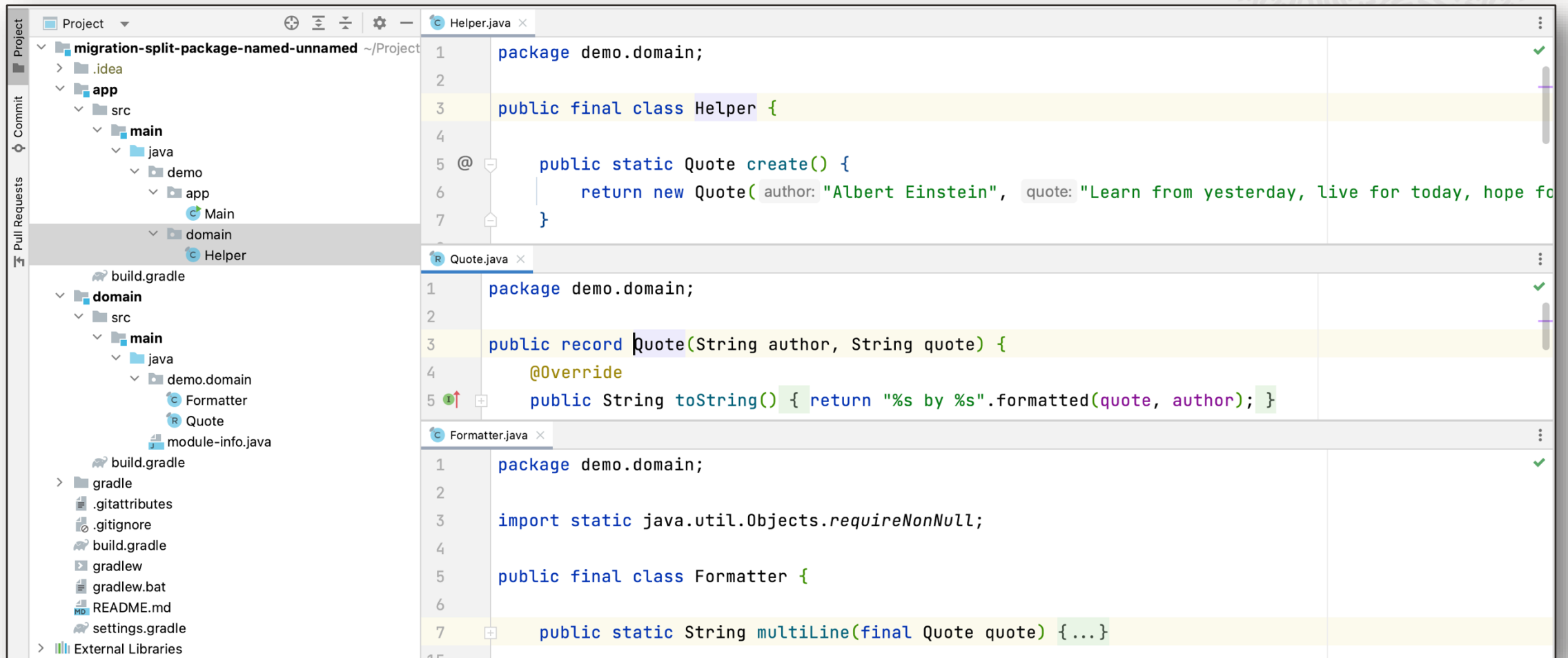


Access Modifier	Accessible from (<i>--class-path</i>)
<code>private</code>	Enclosing top level class
<i>package-private</i>	Anywhere within the same package (even from different JAR files)
<code>protected</code>	Anywhere within the same package and subclasses (even from different JAR files)
<code>public</code>	Anywhere

Access Modifier	Accessible from (<i>--module-path</i>)
<code>private</code>	Enclosing top level class
<i>package-private</i>	Anywhere within the same package not across modules
<code>protected</code>	Anywhere within the same package and subclasses not across modules
<code>public</code>	Anywhere within the same module and to where this is exported



What happens if the same package is found both in a *named* (or *automatic*) module and in the *unnamed* module?



List modules

```
> java --module-path libs --list-modules
```

```
java.base@25.0.1  
java.compiler@25.0.1  
java.datatransfer@25.0.1  
...  
demo.app file:///.../app.jar automatic  
demo.domain file:///.../domain.jar
```

Describe module

```
> jar --describe-module --file app.jar
No module descriptor found. Derived
automatic module.

demo.app automatic
requires java.base mandated
contains demo.app
contains demo.domain
main-class demo.app.Main
```

Running the application

Using the module path and class path

```
> java \  
  --class-path app.jar \  
  --module-path domain.jar \  
  --add-modules demo.domain \  
  demo.app.Main
```

Running the application

Using the module path and class path

```
Exception in thread "main"  
java.lang.NoClassDefFoundError:  
demo/domain/Helper  
    at demo.app.Main.main(Main.java:9)  
Caused by:  
java.lang.ClassNotFoundException:  
demo.domain.Helper  
...
```

When a package is found in an *unnamed* module and a *named* (or *automatic*) module, the one in the *unnamed* module is ignored

Running the application

Using the class path

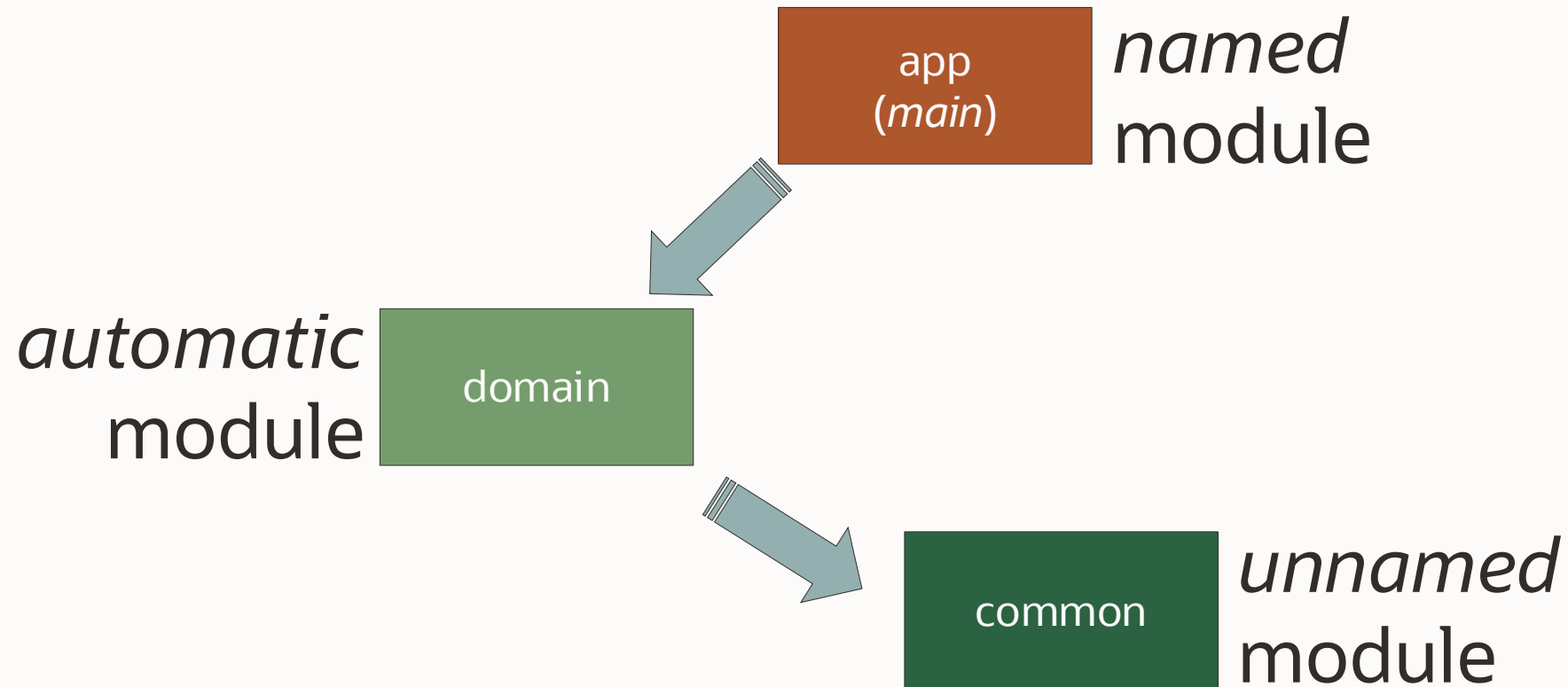
```
> java \  
  --class-path app.jar:domain.jar \  
  demo.app.Main
```

```
"Learn from yesterday, live for today,  
hope for tomorrow. The important thing is  
not to stop questioning."  
by Albert Einstein
```

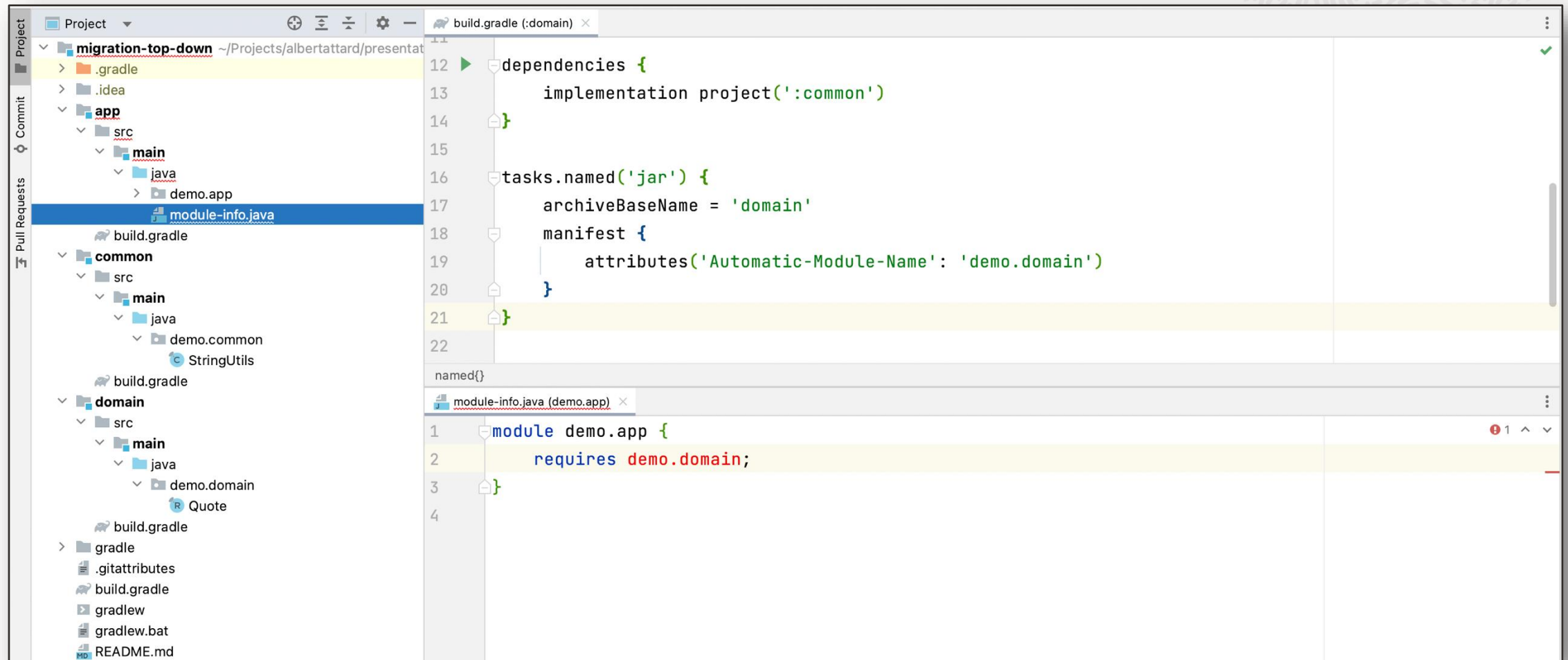
Top-Down-Migration



Top-Down-Migration



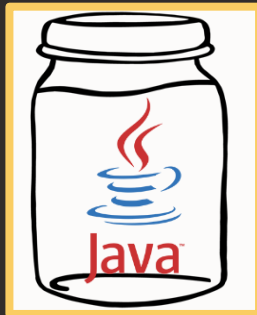
Top-Down-Migration



Top-Down-Migration

```
> java \  
  --class-path common.jar \  
  --module-path domain.jar:app.jar \  
  --module demo.app
```

Unnamed



common.jar

Automatic



domain.jar

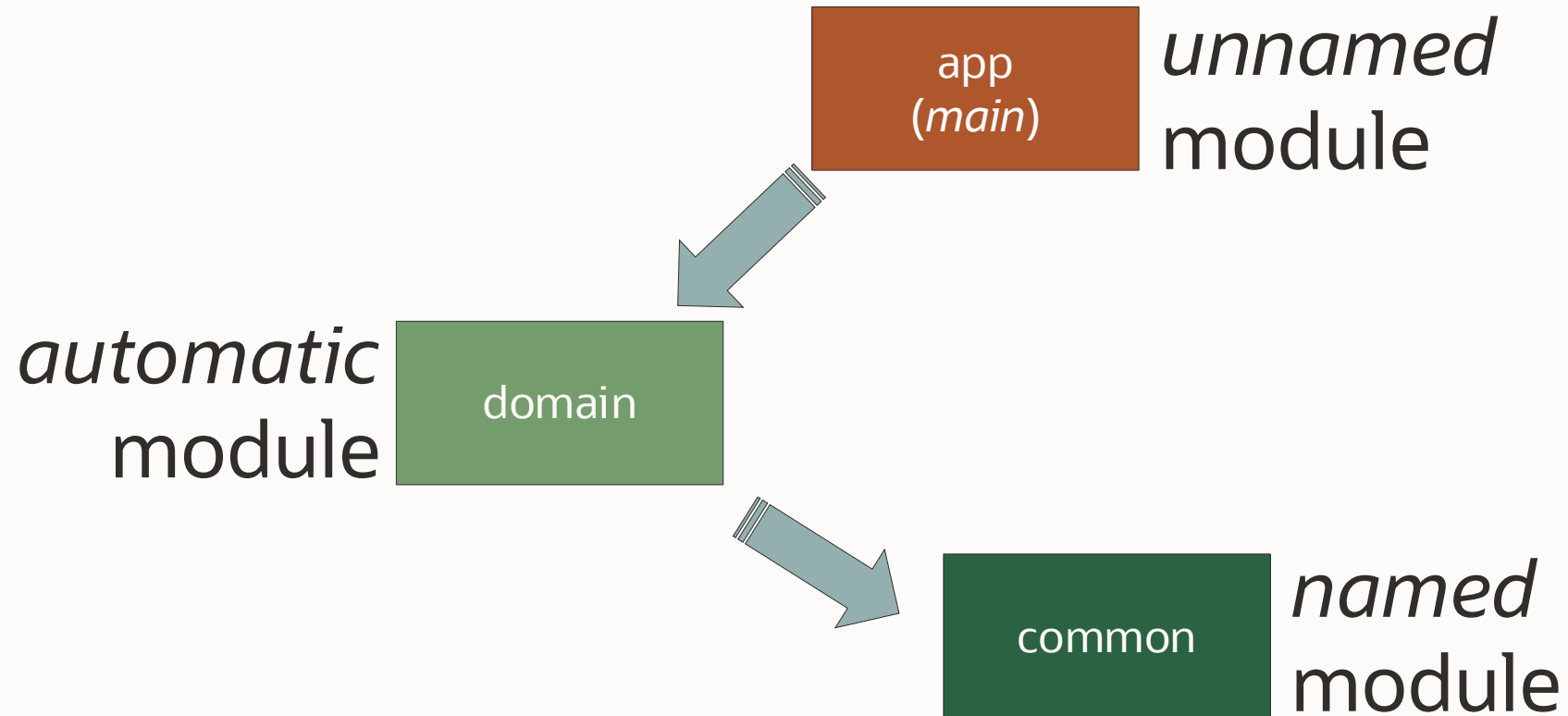
Named



app.jar

Bottom-Up-Migration

Bottom-Up-Migration



Bottom-Up-Migration

The screenshot shows an IDE with a project named **migration-bottom-up**. The project structure in the left sidebar is as follows:

- app**
 - src**
 - main**
 - java**
 - demo.app**
 - Main
 - build.gradle**
- common**
 - src**
 - main**
 - java**
 - demo.common**
 - module-info.java
 - build.gradle**
- domain**
 - src**
 - main**
 - java**
 - demo.domain**
 - Quote
 - build.gradle**
- gradle**
- .gitattributes**
- build.gradle**
- gradlew**
- gradlew.bat**
- README.md**
- settings.gradle**
- External Libraries**

The main editor displays the **build.gradle (:domain)** file with the following code:

```
10
11
12 dependencies {
13     implementation project(':common')
14 }
15
16 tasks.named('jar') {
17     archiveBaseName = 'domain'
18     manifest {
19         attributes('Automatic-Module-Name': 'demo.domain')
20     }
21 }
22
```

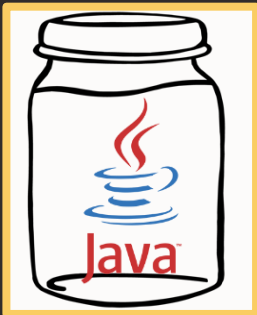
Below it, the **module-info.java (demo.common)** file is shown with the following code:

```
1 module demo.common {
2     exports demo.common;
3 }
4
```

Bottom-Up-Migration

```
> java --class-path app.jar \  
    --module-path domain.jar:common.jar \  
    --add-modules demo.domain,demo.common \  
    demo.app.Main
```

Unnamed



app.jar

Automatic



domain.jar

Named



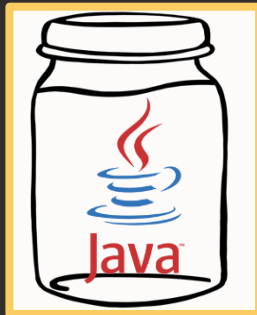
common.jar

Bottom-Up-Migration

ALL-MODULE-PATH

```
> java --class-path app.jar \  
    --module-path domain.jar:common.jar \  
    --add-modules ALL-MODULE-PATH \  
    demo.app.Main
```

Unnamed



app.jar

Automatic



domain.jar

Named



common.jar

The following are **temporary** solutions and should be replaced with a proper solution as soon as possible

Can a module be *required* at runtime?

--add-reads



Application



app

```
module demo.app { }
```



domain

```
module demo.domain {  
  exports demo.domain;  
}
```

Application

The screenshot displays an IDE interface for a Java application. The left sidebar shows a project tree for 'migration-add-reads' with a multi-module structure. The main editor area shows two Java files: 'Main.java' and 'Quote.java'. 'Main.java' is in the 'demo.app' module and imports 'Quote' from 'demo.domain'. It contains a 'Main' class with a 'main' method that creates a 'Quote' object and prints its string representation. 'Quote.java' is in the 'demo.domain' module and defines a 'Quote' record with a 'toString()' method that returns a formatted string. Below the main editor, two 'module-info.java' files are shown: one for 'demo.app' which exports 'demo.domain', and one for 'demo.domain'.

```
1 package demo.app;
2
3 import demo.domain.Quote;
4
5 public class Main {
6     public static void main(final String[] args) {
7         final Quote quote = new Quote( author: "Albert
8         System.out.println(quote);
9     }
10 }
11
```

```
1 package demo.domain;
2
3 public record Quote(String author, String quote) {
4     @Override
5     public String toString() { return "%s by %s".form
6 }
7
```

```
1 module demo.app {}
2
```

```
1 module demo.domain {
2     exports demo.domain;
3 }
4
```

Describe module

```
> jar --describe-module --file domain.jar
```

```
demo.domain
```

```
jar:file:///.../domain.jar!/module-  
info.class
```

```
exports demo.domain
```

```
requires java.base mandated
```

Describe module

```
> jar --describe-module --file app.jar
```

```
demo.app jar:file:///.../app.jar!/module-  
info.class
```

```
requires java.base mandated
```

```
contains demo.app
```

```
main-class demo.app.Main
```

There is no
requires demo.domain!

Running the application

Using the module path

```
> java --module-path app.jar:domain.jar \  
    --module demo.app
```

```
Exception in thread "main"  
java.lang.NoClassDefFoundError:  
demo/domain/Quote  
...
```

Running the application

Using the module path and *--add-reads*

```
> java --module-path app.jar:domain.jar \  
    --add-reads demo.app=demo.domain \  
    --module demo.app
```

```
WARNING: Unknown module: demo.domain  
specified to --add-reads  
...
```

Running the application

Using the module path and *--add-reads* and *--add-modules*

```
> java --module-path app.jar:domain.jar \  
  --add-reads demo.app=demo.domain \  
  --add-modules demo.domain \  
  --module demo.app
```

Unnamed

Automatic

Named



Can a module *export* a package at runtime?

--add-exports

Application

app

domain

```
module demo.domain { }
```

```
module demo.app {  
    requires demo.domain;  
}
```

Application

The screenshot displays an IDE interface for a Java project named 'migration-add-exports'. The project structure is as follows:

- Project
 - app
 - src
 - main
 - java
 - demo.app
 - Main
 - module-info.java
 - domain
 - src
 - main
 - java
 - demo.domain
 - Quote
 - module-info.java

The main editor shows the following code:

Main.java

```
1 package demo.app;
2
3 import demo.domain.Quote;
4
5 public class Main {
6     public static void main(final String[] args) {
7         final Quote quote = new Quote("Albert", "Einstein");
8         System.out.println(quote);
9     }
10 }
11
```

Quote.java

```
1 package demo.domain;
2
3 public record Quote(String author, String quote) {
4     @Override
5     public String toString() { return "%s by %s"; }
6 }
```

The bottom editor shows the module-info.java files:

module-info.java (demo.app)

```
1 module demo.app {
2     requires demo.domain;
3 }
4
```

module-info.java (demo.domain)

```
1 module demo.domain {}
2
```

Describe module

```
> jar --describe-module --file domain.jar
```

```
demo.domain
```

```
jar:file:///.../domain.jar!/module-  
info.class
```

```
requires java.base mandated
```

There is no
exports demo.domain!

Describe module

```
> jar --describe-module --file app.jar
```

```
demo.app jar:file:///.../app.jar!/module-  
info.class
```

```
requires demo.domain
```

```
requires java.base mandated
```

```
contains demo.app
```

```
main-class demo.app.Main
```

Running the application

Using the module path

```
> java \  
    --module-path app.jar:domain.jar \  
    --module demo.app
```

Running the application

Using the module path

```
Exception in thread "main"  
java.lang.IllegalAccessError: class  
demo.app.Main (in module demo.app) cannot  
access class demo.domain.Quote (in module  
demo.domain) because module demo.domain  
does not export demo.domain to module  
demo.app  
...
```

Running the application

Using the module path

```
> java --module-path app.jar:domain.jar \  
    --add-exports \  
demo.domain/demo.domain=demo.app \  
    --module demo.app
```

Unnamed

Automatic

Named



Can a module use reflection on a package at runtime?

--add-opens

Application



app

```
module demo.app { }
```

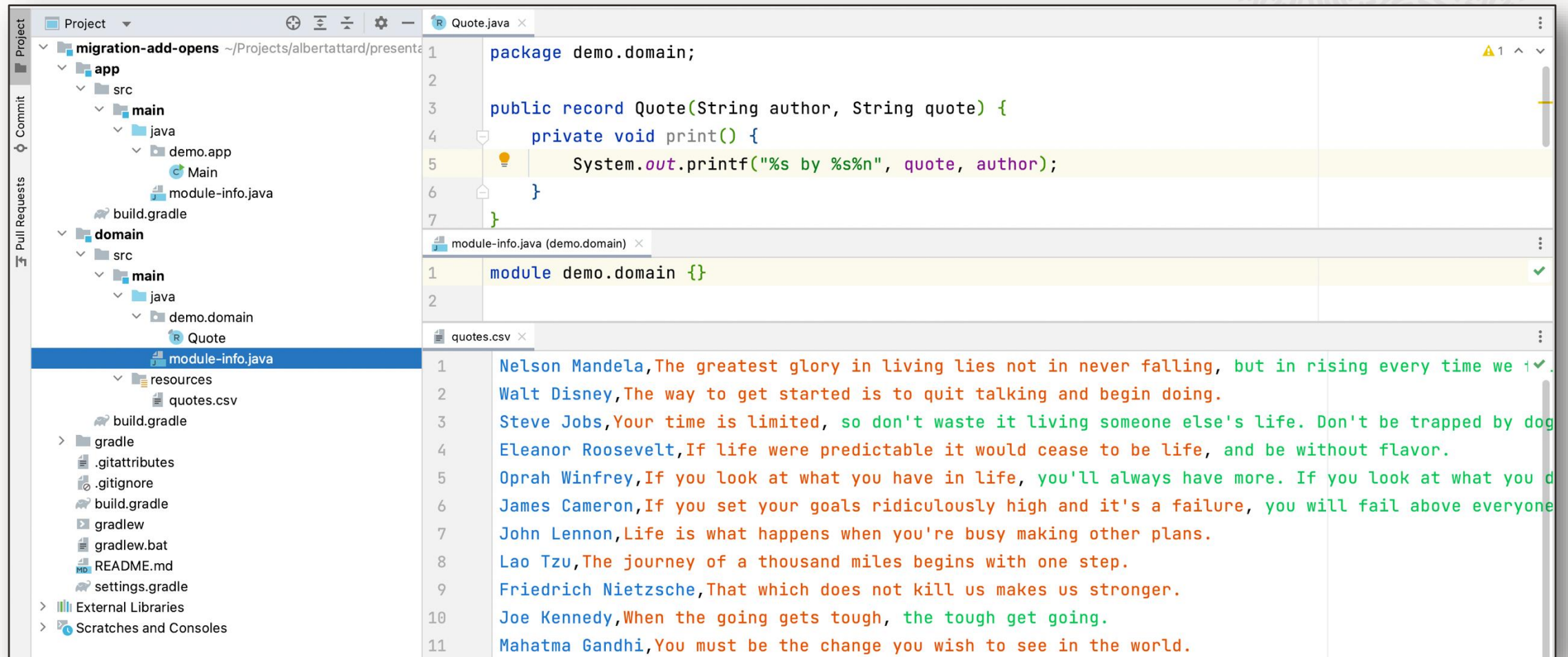


domain

```
module demo.domain { }
```

Application

The *demo.domain* module



Application

The *demo.app* module

The screenshot displays an IDE interface with a project explorer on the left and a code editor on the right. The project explorer shows a hierarchy starting with 'migration-add-opens', followed by 'app', 'src', 'main', 'java', 'demo.app', and 'Main'. Below this, 'module-info.java' is listed. The code editor shows the content of 'Main.java', which includes imports for reflection and charset, and a 'Main' class with a 'main' method. The 'main' method uses reflection to load the 'demo.domain.Quote' class and its 'print' method, then reads from a resource file 'quotes.csv' and invokes the 'print' method on the loaded object. A second tab at the bottom shows the 'module-info.java' file for the 'demo.app' module, which is currently empty.

```
8 import java.lang.reflect.InvocationTargetException;
9 import java.lang.reflect.Method;
10 import java.nio.charset.StandardCharsets;
11
12 public class Main {
13     public static void main(final String[] args) throws ClassNotFoundException, NoSuchMethodException,
14         final Class<?> type = Class.forName( className: "demo.domain.Quote");
15         final Constructor<?> constructor = type.getConstructor(String.class, String.class);
16         final Method print = type.getDeclaredMethod( name: "print");
17         print.setAccessible(true);
18
19         try (InputStream in = type.getResourceAsStream( name: "/quotes.csv");
20             BufferedReader reader = new BufferedReader(new InputStreamReader(in, StandardCharsets.UTF
21         for (String line; (line = reader.readLine()) != null; ) {
22             String[] parts = line.split( regex: ",", limit: 2);
23             final Object instance = constructor.newInstance(parts[0], parts[1]);
24             print.invoke(instance);
25         }
```

```
1 module demo.app {}
2
```

Describe module

```
> jar --describe-module --file domain.jar  
  
demo.domain  
jar:file:///.../domain.jar!/module-  
info.class  
requires java.base mandated
```

There is no
exports demo.domain!

Describe module

```
> jar --describe-module --file app.jar
```

```
demo.app jar:file:///.../app.jar!/module-  
info.class
```

```
requires java.base mandated
```

```
contains demo.app
```

```
main-class demo.app.Main
```

There is no
requires demo.domain!

Running the application

Using the module path

```
> java \  
  --module-path app.jar:domain.jar \  
  --module demo.app
```

```
Exception in thread "main"  
java.lang.ClassNotFoundException:  
demo.domain.Quote ...
```

Running the application

Using the module path and *--add-opens*

```
> java \  
  --module-path app.jar:domain.jar \  
  --add-opens  
demo.domain/demo.domain=demo.app \  
  --module demo.app
```

Running the application

Using the module path and *--add-opens*

```
WARNING: Unknown module: demo.domain  
specified to --add-opens  
Exception in thread "main"  
java.lang.ClassNotFoundException:  
demo.domain.Quote ...
```

Running the application

Using the module path and *--add-opens* and *--add-modules*

```
> java \  
  --module-path app.jar:domain.jar \  
  --add-modules demo.domain \  
  --add-opens  
demo.domain/demo.domain=demo.app \  
  --module demo.app
```

Works!! (*Output omitted for brevity*)

Running the application

Using the module path and *--add-modules* (without and *--add-opens*)

```
> java \  
  --module-path app.jar:domain.jar \  
  --add-modules demo.domain \  
  --module demo.app
```

Running the application

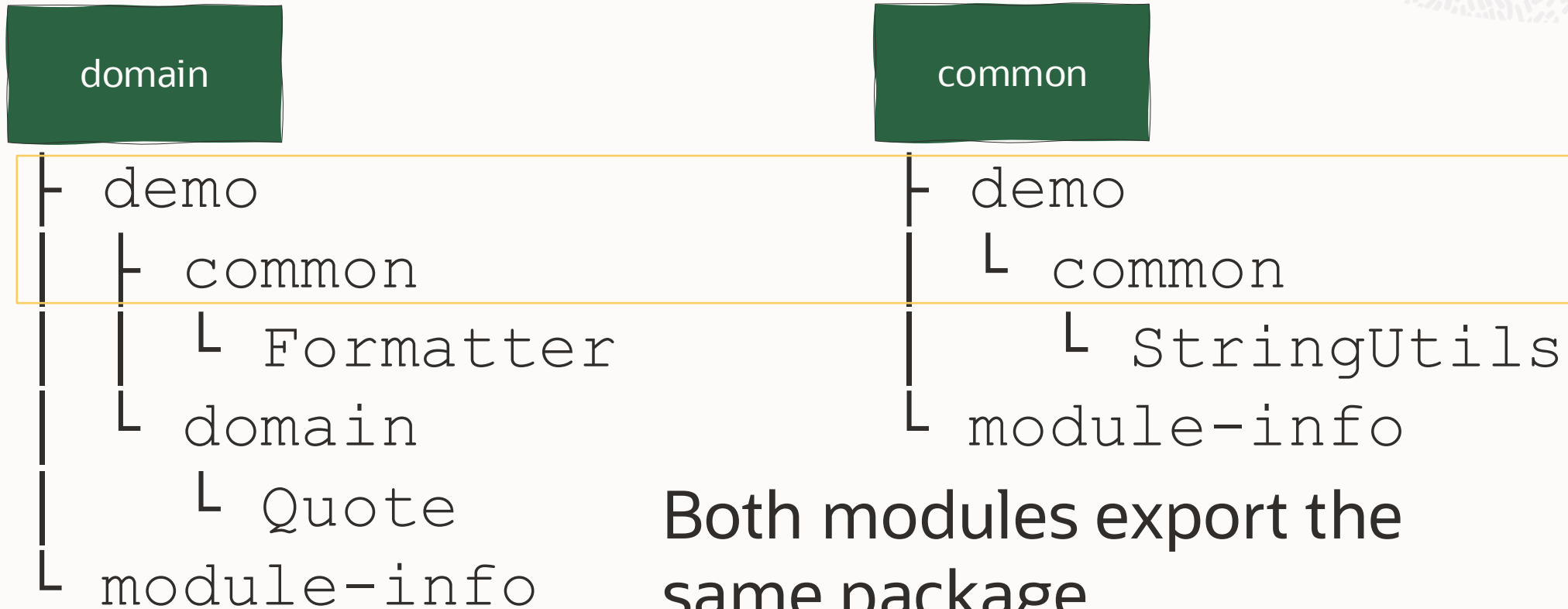
Using the module path and *--add-modules* (without and *--add-opens*)

```
Exception in thread "main"  
java.lang.reflect.InaccessibleObjectExcept  
ion: Unable to make private void  
demo.domain.Quote.print() accessible:  
module demo.domain does not "opens  
demo.domain" to module demo.app
```

Can two *named* (or *automatic*) modules with the same package be used together at runtime?

—
--patch-module

Application



Both modules export the
same package
`demo.common`.

Application

The screenshot displays an IDE interface for a multi-module Java application. The left sidebar shows the project structure:

- Project: migration-patch-module (~/Projects/albertatta)
 - app
 - src
 - main
 - java
 - demo.app
 - Main
 - common
 - src
 - main
 - java
 - demo.common
 - StringUtils
 - module-info.java
 - domain
 - src
 - main
 - java
 - demo
 - common
 - Formatter
 - domain
 - Quote
 - gradle
 - .gitattributes
 - .gitignore

The main editor shows the following code files:

```
1 package demo.app;
2
3 import demo.common.Formatter;
4 import demo.common.StringUtils;
5 import demo.domain.Quote;
6
7 public class Main {
8     public static void main(final String[] args) {
9         final String author = StringUtils.toUpperCase(value: "Albert Einstein");
10        final Quote quote = new Quote(author, quote: "Learn from yesterday, live for today, hope for tomorrow");
11        System.out.println(Formatter.multiline(quote));
12    }
13 }
```

```
1 module demo.common {
2     exports demo.common;
3 }
4
```

```
1 module demo.domain {
2     exports demo.common;
3     exports demo.domain;
4 }
5
```

Describe module

```
> jar --describe-module --file domain.jar
```

```
demo.domain
```

```
jar:file:///.../domain.jar!/module-  
info.class
```

```
exports demo.common
```

```
exports demo.domain
```

```
requires java.base mandated
```

Describe module

```
> jar --describe-module --file common.jar
```

```
demo.common
```

```
jar:file:///.../common.jar!/module-  
info.class
```

```
exports demo.common
```

```
requires java.base mandated
```

Running the application

Using the module path and *--add-modules*

```
> java --module-path libs \  
    --add-modules demo.common,demo.domain \  
    --module demo.app
```

```
Error occurred during initialization of  
boot layer
```

```
java.lang.module.ResolutionException:  
Modules demo.common and demo.domain export  
package demo.common to module demo.app
```

Running the application

Using the module path and *--add-modules* and *--patch-module*

```
> java --module-path libs \  
    --add-modules demo.domain \  
    --patch-module demo.domain=common.jar \  
    --module demo.app
```

```
WARNING: module-info.class ignored in  
patch: common.jar
```

```
...
```

Running the application

Using the module path and *--add-modules* and *--patch-module*

```
WARNING: module-info.class ignored in
patch: ./build/libs/domain.jar
Exception in thread "main"
java.lang.IllegalAccessError: class
demo.app.Main (in module demo.app) cannot
access class demo.domain.Quote (in module
demo.common) because module demo.common
does not export demo.domain to module
demo.app
```

Running the application

Using the module path and *--add-modules* and *--patch-module* and *--add-exports*

```
> java --module-path libs \  
    --add-modules demo.common \  
    --patch-module demo.common=domain.jar \  
    --add-exports  
demo.common/demo.domain=demo.app \  
    --module demo.app
```

```
WARNING: module-info.class ignored in  
patch: ./build/libs/domain.jar
```

Next



Proof of concept

- **Identify** an application
- **Try** migrating it



How can we help

- Ad hoc conversations
- Workshops
- Knowledge Transfer
- Pairing sessions
- Hands-on



We are here to help

You are not alone
Oracle is here to
help



Thank you

Albert Attard

Principal Java Success Consultant

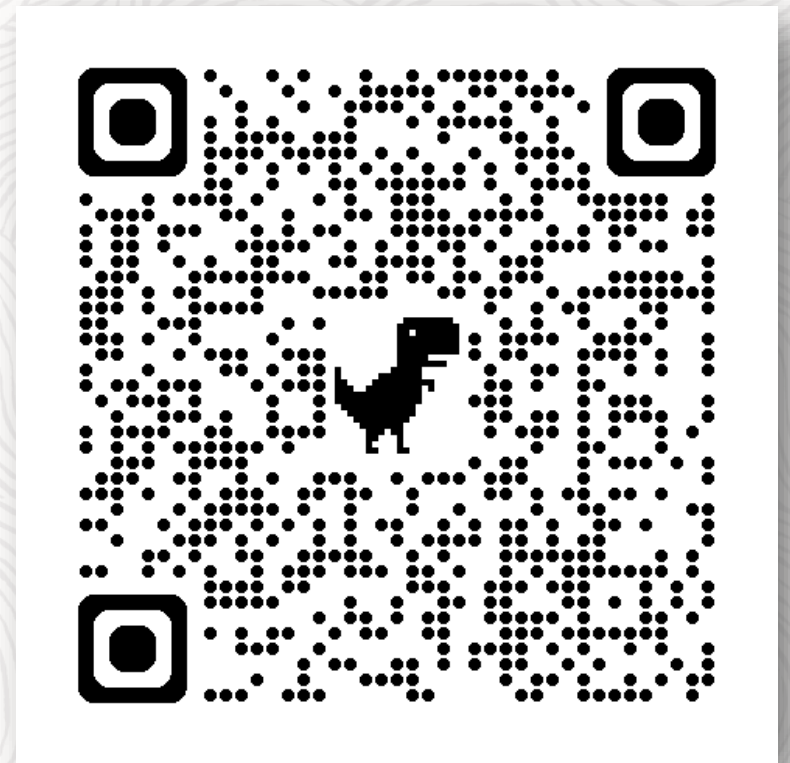
albert.attard@oracle.com



Your feedback is invaluable to us! Please share your thoughts through our [brief survey](#) or by email:

albert.attard@oracle.com.

Thank you for helping us improve!



ORACLE

Our mission is to help people see
data in new ways, discover insights,
unlock endless possibilities.

